



Web UI Component Specification

A comprehensive web-first reference for UI component libraries

Sebastián Gamboa

March 2026

v 1.0

Table of Contents

1 Introduction	5
1.1 What Is This Specification	6
1.2 The Problem We're Solving	7
1.3 When to Use This Specification	8
1.4 When NOT to Rebuild the Wheel	10
1.5 Importance of Having a Framework-Neutral Specification	11
1.6 Target Audiences	12
1.7 How to Use This Specification	13
1.8 A Note on Non-Web Platforms	15
2 Core Principles	16
2.1 Component Philosophy	16
2.2 Design Tokens	20
2.3 Interaction Principles	27
2.4 Consistency Patterns	29
3 Component Catalog	32
3.1 How to Navigate This Catalog	32
3.2 Components Listed by Functional Category	33
3.3 Components Listed by Complexity Tier	39
3.4 Component Specifications	42
Specification Structure	42
#1 Icon	45
#2 Typography	47
#3 - KBD (Keyboard Key)	51
#4 Code Snippet	53
#5 Separator / Divider	57
#6 Spacer	60
#7 Aspect Ratio	62
#8 Image	65

#9 Avatar	68
#10 Badge	71
#11 Chip / Tag	74
#12 Skeleton	78
#13 Loader	82
#14 Color Swatch	85
#15 Link	88
#16 Button	92
#17 Label	98
#18 Form Field	101
#19 Text Input	104
#20 Password Input	108
#21 PIN / OTP Input	111
#22 Textarea	115
#23 Checkbox	119
#24 Radio Button	123
#25 Switch	128
#26 Segmented Control	133
#27 Range Slider	136
#28 Rating	140
#29 File Uploader	143
#30 Color Picker	147
#31 Card	150
#32 Alert / Banner	155
#33 Progress	159
#34 Scroll Area	163
#35 Collapsible	166
#36 Resizable	169
#37 Empty State	172
#38 Breadcrumb	174
#39 Menu	177
#40 Select	181
#41 Combobox / Autocomplete	187
#42 Dropdown Menu	193

#43 Tooltip	197
#44 Popover	201
#45 Toast / Notification	206
#46 Modal / Dialog	210
#47 Drawer	215
#48 Adaptive Menu	220
#49 Date Picker	224
#50 Time Picker	229
#51 DateTime Picker	233
#52 Calendar	236
#53 Tabs	240
#54 Accordion	245
#55 Stepper	249
#56 Pagination	254
#57 Table	258
#58 Sortable List	263
#59 Transfer List	267
#60 Carousel	271
#61 Masonry	276
#62 Timeline	280
#63 Navigation / Menu Bar	284
#64 Sidebar	289
#65 Dock	295
3.5 Component Summary Checklist	300
4 Building Your Web UI System: Step-by-Step Guide	304
4.1 Step 1: Define Your Color Palette	305
4.2 Step 2: Define Your Typography System	307
4.3 Step 3: Define Spacing and Layout System	308
4.4 Step 4: Define Look & Feel Decisions	311
4.5 Step 5: Set Up Development Environment	314
4.6 Step 6: Implement Design Tokens in Code	316
4.7 Step 7: Build Foundational Components	318
4.8 Step 8: Build Compound Components	319
4.9 Step 9: Ensure Test Coverage and Quality	321

4.10 Step 10: Create Documentation and Usage Guidelines.....	324
4.11 Step 11: Gather Feedback and Iterate	326
5 Conclusion	331
6 References.....	333

1 Introduction

Have you ever sat down to build a component — a dropdown, a modal, a form input — and told yourself: *this time I'm doing it properly*? You mapped the edge cases, handled keyboard navigation, wired up the error states, wrote the tests. It felt solid. It felt done. Then six months later, the team migrated to a new framework, or a new project started with a different stack, and you found yourself rebuilding the same thing from scratch. Not because the requirements had changed — they hadn't. The dropdown still needed to close on Escape. The input still needed to announce errors to screen readers. The modal still needed to trap focus. The *what* was identical. Only the *how* changed.

This is one of the quiet, persistent frustrations of UI development. Every time the technology shifts, we don't just carry our code forward — we're expected to re-derive our understanding of the problem, reinvent our solutions, and re-encode years of accumulated UX knowledge into a new set of files. The knowledge itself — the careful thinking about states, interactions, edge cases, accessibility — rarely survives the migration. It's embedded in the implementation, not captured separately, and so it disappears with it.

What makes this particularly costly is that UI component knowledge is genuinely hard to build. It's not just about code. It's about understanding how users actually interact with interfaces: how they navigate with keyboards, how screen readers announce state changes, how error messages should be timed and positioned to help rather than frustrate. Getting this right takes experience, testing, and iteration. Losing it to a stack migration is an avoidable waste.

There's a second shift making this problem more urgent. The way software gets built is changing. AI systems — large language models, coding assistants, agentic tools — are increasingly participating in the development process. They can generate component code, write tests, suggest implementations. But they can only do this well if they have clear, unambiguous behavioral requirements to work from. A well-structured specification of what a component should *do* is exactly what an AI

system needs to produce a correct, accessible, framework-appropriate result. The human-AI development partnership is becoming a real working dynamic, and it demands a different kind of documentation — one that captures intent, not just implementation.

This specification is the answer to both problems. It captures the behavioral requirements for common web UI components in a way that survives technology changes, that can be handed to a new developer or a language model, and that serves as a stable source of truth across the entire lifecycle of a product.

1.1 What Is This Specification

This document is a **web-first specification** for UI component libraries. It describes the expected behavior, features, and interaction patterns of common UI components from a user experience perspective, written primarily for teams building web-based interfaces with HTML, CSS, and JavaScript.

At its core, this is a **functional blueprint** — a specification that defines what components should do, not how they should be built. It covers everything from the simplest stateless elements (icons, dividers, typography) to the most complex interactive patterns (modals, date pickers, data tables), each specified with the same consistent structure: purpose, main features, secondary features, test scenarios, and implementation notes.

The specification is intentionally technology-neutral within the web ecosystem. It doesn't care whether you're using React, Vue, Svelte, Angular, or vanilla JavaScript. It doesn't prescribe a CSS methodology, a state management library, or a build tool. What it does care about is that a modal traps focus correctly, that a form input announces its error state to screen readers, that a dropdown closes when Escape is pressed — behaviors that are true regardless of how the underlying code is written.

This makes the specification useful in several distinct ways. For a developer starting a new library, it's a complete set of requirements to implement against. For a team migrating to a new stack, it's a continuity layer that ensures nothing gets lost in

translation. For a QA engineer, it's a test specification. For an AI assistant, it's a structured prompt that produces correct, accessible component implementations. For a designer, it's a behavioral contract that ensures the components their designs depend on will work as expected.

The specification is also designed to grow with a team. Not every project will need every component, and not every team will require every secondary feature listed in each spec. The specifications are modular — take what's relevant to your current phase, leave what isn't, and return to the rest as your needs expand.

1.2 The Problem We're Solving

If you've worked on multiple web projects across your career, you've likely experienced this: building the same text input component over and over again. First with plain CSS, then with Sass, then with styled-components, then with Tailwind, then with a different framework entirely. Each time, you're encoding the same UX knowledge — inputs should show error states, handle focus, provide clear feedback — into a different technical implementation.

The wheel keeps getting rebuilt, but the wheel's requirements rarely change.

From a user experience perspective, UI components have stable, well-understood patterns. Users expect text inputs to behave a certain way. Modals should trap focus. Dropdowns should be keyboard-navigable. These expectations don't change when you switch from React to Vue, or from one CSS methodology to another. The user sitting in front of the interface has no idea which framework rendered the button they just pressed. They only notice whether it works correctly.

Yet the way most teams build UI components treats this behavioral knowledge as a byproduct of the implementation. It lives in code comments, in the institutional memory of whoever wrote the original component, or nowhere at all. When the

implementation changes — and in web development, it always eventually changes — the knowledge has to be reconstructed from scratch. Senior engineers end up re-explaining focus trapping to new team members. Accessibility requirements get re-litigated every time a component is rewritten. The same mistakes get made in each iteration because there's no persistent record of what was learned.

The field has partial solutions. Design systems capture visual consistency — tokens, colors, spacing, aesthetics. Accessibility standards like WCAG provide authoritative guidance on what accessible components must do. But there's a gap between these two layers: **a comprehensive behavioral specification for common web UI patterns** that is detailed enough to implement from, testable enough to validate against, and stable enough to outlast any particular technology choice. That gap is what this specification fills.

By separating the behavioral specification from the implementation, we create something genuinely reusable — not just code that gets copied, but understanding that persists. A team can use this specification with React today, with a different framework in three years, and with AI-generated code in between, and the components that result will all meet the same behavioral standard. That consistency is the real value.

1.3 When to Use This Specification

Use this specification when:

- **Starting a new web UI library or design system from scratch.** Use the Component Catalog as your requirements document and the Step-by-Step Guide as your build process. You'll have a clear definition of done for every component before writing a line of code.
- **Rebuilding existing components in a new web technology stack.** The specifications describe what each component must do, not how it's implemented. Migrating from Vue to React doesn't change the fact that a

modal needs to trap focus — use this specification to ensure nothing gets missed in the transition.

- **Ensuring behavioral consistency across multiple web projects or teams.** When different teams build different parts of a product, components can drift — similar patterns implemented differently, edge cases handled inconsistently. This specification provides a shared reference that keeps implementations aligned regardless of who built them.
- **Training AI systems to generate UI components.** Language models produce significantly better component code when given explicit behavioral specifications. The structured format of this specification is designed to be consumable by AI coding assistants.
- **Onboarding developers who need to understand expected component behavior.** Instead of re-explaining from scratch what a well-built dropdown looks like, hand new team members this specification. It captures accumulated UX and accessibility knowledge in a form that's approachable for developers at any experience level.
- **Auditing existing components against standard web UX patterns.** Use the Test Scenarios in each component spec as a checklist. If a component in your existing library can't pass its corresponding scenarios, you've found a concrete gap worth addressing.

This specification is NOT:

- **A visual design system** — it doesn't specify colors, fonts, spacing values, or any visual styling. Those decisions belong to your design tokens and brand guidelines, which this specification complements but does not replace.
- **An implementation tutorial** — it describes what components should do, not how to write the code. It won't teach you React hooks or CSS grid; it will tell you what the component built with those tools must accomplish.

- **Opinionated about which web framework or toolchain to use** — React, Vue, Svelte, Angular, vanilla JS — all are valid. The specifications are written to be implementable in any of them.
- **A replacement for accessibility standards** — it references WCAG 2.2 (Campbell et al., 2023) and the ARIA Authoring Practices Guide (W3C WAI, n.d.) throughout, but for authoritative accessibility requirements, always consult those primary sources directly.
- **Exhaustive for every possible component variation** — it covers the foundational patterns that appear in most web applications. Highly specialized or domain-specific components are outside its scope.

1.4 When NOT to Rebuild the Wheel

Before investing in building a custom UI library, it's worth asking honestly whether that investment is justified. Custom libraries carry real costs: initial build time, ongoing maintenance, documentation burden, and the opportunity cost of not shipping product features instead. The decision to build should be deliberate.

- **Does an existing library already fit your needs?** If you're building a standard web application and a library like Material-UI, Chakra, or Radix already provides 90% of what you need — with solid accessibility and a good API — use it. Theming and extending existing libraries is almost always faster than building from scratch, and you inherit years of bug fixes and edge case handling for free. This specification can still serve as a behavioral reference for auditing or extending what you adopt.
- **Is your visual design truly custom?** Many teams convince themselves they need a custom library when what they actually need is a custom *theme*. Most mature component libraries expose design tokens and style overrides that can take you very far from the default appearance. Before committing to a build, explore what theming can achieve — you may be surprised how much is possible without writing a single component.

- **Do you have the resources to maintain it?** A UI library is not a project with a finish line — it's an ongoing commitment. Browsers ship new features and deprecate old behaviors. Accessibility standards evolve. Your product's component needs grow over time. The team members who built the original components move on. If your organization can't commit to sustained maintenance, a well-chosen third-party library with an active community is a significantly safer long-term bet.

Rebuild or build from scratch when:

- You're working in a constrained or specialized web environment — micro-frontends with strict isolation requirements, embedded widgets, performance-critical applications where bundle size is a hard constraint — where standard libraries introduce unacceptable tradeoffs.
- You need components or interaction patterns that don't exist in any standard library and are too specialized to contribute back to one.
- You're migrating to a fundamentally different web technology stack and the migration is a genuine opportunity to realign the library with current needs.
- You have very specific brand or UX requirements — not just visual, but behavioral — that cannot be achieved through theming or extension of existing libraries.

1.5 Importance of Having a Framework-Neutral Specification

Although this specification is web-first, it deliberately avoids prescribing *how* components should be built. This distinction — between behavioral specification and technical implementation — is the design decision that makes the specification durable.

The web framework landscape changes constantly. React was novel; now it's standard; something will eventually succeed it. CSS methodologies cycle through phases of enthusiasm and abandonment. Build tools get replaced. What doesn't

change is that users expect a modal to be dismissible with Escape, that a form error message needs to be associated with its input, that a navigation menu needs to be reachable by keyboard. These behavioral expectations are stable across years and across technology generations.

By keeping the specification framework-neutral, this document stays relevant across that change. A team that adopts it with React today can still use it when they migrate to a different stack in three years. The test scenarios don't change. The accessibility requirements don't change. The behavioral contract doesn't change. Only the implementation does.

This separation has additional concrete benefits:

1. **Longevity** — Behavioral expectations outlast any particular framework or tooling trend. This specification is designed to be as useful in five years as it is today.
2. **Flexibility** — Teams can apply the specification to their specific stack, constraints, and context without being forced into architectural decisions that don't fit their situation.
3. **Clarity** — Separating “what” from “how” makes both clearer. Developers can focus on implementation choices without conflating them with behavioral requirements, and reviewers can evaluate both dimensions independently.
4. **AI-friendliness** — Language models generate correct implementations in any web stack when given clear, unambiguous behavioral specs. The structured format of this specification is optimized for that use case.
5. **Cross-functional collaboration** — Designers, developers, QA engineers, and AI systems can all work from the same source of truth, without needing to understand each other's implementation details.

1.6 Target Audiences

This specification is designed to be useful across the different roles involved in building and maintaining a web UI system — from the engineers writing the code, to the designers shaping the interactions, to the AI systems increasingly

participating in the build process. The specifications are structured so that each audience finds exactly what they need: testable behaviors for developers, interaction contracts for designers, and unambiguous requirements for AI systems. The common thread is that everyone working on a UI system benefits from a shared, stable definition of how components are supposed to behave.

Developers — Each component specification provides a complete set of behavioral requirements that can be directly translated into code, regardless of framework. The Test Scenarios section doubles as a unit test specification: if you write tests that cover every scenario listed, you have meaningful coverage of the component's expected behavior. The Notes section captures implementation pitfalls and non-obvious decisions, saving time that would otherwise be spent discovering them through trial and error.

UX Designers — The specification grounds interaction patterns in established usability research (Nielsen, 1994) and web accessibility standards, giving designers a behavioral vocabulary that bridges the gap between design intent and implementation reality. Rather than describing how a dropdown should *look*, it describes how it should *respond* — what happens on keyboard interaction, how error states are communicated, what feedback a user receives after each action. This makes design reviews and developer handoffs significantly more precise.

AI Systems (LLMs) — The structured, consistent format of each specification — purpose, main features, secondary features, test scenarios, notes — is designed to be machine-readable as well as human-readable. An AI coding assistant given a component specification from this document can generate a correct, accessible, framework-appropriate implementation with substantially less ambiguity than a natural language description alone.

1.7 How to Use This Specification

There are multiple valid entry points into this specification, and the right one depends on what you're trying to accomplish. It isn't meant to be read cover to cover in a single sitting — it's a reference document as much as a narrative one, and

different roles will extract value from it in different ways. A developer building a new library might work through it sequentially. A designer auditing component behavior might jump directly to specific entries in the Component Catalog. An AI system might query individual specifications to build or validate components.

1. **Read Core Principles** to understand the foundational philosophy and design tokens. If you're building a new system from scratch, start here — these principles inform every component specification and should shape your architectural decisions before you write any component code.
2. **Browse the Component Catalog** to see behavioral specifications for all 65 components. Use the category and tier indexes to find what's relevant to your current work. You don't need to read every component — navigate to the ones you're building or auditing.
3. **Reference the Step-by-Step Guide** when building a new web UI system from scratch. It walks through the build process from design tokens to component testing to documentation, in a recommended sequence that minimizes rework.
4. **Extract what you need** — this specification covers more than most projects will use at any given time. Pick the components and features relevant to your current phase, and return to the rest as your library grows.

Example usage with an AI system:

"I'm building a design system for a SaaS web application using React, TypeScript, and Tailwind CSS. I've already defined my color tokens and typography scale. Using the Web UI Component Specification as the behavioral reference, implement the following Tier 1 and Tier 2 components with full test coverage for all Main Features and Accessibility requirements: Button (with Icon Button and Button Group variants), Text Input (with Email and Search specializations), Form Field, Label, Checkbox, Radio Button, and Modal. Skip Internationalization and touch-screen secondary features for now — we'll add those in a later pass."

1.8 A Note on Non-Web Platforms

This specification is written for the web. Concepts, examples, test scenarios, and implementation notes throughout use web vocabulary — HTML elements, CSS properties, ARIA roles, browser events, and JavaScript APIs are the assumed context.

That said, the **behavioral intent** of every specification translates beyond the browser. The principles of focus management, keyboard navigation, error states, and accessible semantics are universal UX concerns that apply wherever a UI is rendered. If you are building for a non-web platform — native mobile (iOS/Android), desktop (Electron, Tauri, native frameworks), or an embedded UI system — you can use this specification as a reference for *what* your components should do and reinterpret each specification in your platform’s terms:

Web concept	Platform reinterpretation
HTML semantic elements (<code><button></code> , <code><nav></code>)	Native widget or view types with equivalent roles
CSS classes / style props	Platform styling system (StyleSheet, SwiftUI modifiers, etc.)
ARIA attributes	Native accessibility APIs (contentDescription, accessibilityLabel, etc.)
<code>prefers-reduced-motion</code> media query	System-level reduced-motion / accessibility setting
44×44px touch target (CSS pixels)	Platform-equivalent minimum interactive area (pt, dp, etc.)
Browser focus and tab order	Platform focus ring and keyboard/controller navigation model

The component behaviors, test scenarios, and interaction principles remain valid across all these contexts — only the implementation vocabulary changes.

2 Core Principles

2.1 Component Philosophy

Good UI components are deceptively simple to use and surprisingly hard to build well. The gap between a component that *appears* to work and one that *actually* works — under keyboard navigation, with a screen reader, across edge cases, in the hands of a user who interacts with it differently than you expected — is where most of the real engineering effort lives. This specification takes a specific philosophical stance on how to close that gap, and understanding that stance is essential before reading any individual component specification.

The components described here follow a **presentational** approach. They are responsible for rendering visual state and communicating user interactions — nothing more. They do not contain business logic. They do not decide whether a form field's value is valid. They do not determine what happens when a button is clicked. These concerns belong to the layer above: the application, the form controller, the state management system. A component's job is to show what it's told and report what the user does.

This separation is not just an architectural preference — it's what makes components genuinely reusable. A component that embeds validation logic, or that makes assumptions about what should happen on submit, is only usable in the contexts it was designed for. A component that purely receives state and emits events can be used anywhere. It can be tested in isolation. It can be understood without knowing anything about the application it's embedded in. It can be handed to an AI system with a clear behavioral specification and implemented correctly the first time. The presentational philosophy is what makes this specification possible.

The five design principles that follow from this philosophy — simple and focused, state receivers, event emitters, composable, and flexible — are not independent rules. They reinforce each other. A component that receives state rather than

calculating it is naturally simpler. A component that emits events rather than handling them is naturally composable. Understanding them as a unified stance, rather than a checklist, is what produces components that feel coherent rather than assembled.

Simple and Focused

The first and most important constraint is simplicity. A component should do one thing well: render its current state visually and respond to user interactions. The moment a component starts making decisions — validating input, calculating what should happen next, managing application state — it becomes harder to test, harder to reuse, and harder to reason about. Keeping components “dumb” isn’t a limitation; it’s the quality that makes them composable across an entire system.

- Components handle visual states and user interactions
- They do NOT contain business logic or validation rules
- They are “dumb” in the best sense: predictable, testable, and reusable

State Receivers, Not State Calculators

Think of a component as a display, not a decision-maker. It should accept whatever state it’s given — an error flag, a loading indicator, a current value — and render it faithfully. The question of why a field is in an error state, or when it should be, belongs to the layer above. When a component tries to own that logic, it can only work correctly in the specific context it was built for. When it simply reflects state, it works everywhere.

- Components accept state from parent components or external systems (e.g., `hasError`, `isDisabled`, `value`)
- They display that state visually
- They do NOT determine what state they should be in

Event Emitters

Components communicate upward through events, not by acting on their own judgment. When a user types, the component fires `onChange`. When they leave the field, it fires `onBlur`. What happens next — whether to validate, navigate, submit, or do nothing — is the application's responsibility. This contract keeps components honest: they report faithfully, and the system decides. It also makes testing dramatically simpler, since you can verify a component's behavior by observing its events without needing to simulate an entire application around it.

- Components notify when user interactions occur (e.g., `onChange`, `onBlur`, `onFocus`, `onClick`)
- They do NOT decide what happens in response to those events
- External logic (form controllers, validation libraries, state management) handles the decisions

Composable and Extendable

A well-designed component is a starting point, not an endpoint. Base components should be buildable-upon — a `text-input` becomes an `email-input` by adding format validation feedback; it becomes a `search-input` by adding a clear button and search icon. Specializations inherit everything from their base and only describe what's different. This keeps the system from growing into dozens of one-off components that share no common lineage and drift apart over time.

- Base components can be extended to create specialized variants

e.g. `text-input` → `email-input`, `money-input`, `search-input`

- Specialized components inherit base behavior and add specific features

Flexible for Customization

Components will inevitably need to adapt to different contexts — different container widths, different surrounding layouts, different visual contexts. A good component allows this without requiring consumers to wrap it in extra `div`s or fight its

internals. The key is knowing what to protect and what to expose: visual properties like margin, max-width, and display can be freely overridden; behavioral and accessibility properties — focus management, ARIA attributes, keyboard handling — should be defended. Flexibility in service of reuse, not flexibility at the cost of correctness.

- Components should allow basic styling/positioning adjustments without requiring wrappers
- Accept standard web extension mechanisms (e.g., `className`, `style`, `data` attributes) while keeping core behavior intact
- Non-critical visual properties (margins, max-width, display) can be overridden by consumers
- Critical behavioral and accessibility properties should be protected/enforced
- Balance: Make common adjustments easy, but maintain component integrity

Example: Text Input Component

```
// Component receives state
<TextInput
  value="john@example"
  hasError={!!error}
  errorMessage={errorMessage}
  onChange={handleChange} // Component notifies of changes
  onBlur={handleBlur}      // Component notifies when user leaves field
/>

// External validation logic (NOT in component)
const handleBlur = (value) => {
  const isValid = validateEmail(value);
  setError(!isValid);
  setErrorMessage(isValid ? "" : "Invalid email format");
}
```

Note for non-web platforms: The prop names above use web/React conventions (`onChange`, `onBlur`, `className`). On other platforms, translate these to your framework's equivalents — the behavioral contract (receive state, emit events, avoid internal logic) remains the same regardless of syntax.

2.2 Design Tokens

Design tokens are the **foundational variables** that create visual consistency across your UI system. They are the bridge between design decisions and code — named, reusable values that encode choices like “the primary action color,” “the base spacing unit,” or “the duration of a standard transition,” in a way that components can consume without hardcoding specific values.

The importance of tokens extends well beyond convenience. Without them, visual decisions get scattered across a codebase as magic numbers and hardcoded hex values. When a brand color changes, or a spacing system is refined, updates become a search-and-replace exercise across dozens of files — error-prone and never quite complete. Tokens centralize those decisions in one place, so a single change propagates automatically to every component that references that token. This is what makes a UI system feel coherent and what makes it maintainable over time.

Tokens also serve as the vocabulary shared between design tools and code. When a designer specifies that a button background should use `color-primary-600`, and that same name exists in the codebase as a CSS Custom Property, the design intent translates directly without interpretation or loss. This shared language reduces the most common category of design-to-development friction: the gap between what was designed and what was built.

Design tokens should be defined before building any components. Components that are built without tokens become tightly coupled to specific values, making theming,

dark mode, and future design updates significantly harder. Defining tokens first is the investment that pays for itself repeatedly throughout the life of a UI system.

Color Palette

Color is the most immediate signal your UI sends. Before a user reads a label or processes a layout, they've already registered whether something looks like a primary action, an error, or a disabled state. A well-defined color palette makes those signals consistent and intentional — not just aesthetically, but semantically. The categories below aren't arbitrary groupings; they map to real communication roles that components will rely on throughout the system.

- **Primary colors:** Main brand colors used for key actions and emphasis
- **Secondary colors:** Supporting colors for variety and hierarchy
- **Semantic colors:** Colors with specific meanings
 - Success (typically green tones)
 - Error/Danger (typically red tones)
 - Warning (typically yellow/orange tones)
 - Info (typically blue tones)
- **Neutral colors:** Grayscale for text, backgrounds, borders, and dividers
- **State colors:** Focus indicators, hover states, active states

Considerations:

- Ensure sufficient contrast ratios for accessibility (WCAG 2.2 AA minimum: 4.5:1 for normal text, 3:1 for large text and UI components; Campbell et al., 2023, SC 1.4.3, 1.4.11)
- Test colors in both light and dark modes if applicable
- Consider color blindness scenarios (don't rely on color alone to convey information)

Light Mode and Dark Mode

Supporting both light and dark color schemes is no longer optional for a serious web UI system — it's an expectation. System-level dark mode preferences are now standard across macOS, Windows, iOS, and Android, and users notice when an

application ignores them. Beyond preference, dark mode has real accessibility implications: many users with light sensitivity, migraines, or visual impairments rely on it. Others simply spend long hours in low-light environments where a bright white interface causes genuine discomfort. Designing for both modes from the start is significantly easier than retrofitting dark mode onto a system built only for light — and the token-based approach described in this section is what makes it tractable.

Approach to Theming

Semantic Color Tokens

Instead of hardcoding specific color values, use semantic naming that can adapt to themes:

- `color-background` (white in light, dark gray in dark)
- `color-text-primary` (near-black in light, near-white in dark)
- `color-text-secondary` (gray in light, light gray in dark)
- `color-surface` (for cards/panels, adapts to theme)
- `color-border` (subtle in light, more prominent in dark)

Design Considerations for Dark Mode

- **Not just inverted colors:** Dark mode isn't simply inverting light mode — it requires careful adjustment
- **Reduced contrast:** Pure white (`#FFFFFF`) on pure black (`#000000`) causes eye strain; use off-white (`#E0E0E0`) on dark gray (`#121212`)
- **Elevated surfaces:** In dark mode, elevated surfaces (cards, modals) should be *lighter* than the background, not darker
- **Color saturation:** Reduce color saturation in dark mode for better readability (bright colors are harsh on dark backgrounds)
- **Shadows:** Shadows work differently in dark mode; consider using lighter borders or subtle glows instead

Web Implementation Strategies

1. **CSS Custom Properties:** Define theme tokens that change based on a `data-theme` attribute or a `.dark` class on the root element
2. **Preference detection:** Respect system preference with the `prefers-color-scheme` media query as the default
3. **User toggle:** Allow users to override the system preference via a theme switcher control
4. **Persistence:** Remember the user's theme choice across sessions (localStorage or a user profile setting)

Testing Dark Mode

- Test all components in both themes
- Verify contrast ratios in both modes
- Ensure images and icons work in both modes (some may need theme-specific versions)
- Check that focus indicators remain visible in both themes
- Test with actual users in different lighting conditions

Useful Resources

- Material Design Dark Theme guidelines (Google LLC, n.d.)
- Apple Human Interface Guidelines — Dark Mode (Apple Inc., n.d.)
- WCAG 2.2 contrast ratio requirements (Campbell et al., 2023)

Typography Scale

Typography is the invisible infrastructure of readability. Users don't notice a good type system — they just find the interface easy to read, scan, and navigate. They notice a bad one immediately, even if they can't articulate why. A type scale isn't just a list of font sizes; it's a set of relationships that create visual rhythm and hierarchy throughout the entire interface. Defined once as tokens, these

relationships apply consistently to every heading, paragraph, label, and code snippet across the entire product.

- **Font families:** Primary (body text), secondary (headings), monospace (code)
- **Font sizes:** Defined scale using CSS units (e.g., 12px, 14px, 16px, 20px, 24px, 32px, 48px)
- **Font weights:** Available weights (e.g., 400-regular, 500-medium, 600-semibold, 700-bold)
- **Line heights:** Corresponding line heights for each size (typically 1.2–1.8)
- **Letter spacing:** Adjustments for different sizes or use cases

Considerations

- Base font size should be at least 16px for body text to ensure readability at default browser zoom
- Ensure text remains readable at different browser zoom levels (up to 200% per WCAG)
- Consider font loading performance — each weight and style variant is a separate network request

Spacing System

Spacing is one of those things that's nearly invisible when done well and immediately obvious when done poorly. Inconsistent spacing — elements too close together, breathing room that varies unpredictably from component to component — makes an interface feel unfinished even when the individual elements are well-designed. A spacing system solves this by replacing arbitrary pixel decisions with a small set of named increments derived from a single base unit. Once defined, every gap, padding value, and margin in the system comes from that scale. The result is visual rhythm that holds together across the entire UI without anyone having to consciously enforce it.

- **Base unit:** Foundation for all spacing (commonly 4px or 8px)
- **Scale:** Consistent increments based on the base unit (e.g., 4, 8, 12, 16, 24, 32, 48, 64px)

- **Usage categories:**

- Gaps: Space between related elements
- Padding: Space inside containers
- Margins: Space outside components

Considerations

- Touch targets should be at least 44×44px (Apple Inc., n.d.) or 48×48dp (Google LLC, n.d.) for accessibility
- Maintain consistent rhythm and alignment across the interface

Border Radius

Border radius is a small decision that has a surprisingly large impact on the personality of a UI. Tight corners read as formal and precise; generous rounding reads as friendly and approachable. Neither is inherently better — but inconsistency between them is always a problem. A button with 4px corners next to a card with 12px corners next to a modal with 8px corners feels like it was assembled from three different design systems. Defining a small set of radius values upfront and applying them by role — not by component — keeps the system coherent.

- **Values:** Defined set of border radius values (e.g., 0, 2px, 4px, 8px, 16px, full/circle)
- **Usage:** Small (buttons, inputs), medium (cards), large (modals), full (pills, avatars)

Shadow/Elevation Levels

Shadows do more than add visual depth — they communicate hierarchy and interactivity. An element with a shadow sits above the page; it demands attention. A modal carries more shadow than a card because it's operating at a higher level of the interface. When shadows are applied consistently by elevation rather than by whim, users develop an intuitive sense of the interface's layer structure without ever consciously thinking about it. Define your shadow levels once, assign them to

elevation roles, and every component that uses them inherits a shared spatial language.

- **Levels:** Graduated shadow system to indicate elevation (e.g., level 1–5)
- **Usage:** Subtle (cards) to prominent (modals, dropdowns)
- **Consistency:** Same elevation = same shadow across all components

Breakpoints

Every component in this specification will be used across a range of screen sizes, from narrow mobile viewports to wide desktop displays. Breakpoints are the points at which the layout and behavior of components adapt to available space. Rather than treating responsive behavior as a component-by-component problem, defining breakpoints as named system tokens — mobile, tablet, desktop — creates a shared vocabulary that keeps responsive decisions consistent. A component that collapses at “mobile” width collapses at the same pixel value as every other component that does the same, because they’re all referencing the same token.

- **Mobile:** Small screens (e.g., < 640px)
- **Tablet:** Medium screens (e.g., 640px–1024px)
- **Desktop:** Large screens (e.g., > 1024px)
- **Additional:** Extra-large, or custom breakpoints as needed

Considerations

- Test components at breakpoint boundaries
- Design mobile-first, then progressively enhance for larger screens
- Ensure touch targets remain adequate at all viewport sizes

Transition/Animation Durations

Motion in a UI isn’t decoration — it’s communication. A transition that’s too fast feels abrupt; one that’s too slow feels sluggish; one that’s inconsistent between components feels broken. Defining a small set of named duration values — fast, normal, slow — and assigning them to specific types of interactions creates a motion language that users feel even if they don’t consciously register it. Hover feedback is

always fast. Modal entrances are always deliberate. State changes are always in the middle. Consistency in timing is what separates an interface that feels polished from one that just happens to have animations.

- **Fast:** Quick feedback (e.g., 100–150ms) for hovers, focus states
- **Normal:** Standard transitions (e.g., 200–300ms) for most UI state changes
- **Slow:** Deliberate animations (e.g., 400–500ms) for major layout or overlay changes
- **Easing:** Consistent CSS easing functions (ease-in, ease-out, ease-in-out, or custom cubic-bezier)

Considerations

- Always respect the `prefers-reduced-motion` media query — provide instant or minimal transitions for users who opt out of motion
- Animations should enhance, not impede, user interaction

2.3 Interaction Principles

Interaction principles are the behavioral commitments that a UI system makes to its users. They answer a fundamental question that every component must resolve dozens of times during implementation: *when something happens, what should the component do?* Without explicit principles, these decisions get made ad hoc — sometimes consistently, often not — and the result is an interface that feels unpredictable, because it is.

The six principles below are adapted from Nielsen’s 10 Usability Heuristics (Nielsen, 1994), one of the most empirically grounded frameworks in interaction design, and translated to the specific concerns of component-level behavior. They are not guidelines to apply when convenient — they are constraints to satisfy in every component, in every state, for every user. A component that violates these principles isn’t just inconsistent; it’s failing the people who depend on it.

Treating these as first-class requirements rather than aspirational guidelines changes how component development works in practice. It means that “does this provide clear feedback?” is a test condition, not a design suggestion. It means that “does this respect user control?” is a review criterion, not a preference. The Test Scenarios in the Component Catalog reflect this — they are written specifically to verify that each principle is upheld for each component.

1. Provide Clear Feedback

- Every user action should have an immediate, visible response
- Loading states should be indicated clearly
- Success and error states should be distinguishable
- Changes should be perceivable (visual, and where appropriate, announced to screen readers)

2. Maintain Consistent State Representation

- The same state should look the same across all components
- Focus states should be visually consistent
- Error states should follow the same pattern
- Disabled components should be clearly distinguishable from enabled ones

3. Respect User Control

- Users should be able to dismiss, cancel, or undo actions when appropriate
- No unexpected behavior (e.g., auto-submit, sudden navigation)
- Confirmations for destructive actions
- The Escape key should close overlays, modals, and dropdowns

4. Progressive Disclosure

- Show complexity only when needed
- Start simple, reveal advanced features on demand
- Don't overwhelm users with all options at once
- Provide sensible defaults

5. Preserve User Input

- Don't lose user data unexpectedly
- Warn before discarding unsaved changes
- Maintain form state when appropriate

6. Use Motion Purposefully

- Animations should communicate meaning and guide attention, not serve as decoration
- Purposeful motion examples:
 - Slide-in from right suggests forward progression or new content
 - Slide-out to left suggests backward navigation or dismissal
 - Fade-in suggests appearance or reveal
 - Shake/wiggle suggests error or rejection
 - Scale-up suggests emphasis or selection
 - Collapse/expand shows relationship between summary and detail
- Avoid gratuitous motion: Every animation should have a purpose (draw attention, show relationship, indicate state change)
- Respect user preferences: Always honor `prefers-reduced-motion` for users who find motion distracting or disorienting
- Keep it subtle: Animations should enhance understanding, not slow down interaction

2.4 Consistency Patterns

One of the quieter benefits of a well-built component system is that developers stop making micro-decisions. Not because the decisions don't matter — they do — but because they've already been made, once, and encoded into patterns that apply everywhere. When every form input uses the same prop name for its error state, you don't have to look it up. When every interactive element responds to focus the same way, you don't have to test it separately. When every component that can be disabled looks disabled in the same way, users build accurate mental models without extra effort. The patterns below are the shared conventions that make this

possible. They aren't constraints for their own sake — they're the accumulated decisions that prevent a system from fragmenting into a collection of individually reasonable but collectively inconsistent components. Adopt them early, and they pay for themselves every time a new component is built or a new developer joins the team.

State Naming Conventions

Use consistent naming for common states across all components:

- `isDisabled / disabled`: Component cannot be interacted with
- `isLoading / loading`: Component is performing an async action
- `hasError / error`: Component is in an error state
- `isRequired / required`: Input is mandatory
- `isReadOnly / readOnly`: Value is visible but not editable
- `isActive / active`: Component is currently selected or in use

Common Props/Parameters

Similar components should accept similar parameters:

- All form inputs: `value`, `onChange`, `onBlur`, `onFocus`, `disabled`, `required`
- All interactive elements: `onClick`, `disabled`, `aria-label`
- All components with children: `children` or `content`

Visual Feedback Patterns

- Focus: Same focus indicator style across all interactive elements (outline, ring, or border change)
- Hover: Consistent hover behavior (color shift, opacity change, or elevation increase)
- Error: Same visual treatment everywhere (border color, icon, message position)
- Success: Same visual treatment across all components
- Disabled: Consistent reduced opacity or muted color to indicate non-interactive state

Accessibility Patterns

Accessibility is not a feature to be added at the end — it's a dimension of correctness that every component either satisfies or fails. A component that a keyboard user can't operate isn't less accessible; it's broken for that user. A component that a screen reader misrepresents isn't imperfect; it's unusable for the people who depend on that technology. This framing matters because it changes how accessibility requirements are prioritized: not as a secondary checklist, but as part of the definition of done.

The accessibility requirements throughout this specification follow WCAG 2.2 (Campbell et al., 2023) and the ARIA Authoring Practices Guide (W3C WAI, n.d.) — the two primary standards that define what accessible web components must do. These standards exist because access to information and functionality matters, and because the web's strength is its universality. A UI system that works only for users without disabilities, or only for users with a mouse, is not meeting the bar this specification sets.

The patterns below summarize the non-negotiable baseline that applies to every component in the Component Catalog. They are a starting point, not a ceiling:

- All interactive elements are keyboard accessible — every action reachable by mouse must be reachable by keyboard
- Focus order follows logical reading order, matching the visual and DOM structure
- Labels are programmatically associated with their inputs via `for/id` or `aria-labelledby` — never implied by visual proximity alone
- ARIA roles, states, and properties are used correctly and consistently — not added speculatively, but applied according to the patterns defined in the ARIA APG
- Color is never the only means of conveying information — icons, text labels, or patterns always accompany color-based signals so that color-blind users receive the same information

3 Component Catalog

This catalog is the core reference of this specification — the place where principles meet practice. It documents the behavioral requirements for each UI component in the system: what it must do, how it must respond, what it must communicate to assistive technologies, and what a correct implementation looks like when tested. Each entry follows a consistent structure covering purpose, main features, secondary features organized by concern, test scenarios, and implementation notes. Together, they form a library of accumulated UX and accessibility knowledge that any developer, designer, or AI system can work from directly.

3.1 How to Navigate This Catalog

The catalog is large by design — a comprehensive reference isn't meant to be read front to back in one sitting. Most readers will come to it with a specific need: a component to implement, a behavior to verify, a decision to make. The three entry points below are organized to match those different modes of use. Whether you're planning a build, looking up a specific component, or trying to understand how complexity accumulates across a system, there's a path through the catalog that serves you.

By Functional Category: Components grouped by their primary purpose (form inputs, navigation, feedback, etc.). Useful when you need a specific type of component.

By Complexity Tier: Components ordered from basic to advanced. Useful when building a library from scratch—start with basic components that others depend on.

Component Specifications: Detailed behavioral specifications for each component, including specialized variants.

3.2 Components Listed by Functional Category

Grouping components by what they do — rather than how complex they are to build — is the most natural way to explore the catalog when you're looking for a specific type of functionality. The ten categories below cover the full range of UI concerns, from the most elemental content primitives to sophisticated data visualization and navigation patterns. They're ordered from foundational to complex: the earlier categories tend to be prerequisites for the later ones, both technically and conceptually. If you're new to the catalog, reading through the category descriptions alone gives you a useful map of the system before diving into individual specifications.

1. Content & Typography

The most fundamental building blocks for presenting textual and visual content. These components have no interactivity of their own and focus entirely on rendering information clearly and accessibly. They form the baseline visual language of any UI system — everything from page headings to inline code to images relies on these elements being well-defined before anything else is built. Establishing these components early ensures typographic consistency throughout the entire library.

Components:

- Icon (1)
- Typography (2) (*Heading, Body Text, Caption, Label Text, Code*)
- KBD (3)
- Code Snippet (4)
- Image (8)
- Color Swatch (14)

2. Layout & Structure

Components responsible for organizing and positioning other components on the page. They are invisible or structural in nature — they don't carry content themselves but determine how content is arranged spatially. A consistent layout system ensures visual rhythm, predictable spacing, and adaptability across screen sizes. These components are prerequisites for building any meaningful interface and underpin the structure of every page and view.

Components:

- Separator / Divider (5)
- Spacer (6)
- Aspect Ratio (7)
- Card (31) (*Multirow Card, Multirow Card List*)
- Scroll Area (34)
- Collapsible (35)
- Resizable (36)
- Accordion (54)

3. Identity & Status Indicators

Small, focused components that represent identity, state, or status at a glance. They are typically not interactive on their own but enhance other components with contextual meaning — a badge on a button signals unread notifications, a skeleton tells the user content is loading, an avatar identifies a person. These components are foundational to building interfaces that communicate system status without requiring user action, and they appear across virtually every category of the UI.

Components:

- Avatar (9) (*Avatar Group*)
- Badge (10)
- Chip / Tag (11) (*Chip Group*)
- Skeleton (12) (*Skeleton Layout*)
- Loader (13) (*Circular Loader*)

4. Actions & Navigation Primitives

The core interactive primitives through which users initiate actions and move through an application. Buttons and links are the most frequently used interactive elements in any UI. Getting these right — including all their states, variants, and accessibility requirements — is essential before building any more complex interactive component. These primitives are consumed directly by users and also composed into virtually every other interactive component in the library.

Components:

- Link (15)
- Button (16) (*Icon Button, Close Button, Circular Button, Button Group*)

5. Form Inputs & Controls

Components for collecting structured data from users. This is the most extensive category because data input is one of the primary functions of most applications. It includes everything from simple text fields and checkboxes to complex date/time pickers and color selectors. All form inputs share common concerns: clear labeling, validation feedback, accessible error handling, and keyboard operability. The Form Field wrapper ties these concerns together into a reusable, consistent pattern for any stack or platform.

Components:

- Label (17)
- Form Field (18)
- Text Input (19) (*Email Input, Number Input, Money Input, Search Input*)
- Password Input (20)
- PIN / OTP Input (21)
- Textarea (22)
- Checkbox (23) (*Checkbox Group*)
- Radio Button (24) (*Radio Group*)
- Switch (25) (*Two-Way Button, Toggle Group*)
- Segmented Control (26)

- Range Slider (27)
- Rating (28)
- File Uploader (29)
- Color Picker (30)
- Select (40) (*Multi-Select, Searchable Select, Grouped Select*)
- Combobox / Autocomplete (41) (*Search Bar, Autocomplete*)
- Date Picker (49) (*Date Range Picker*)
- Time Picker (50) (*Time Range Picker*)
- DateTime Picker (51)

6. Feedback & Communication

Components that communicate system status, outcomes, and contextual information to the user without necessarily requiring a response. These range from persistent alerts and banners that demand attention, to subtle tooltips that appear on hover, to temporary toasts that confirm an action completed. A well-designed feedback system reassures users that the interface is responding to their actions and keeps them informed without overwhelming them. These components are critical for perceived responsiveness and user trust.

Components:

- Alert / Banner (32) (*Inline Alert, Actionable Alert, Banner*)
- Progress (33) (*Progress Bar, Progress Circle, Indeterminate Progress*)
- Empty State (37)
- Tooltip (43)
- Toast / Notification (45)

7. Overlays & Floating Elements

Components that render above the main content layer, either anchored to a trigger element or centered in the viewport. They introduce an additional interaction layer and require careful management of focus, z-index stacking, and dismissal behavior. These components vary from non-blocking (dropdown, popover) to fully blocking (modal), and from ephemeral (tooltip) to persistent until dismissed (modal,

drawer). Understanding the distinction between each pattern is critical for correct usage and a consistent user experience.

Components:

- Dropdown Menu (42) (*Context Menu*)
- Popover (44)
- Modal / Dialog (46) (*Prompt / Confirmation Dialog*)
- Drawer (47) (*Side Drawer, Bottom Drawer*)

8. Adaptive Elements

Components whose rendering or behavior adapts based on context, device type, or available space. Rather than exposing separate implementations for desktop and mobile, adaptive components present a unified interface that intelligently selects the most appropriate pattern for the environment. This category is intentionally small but conceptually important — it represents the bridge between platform-specific UX conventions and a unified component API, and is particularly relevant for applications targeting multiple device types simultaneously.

Components:

- Adaptive Menu (48)

9. Navigation & Wayfinding

Components that help users understand where they are and move through the application's structure at a macro level. Navigation components operate at the application or page level rather than within a single feature. They include primary navigation bars, sidebars, tabbed interfaces, steppers for multi-step flows, and pagination for long content lists. These are typically among the last components built because they depend on many foundational and intermediate components, and because their design is closely tied to the overall information architecture of the application.

Components:

- Breadcrumb (38)
- Menu (39)
- Tabs (53)
- Stepper (55) (*Horizontal Stepper, Vertical Stepper*)
- Pagination (56)
- Navigation / Menu Bar (63)
- Sidebar (64)
- Dock (65)

10. Data Display & Visualization

Components for presenting structured, relational, or sequential data in a meaningful visual format. Unlike basic content components (which display static text or images), data display components organize and contextualize larger datasets — tables sort and filter rows, timelines sequence events, carousels present items in a browsable collection. These components often have the highest complexity in terms of interaction and state management, particularly when data is dynamic or user-manipulable, and frequently depend on many other components to function correctly.

Components:

- Calendar (52)
- Table (57)
- Sortable List (58)
- Transfer List (59)
- Carousel (60)
- Masonry (61)
- Timeline (62)

3.3 Components Listed by Complexity Tier

If the functional category view tells you what a component does, the complexity tier view tells you when to build it. The three tiers below reflect how components depend on each other: foundational components come first because everything else is built on top of them. Skipping ahead — trying to build a Modal before you have a solid Button, or a Date Picker before you have a reliable Text Input — leads to rework when the foundations shift. Think of this ordering not as a rigid prescription, but as a dependency graph translated into a build sequence. Follow it when starting from scratch; adjust it when your project already has some pieces in place.

Tier 1 — Basic

These are the atoms of the system — stateless or near-stateless components with minimal internal logic and no dependencies on other components. They don't compose anything; they are what everything else composes from. Getting these right early pays dividends throughout the entire build, because their patterns, props, and accessibility behaviors will be inherited, referenced, and reused at every level above them.

- Icon (1)
- Typography (2)
- KBD (3)
- Code Snippet (4)
- Separator / Divider (5)
- Spacer (6)
- Aspect Ratio (7)
- Image (8)
- Avatar (9)
- Badge (10)
- Chip / Tag (11)
- Skeleton (12)

- Loader (13)
- Color Swatch (14)
- Link (15)
- Button (16)

Tier 2 — Intermediate

Intermediate components manage their own state, respond to user interaction, and often compose one or more Tier 1 components internally. This is where most of the application's data collection happens — form inputs, controls, and the structural containers that organize them. The complexity here is moderate but meaningful: each component introduces its own focus management, validation feedback, or interaction model that must integrate cleanly with the layer above.

- Label (17)
- Form Field (18)
- Text Input (19)
- Password Input (20)
- PIN / OTP Input (21)
- Textarea (22)
- Checkbox (23)
- Radio Button (24)
- Switch (25)
- Segmented Control (26)
- Range Slider (27)
- Rating (28)
- File Uploader (29)
- Color Picker (30)
- Card (31)
- Alert / Banner (32)
- Progress (33)
- Scroll Area (34)
- Collapsible (35)
- Resizable (36)
- Empty State (37)

- Breadcrumb (38)
- Menu (39)

Tier 3 — Advanced

Advanced components are where the real engineering depth lives. They require careful focus management, overlay positioning, keyboard interaction models that go well beyond Tab and Enter, and in many cases significant composition of components from the two tiers below. A Modal, a Date Picker, a Data Table — these are not hard because their concepts are complicated, but because their correct implementation demands that every detail be right simultaneously: accessibility, responsive behavior, keyboard navigation, animation, and state management all intersecting in a single component. The specifications in this tier are correspondingly thorough.

- Select (40)
- Combobox / Autocomplete (41)
- Dropdown Menu (42)
- Tooltip (43)
- Popover (44)
- Toast / Notification (45)
- Modal / Dialog (46)
- Drawer (47)
- Adaptive Menu (48)
- Date Picker (49)
- Time Picker (50)
- DateTime Picker (51)
- Calendar (52)
- Tabs (53)
- Accordion (54)
- Stepper (55)
- Pagination (56)
- Table (57)
- Sortable List (58)
- Transfer List (59)

- Carousel (60)
- Masonry (61)
- Timeline (62)
- Navigation / Menu Bar (63)
- Sidebar (64)
- Dock (65)

3.4 Component Specifications

This is the full specification library — the detailed behavioral reference for every component in the catalog. Each entry is written to be complete enough to implement from, precise enough to test against, and structured consistently enough that once you’ve read a few, the format becomes invisible and the content takes over. Entries range from compact (a Spacer or Divider needs very little specification) to extensive (a Table or Date Picker touches nearly every secondary feature category). In all cases, the goal is the same: to capture exactly what a correct implementation looks like, without prescribing how to build it.

Note on specializations: *When building or specifying a component and its specializations, treat them as a single unit. A specialization inherits all features of its base component and only documents what is added or different.*

Specification Structure

For components with full detailed specifications, the structure follows a consistent pattern to ensure comprehensive coverage of all behavioral requirements:

Description A clear explanation of what the component is and its primary use cases. This establishes the component’s role in the UI system and helps determine when to use it versus similar components.

Main Features The core, essential behaviors that define the component. These are the “must-have” features without which the component would not fulfill its purpose. Every implementation should include all main features.

Secondary Features Additional capabilities organized by cross-cutting concerns. Not all secondary feature categories apply to every component. For example:

- **Accessibility:** Screen reader support, ARIA attributes, keyboard focus indicators, programmatic associations
- **Keyboard Navigation:** Tab order, keyboard shortcuts, Enter/Escape handling, focus management
- **Touch-screen:** Touch target sizing, native keyboard invocation, touch-friendly spacing, gesture support
- **Responsive Behavior:** Sizing adaptations, text overflow handling, viewport-aware positioning
- **Internationalization:** RTL support, multi-byte characters, locale-appropriate formatting
- **Validation & Feedback:** Error states, success indicators, real-time feedback, interaction callbacks
- **Animation & Transitions:** Entrance/exit animations, state change transitions, motion preferences
- **Variants & Features:** Size variants, style variants, specialized configurations
- **Advanced Features:** Optional sophisticated capabilities like filtering, grouping, custom rendering

***Note:** Simpler components (like KBD, Spacer, Divider) may have few or no secondary features, while complex components (like Table, Modal, Date Picker) will have many. The absence of a secondary feature category simply means it doesn't apply to that particular component.*

Test Scenarios Concrete, testable behaviors that verify the component works correctly. These can be translated directly into unit tests or manual QA checklists. Each scenario describes an interaction and its expected outcome.

Notes Implementation guidance, platform-specific considerations, common pitfalls, related components, and decision points that depend on context. This section captures wisdom that doesn't fit neatly into features or tests.

#1 Icon

Display system for icons (SVG, font-based, or image-based). Handles sizing, color inheritance, and accessibility labeling.

Description

An icon is a small graphical symbol used to represent actions, objects, or concepts in a compact, recognizable form. Icons enhance usability by providing visual cues that complement or replace text labels. A well-designed icon system ensures consistent sizing, styling, and accessibility across all icons in the application.

Main Features

- **Multiple format support:** Handles SVG, font-based icons, or image files with a consistent API
- **Configurable sizing:** Accepts size props (small, medium, large, or custom dimensions)
- **Color inheritance:** Inherits text color from parent by default, can be overridden
- **Accessibility labeling:** Supports aria-label or aria-hidden depending on whether icon is decorative or semantic
- **Visual consistency:** All icons follow the same visual weight, corner radius, and style guidelines

Secondary Features

Accessibility

- **Decorative vs semantic distinction:** Icons can be marked as decorative (aria-hidden) or semantic (with aria-label)
- **Screen reader support:** Semantic icons have accessible labels that describe their meaning

- **Text alternatives:** When icon stands alone without adjacent text, includes clear aria-label
- **Redundant information:** When icon accompanies text, icon can be hidden from screen readers to avoid repetition

Responsive Behavior

- **Scalable rendering:** Icons remain crisp at all sizes (especially important for SVG)
- **Proportional sizing:** Maintains aspect ratio when scaled
- **Pixel-perfect alignment:** Aligns cleanly on pixel grid to avoid blurry rendering

Internationalization

- **Direction-sensitive icons:** Icons like arrows, back/forward, or directional chevrons can mirror for RTL languages
- **Cultural sensitivity:** Certain icons may need alternatives for different regions (e.g., mailbox styles vary by country)

Test Scenarios

- **Renders correctly:** Icon displays the correct graphic for the specified name
- **Size variations:** Icon renders at correct dimensions for small, medium, large, and custom size values
- **Color inheritance:** Icon inherits parent text color when no explicit color is provided
- **Color override:** When explicit color prop is provided, icon displays in that color
- **Decorative icon:** When marked decorative, icon has aria-hidden="true" attribute
- **Semantic icon:** When marked semantic, icon has appropriate aria-label attribute
- **Missing icon handling:** When icon name doesn't exist, displays fallback icon or error indicator

- **RTL mirroring:** Directional icons (arrows, chevrons) flip correctly in RTL mode when configured
- **Scaling quality:** SVG icons remain sharp at all sizes without pixelation
- **Accessibility audit:** Screen reader correctly ignores decorative icons and announces semantic icons

Notes

- **Icon libraries:** Popular libraries include Font Awesome, Material Icons, Heroicons, Lucide, Phosphor. Choose based on style consistency needs
- **SVG vs font icons:** SVG generally preferred for better accessibility, multi-color support, and easier customization. Font icons can be simpler for basic single-color needs
- **Icon naming conventions:** Use consistent, descriptive names (e.g., “user-circle”, “chevron-right”, “trash”) across the system
- **Bundle size:** Be mindful of including only needed icons to reduce bundle size. Consider tree-shaking or icon subsetting
- **Custom icons:** When adding custom icons to system, ensure they match the visual style (stroke width, corner radius, level of detail) of existing icons

#2 Typography

Text display components with semantic meaning and visual hierarchy.

Specializations: Heading (H1–H6), Body Text, Caption, Label Text, Code (inline)

Description

Typography components provide semantic, accessible text rendering with consistent visual hierarchy. They map to HTML semantic elements (h1-h6, p, span, code) while ensuring the visual appearance aligns with the design system’s type scale. Typography components are the foundation of all textual content and ensure readability, scannability, and proper document structure.

Main Features

- **Semantic HTML mapping:** Renders appropriate semantic elements (h1-h6 for headings, p for paragraphs, etc.)
- **Visual hierarchy:** Clear distinction between heading levels, body text, captions, and labels
- **Consistent styling:** Font family, size, weight, line height, and letter spacing follow design system tokens
- **Flexible variants:** Supports different weights (light, regular, medium, bold) and styles (italic, underline)
- **Color inheritance:** Inherits color from parent context by default, can be overridden

Secondary Features

Accessibility

- **Semantic structure:** Proper heading hierarchy (h1 > h2 > h3) for screen readers and document outline
- **Contrast compliance:** Text colors meet WCAG contrast requirements against backgrounds (minimum 4.5:1 for body, 3:1 for large text)
- **Readable line length:** Body text maintains optimal line length (45-75 characters) for readability
- **No all-caps for long text:** Avoids ALL CAPS for extended text as it's harder to read

Responsive Behavior

- **Fluid typography:** Font sizes scale appropriately across breakpoints (smaller on mobile, larger on desktop)
- **Readable minimum sizes:** Body text never goes below 16px on mobile to avoid forced zoom
- **Line height adjustments:** Heading line heights are tighter (1.2-1.3), body text more generous (1.5-1.6)

- **Viewport-based scaling:** Optional use of viewport units (vw, vh) for hero headings or large text

Internationalization

- **Multi-script support:** Handles Latin, Cyrillic, Arabic, CJK, and other character sets
- **Font fallbacks:** Specifies appropriate fallback fonts for scripts not covered by primary font
- **Line-breaking rules:** Respects language-specific line-breaking rules (e.g., no line breaks between certain Japanese characters)
- **Bi-directional text:** Properly renders RTL scripts like Arabic and Hebrew

Variants & Features

- **Heading hierarchy:** H1 (page title, largest), H2 (section), H3 (subsection), H4-H6 (lower-level headings)
- **Body text styles:** Paragraph, caption (smaller), label (form labels, UI text), lead paragraph (larger intro text)
- **Inline code:** Monospace text for code snippets with distinct background
- **Truncation options:** Single-line truncation with ellipsis, multi-line clamping
- **Alignment:** Left (default), center, right, justify

Test Scenarios

- **Heading renders correct level:** H1 component renders `<h1>` tag, H2 renders `<h2>`, etc.
- **Visual hierarchy:** H1 is visibly larger than H2, H2 larger than H3, etc.
- **Font properties applied:** Component displays correct font family, size, weight, line height per design tokens
- **Color inheritance:** Text inherits parent color when no explicit color prop is provided
- **Color override:** When color prop is provided, text displays in specified color

- **Semantic structure:** Heading hierarchy is logical (no skipping levels, e.g., h1 → h3)
- **Contrast compliance:** Text color has sufficient contrast against background (automated test with axe or similar)
- **Line length:** Body text containers respect max-width to maintain readable line length
- **Responsive scaling:** Font sizes adjust at breakpoints as defined in design tokens
- **Truncation:** When truncation is enabled, long text shows ellipsis (...) at cutoff point
- **RTL support:** Text aligns correctly (right-aligned) in RTL mode
- **Multi-byte characters:** Displays emoji, accented characters, and non-Latin scripts correctly
- **Inline code styling:** Code component displays with monospace font and distinct background

Notes

- **Don't skip heading levels:** Always maintain logical hierarchy (h1 → h2 → h3), don't skip from h1 to h3 for styling reasons
- **Separate semantics from styling:** If you need h3 styling on an h2 element, use CSS classes or style props rather than using the wrong semantic level
- **Body text default:** When in doubt, use body text component for general content
- **Accessibility over aesthetics:** If a design calls for very light gray text, verify it meets contrast requirements before implementing
- **Font loading:** Consider font-display: swap to prevent invisible text during font loading
- **Variable fonts:** Modern systems may use variable fonts for flexible weight/width without multiple font files

#3 - KBD (Keyboard Key)

Displays keyboard shortcut keys or key combinations with appropriate visual styling (e.g., `Ctrl + K`).

Description

The KBD component visually represents keyboard keys or keyboard shortcuts in a consistent, recognizable format. It helps users understand what keys to press for actions, commands, or navigation. Common uses include documentation, tooltips for toolbar buttons, help text, and onboarding tutorials.

Main Features

- **Distinct visual style:** Displays with styling that clearly indicates it's a keyboard key (typically bordered, slightly raised appearance)
- **Key combinations:** Supports rendering multiple keys connected with separators (e.g., “Ctrl + Shift + K”)
- **Consistent sizing:** Maintains proportional sizing with surrounding text
- **Monospace or system font:** Uses appropriate font that's clear and recognizable as a key

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<kbd>` HTML element for proper semantic meaning
- **Screen reader support:** Screen readers announce the element as keyboard input
- **Abbreviation expansion:** For abbreviated keys (e.g., “Ctrl”), can include aria-label with full word (“Control”)

Responsive Behavior

- **Text wrapping:** Long key combinations wrap gracefully to next line if needed
- **Touch context:** On touch devices, may include note that keyboard shortcuts don't apply

Internationalization

- **Platform-specific keys:** Displays platform-appropriate modifier keys:
 - macOS: Cmd (⌘), Option (⌥), Shift (⇧), Control (⌞)
 - Windows/Linux: Ctrl, Alt, Shift, Win
- **Localized key names:** Key names can be localized (e.g., “Enter” vs “Eingabe” in German)
- **Symbol vs text:** Supports both symbol glyphs (⌘) and text labels (Cmd) based on preference

Test Scenarios

- **Renders single key:** Component displays single key (e.g., “Enter”) with appropriate styling
- **Renders key combination:** Component displays multiple keys with separator (e.g., “Ctrl + C”)
- **Visual distinction:** KBD element is visually distinct from surrounding text (border, background, padding)
- **Semantic HTML:** Component renders using `element`
- **Platform detection:** On macOS, shows “Cmd”; on Windows/Linux, shows “Ctrl” for command modifier
- **Symbol rendering:** When configured to use symbols, displays ⌘ instead of “Cmd”
- **Screen reader announcement:** Screen reader correctly identifies element as keyboard input
- **Abbreviation support:** When abbreviated key has aria-label (e.g., “Ctrl” with `aria-label=“Control”`), screen reader announces full word
- **Font rendering:** Text is clear and legible in chosen font (monospace or system)

- **Wrapping behavior:** Long key combinations (e.g., “Ctrl + Shift + Alt + Delete”) wrap without breaking individual key labels

Notes

- **Platform detection:** Detect user’s OS to show appropriate modifier keys (Cmd vs Ctrl)
- **Symbol fonts:** If using symbol glyphs (⌘, ↑, ~), ensure font is loaded and includes these characters
- **Separator consistency:** Use consistent separator throughout application (typically ” + ” with spaces)
- **Capitalization:** Be consistent with capitalization (e.g., “Shift” vs “SHIFT”)
- **Touch devices:** Consider hiding or de-emphasizing keyboard shortcuts on touch-only devices where they don’t apply
- **Multiple representations:** Some systems show the key to press (e.g., “K”) and the modifier in different styles
- **Common shortcuts:** Document platform conventions for common shortcuts (copy, paste, undo, etc.)

#4 Code Snippet

Displays inline or block code with optional syntax highlighting and copy-to-clipboard action.

Description

Code Snippet displays source code, command-line instructions, configuration examples, or other pre-formatted text in a way that preserves formatting and enhances readability. It supports both inline code (within a paragraph) and block code (standalone sections). Syntax highlighting improves comprehension for programming languages, and a copy button reduces errors when users need to reproduce the code.

Main Features

- **Inline and block variants:** Inline code renders within text flow; block code renders as a distinct section
- **Monospace font:** Uses fixed-width font for proper alignment and character distinction
- **Whitespace preservation:** Maintains indentation, line breaks, and spaces exactly as provided
- **Syntax highlighting:** Optional color-coding for keywords, strings, comments, etc., based on language
- **Copy-to-clipboard:** Optional button to copy code content to clipboard
- **Line numbers:** Optional line numbering for reference (typically for block code only)

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<code>` for inline, `<pre><code>` for block code
- **Language annotation:** Includes language attribute for screen readers and syntax highlighters
- **Skip to code:** Long code blocks may include skip link to bypass for screen reader users
- **Contrast requirements:** Background and text colors meet contrast standards (especially important with syntax highlighting)
- **Copy button:** Copy button is keyboard accessible with clear label

Keyboard Navigation

- **Tab handling:** In editable code areas, Tab inserts tab character rather than moving focus
- **Copy keyboard shortcut:** Standard Ctrl+C (or Cmd+C) works to copy selected code
- **Copy button activation:** Copy button can be activated with Enter or Space when focused

Touch-screen

- **Touch-friendly copy button:** Copy button has adequate size (minimum 44x44px tap target)
- **Text selection:** Long-press enables text selection on touch devices
- **Scroll support:** Horizontally scrollable for long lines of code

Responsive Behavior

- **Horizontal scrolling:** Long code lines scroll horizontally rather than wrapping (preserves formatting)
- **Optional line wrapping:** Some implementations offer line wrapping as an option for readability on mobile
- **Font size:** Code remains readable at smaller viewport sizes (minimum 14px recommended)
- **Copy button positioning:** Copy button remains accessible/visible even in scrollable code blocks

Internationalization

- **Multi-byte character support:** Properly handles non-ASCII characters in code (useful for string literals, comments)
- **RTL within code:** Generally maintains LTR flow even in RTL interfaces (code is LTR by convention)

Advanced Features

- **Language detection:** Automatically detects programming language from content or filename
- **Line highlighting:** Can highlight specific lines to draw attention
- **Diff highlighting:** Shows additions (green) and deletions (red) for code comparisons
- **Editable code:** Some implementations allow editing the code in place
- **Code execution:** Advanced implementations may execute code and show output

Test Scenarios

- **Inline code renders:** Inline code component renders `<code>` within text with distinct background/styling
- **Block code renders:** Block code component renders `<pre><code>` with preserved formatting
- **Whitespace preservation:** Indentation, line breaks, and multiple spaces are preserved exactly
- **Monospace font:** Code displays in monospace font for proper alignment
- **Syntax highlighting:** When language is specified, keywords, strings, etc., are color-coded appropriately
- **Copy button appears:** When copy button is enabled, it's visible and positioned clearly
- **Copy functionality:** Clicking/tapping copy button copies code to clipboard
- **Copy feedback:** After copying, button shows confirmation (e.g., “Copied!” or checkmark icon)
- **Copy keyboard shortcut:** User can select code and copy with Ctrl+C (or Cmd+C)
- **Line numbers:** When enabled, line numbers display correctly aligned with code lines
- **Horizontal scroll:** Long lines trigger horizontal scroll without breaking layout
- **Language attribute:** Code element includes language class or attribute (e.g., `language-javascript`)
- **Accessible copy button:** Copy button has aria-label and is keyboard accessible
- **Contrast check:** Syntax highlighting colors meet contrast requirements
- **RTL context:** Code maintains LTR flow even when page is in RTL mode

Notes

- **Syntax highlighting libraries:** Popular options include Prism.js, Highlight.js, Shiki. Choose based on bundle size, language support, and theming needs

- **Client vs server rendering:** Syntax highlighting can be done server-side (faster initial render) or client-side (smaller server payload)
- **Light and dark themes:** Ensure syntax highlighting themes exist for both light and dark modes
- **Line wrapping trade-off:** Wrapping improves mobile readability but can obscure code structure. Consider toggle or default based on context
- **Tab character:** Real tab characters can render inconsistently. Some systems convert tabs to spaces (typically 2 or 4)
- **Security:** When displaying user-generated code, sanitize to prevent XSS attacks
- **Editable code:** If code is editable, clearly indicate this to users and consider validation/linting

#5 Separator / Divider

Visual horizontal or vertical line used to separate content sections. Can be decorative or semantic.

Description

A separator (also called divider) is a thin line that visually divides content into distinct sections or groups. It helps organize information, improve scannability, and establish visual hierarchy without requiring whitespace alone. Separators can be purely decorative or carry semantic meaning (indicating a thematic break in content).

Main Features

- **Orientation:** Renders as horizontal (default) or vertical line
- **Visual styling:** Thin line with subtle color (typically border or background color with reduced opacity)
- **Full-width or inset:** Can span full container width or be inset with padding/margins

- **Thickness control:** Configurable line thickness (typically 1px, sometimes 2px for emphasis)
- **Semantic vs decorative:** Can render as semantic `<hr>` element or decorative `<div>` based on use case

Secondary Features

Accessibility

- **Semantic separation:** When representing thematic break, uses `<hr>` element which screen readers recognize
- **Decorative separation:** When purely visual, uses `<div>` with `role="presentation"` or `aria-hidden="true"`
- **No reliance on separator alone:** Content structure remains clear even without visual separator (e.g., through headings, spacing)

Responsive Behavior

- **Orientation change:** Vertical separators may become horizontal on narrow viewports
- **Spacing adjustments:** Margins above/below separator may change at different breakpoints
- **Visibility:** Some separators may be hidden on mobile to reduce visual noise

Variants & Features

- **Solid, dashed, dotted:** Different line styles for visual variety
- **With text label:** Separator with centered text label (e.g., “OR” between options)
- **Gradient fade:** Line that fades in opacity toward edges
- **Inset amounts:** Different inset options (full-width, slight inset, heavy inset)

Test Scenarios

- **Horizontal separator renders:** Component displays horizontal line across container

- **Vertical separator renders:** When orientation is vertical, displays vertical line at specified height
- **Line thickness:** Separator displays at correct thickness (1px, 2px, etc.)
- **Line color:** Separator uses correct color from design tokens
- **Full-width behavior:** Full-width separator extends to container edges with no margin
- **Inset behavior:** Inset separator respects specified padding/margin on sides
- **Semantic HTML:** When semantic, renders as `<hr>` element
- **Decorative HTML:** When decorative, renders as non-semantic element with appropriate ARIA attributes
- **Vertical alignment:** Vertical separators align properly with adjacent content
- **Style variants:** When style is set (solid, dashed, dotted), displays correct line style
- **Label variant:** When separator includes text label, label is centered and properly positioned
- **Responsive orientation:** On narrow viewports, vertical separator becomes horizontal (if configured)
- **Spacing:** Separator maintains appropriate spacing from surrounding content

Notes

- **Semantic use cases:** Use `<hr>` for thematic breaks (e.g., between article sections, before footer). Use decorative dividers for visual organization (e.g., between list items, in navigation menus)
- **Alternatives to separators:** Consider if whitespace, background color changes, or borders on containers could achieve the same organization with less visual weight
- **Density:** Too many separators create visual clutter. Use sparingly and consistently
- **Color subtlety:** Separators should be subtle (typically 10-20% opacity of border color) unless they need to be emphatic
- **Vertical separator height:** Vertical separators often need explicit height or flex properties to render correctly

- **OR separator pattern:** When using separator with “OR” text between form options, ensure adequate touch targets for options on both sides

#6 Spacer

Invisible layout utility component for adding consistent spacing between elements without relying on margins.

Description

A spacer is an invisible, non-semantic component used purely for layout purposes. It adds explicit, consistent spacing between elements by taking up space in the layout. Spacers help maintain a spacing system without applying margins directly to content components, which can simplify component reusability and prevent margin-collapsing issues.

Main Features

- **Invisible rendering:** Takes up space but is not visible to users
- **Flexible sizing:** Accepts size values from the design system’s spacing scale
- **Axis control:** Can add space horizontally (inline) or vertically (block)
- **Responsive sizing:** Size can change at different breakpoints
- **Non-semantic:** Purely presentational, carries no meaning for screen readers or document structure

Secondary Features

Accessibility

- **Hidden from assistive tech:** Has `aria-hidden="true"` or similar to ensure screen readers skip it
- **No interactive content:** Never contains focusable or clickable elements
- **Semantic separation:** Doesn’t replace semantic separation (like `<hr>`) or proper content structure

Responsive Behavior

- **Breakpoint-specific sizing:** Spacer can use different spacing values at different screen sizes
- **Conditional rendering:** May be hidden entirely at certain breakpoints if spacing needs change
- **Flexible growth:** In flex or grid layouts, can grow to fill available space

Variants & Features

- **Size presets:** Small, medium, large, or custom values from spacing scale
- **Flexible spacer:** Can grow to fill remaining space in flex layouts (like CSS `flex-grow: 1`)
- **Directional variants:** Explicit horizontal vs vertical spacers

Test Scenarios

- **Takes up space:** Spacer occupies the specified height (vertical) or width (horizontal) in layout
- **Invisible rendering:** Spacer is not visible (no background, border, or content)
- **Size values:** Spacer renders at correct dimensions based on spacing scale (e.g., “medium” = 16px)
- **Orientation:** Vertical spacer adds height; horizontal spacer adds width
- **Responsive sizing:** At different breakpoints, spacer size changes as specified
- **Hidden from screen readers:** Spacer has `aria-hidden="true"` or is implemented in a way that screen readers skip it
- **No interactive content:** Spacer cannot be focused, clicked, or interacted with
- **Flex growth:** When configured as flexible, spacer grows to fill available space in flex container
- **Multiple spacers:** Multiple spacers in sequence combine spacing correctly (no margin collapse issues)
- **Layout impact:** Spacer correctly affects layout of surrounding elements

Notes

- **Alternatives:** Before using spacers, consider if CSS gap (in flex/grid), padding on containers, or margins on components would be cleaner
- **Use case:** Spacers are particularly useful in component composition where you can't easily control margins on child components
- **Stack components:** Often paired with Stack components (vertical or horizontal) that use gaps instead of spacers for cleaner code
- **Don't overuse:** Excessive spacers can make layouts harder to maintain. Prefer semantic spacing (gap, padding) where possible
- **Implementation:** Can be as simple as a `<div>` with height/width and `aria-hidden`, or a zero-width space character for inline contexts
- **Flexible spacers:** Useful for pushing items apart in toolbars or navigation (e.g., logo on left, flexible spacer, nav items on right)
- **Naming:** Some systems call this “Box” with specific spacing props, others have dedicated Spacer component

#7 Aspect Ratio

Container that enforces a proportional width-to-height ratio for its content (useful for images, videos, maps).

Description

An aspect ratio container maintains a consistent proportional relationship between width and height, regardless of the container's actual width. This is essential for media content (images, videos, iframes) that needs to scale responsively without distortion or layout shifts. Common aspect ratios include 16:9 (widescreen video), 4:3 (standard video), 1:1 (square), and 21:9 (ultrawide).

Main Features

- **Ratio enforcement:** Maintains specified width-to-height ratio as container scales
- **Responsive scaling:** Container width adjusts to parent, height adjusts proportionally
- **Content containment:** Child content fills the container while respecting the ratio
- **Common ratio presets:** Supports standard ratios (16:9, 4:3, 1:1, 21:9, etc.)
- **Custom ratios:** Accepts custom ratio values (e.g., 3:2, 5:4)

Secondary Features

Responsive Behavior

- **Flexible width:** Container width adapts to parent container or viewport width
- **Automatic height:** Height is calculated automatically based on width and ratio
- **No layout shift:** Reserves space before content loads, preventing cumulative layout shift (CLS)
- **Breakpoint-specific ratios:** Can use different aspect ratios at different screen sizes (e.g., 16:9 on desktop, 1:1 on mobile)

Variants & Features

- **Object fit control:** Child content can fill, contain, cover, or scale-down within ratio container
- **Alignment options:** Content can be aligned within the ratio box (top, center, bottom, left, right)
- **Overflow handling:** Content that exceeds ratio boundaries can be clipped or scrollable

Test Scenarios

- **Maintains ratio:** Container maintains specified aspect ratio (e.g., 16:9) as width changes
- **Height calculation:** Height is automatically calculated based on width and ratio
- **Common ratios:** Preset ratios (16:9, 4:3, 1:1) render with correct proportions
- **Custom ratio:** When custom ratio is provided (e.g., 3:2), container uses that ratio
- **Content fills container:** Child content (image, video, iframe) fills the aspect ratio container
- **Responsive scaling:** As parent width changes, container scales while maintaining ratio
- **No layout shift:** Container reserves correct space before content loads (measured via CLS metric)
- **Breakpoint ratios:** When different ratios are specified per breakpoint, correct ratio applies at each size
- **Object fit:** When object-fit is set to cover/contain, content scales appropriately
- **Empty container:** Container maintains ratio even when empty (no content loaded yet)

Notes

- **Native CSS solution:** Modern CSS supports `aspect-ratio` property, eliminating need for padding-hack workarounds
- **Legacy browser support:** For older browsers, implement using padding-bottom percentage technique
- **Common use cases:** YouTube embeds, hero images, product photos, map embeds, responsive iframes
- **Layout shift prevention:** Critical for Core Web Vitals and user experience—prevents content jumping during page load
- **Video embeds:** Essential for responsive video embeds (YouTube, Vimeo, etc.) to prevent black bars or distortion

- **Art direction:** Consider if different aspect ratios at different breakpoints improve the experience (e.g., landscape on desktop, portrait on mobile)

#8 Image

Optimized image display with loading state, error fallback, and alt text support.

Description

The Image component provides an accessible, performant way to display images with proper loading states, error handling, and responsive behavior. It ensures images are optimized for web delivery, accessible to all users, and don't cause layout shifts during load. Beyond basic `<img>` functionality, it handles lazy loading, responsive images, and graceful degradation.

Main Features

- **Image rendering:** Displays image from provided source URL
- **Alt text support:** Requires meaningful alt text for accessibility
- **Loading states:** Shows placeholder or skeleton while image loads
- **Error handling:** Displays fallback UI when image fails to load
- **Lazy loading:** Defers loading images until they're near viewport (improves performance)
- **Responsive images:** Serves appropriately sized images based on viewport and pixel density

Secondary Features

Accessibility

- **Alt text requirement:** Enforces or strongly encourages alt text for all images
- **Decorative images:** Supports empty alt (`alt=""`) for purely decorative images

- **Meaningful descriptions:** Alt text describes image content and purpose, not just filename
- **Screen reader support:** Alt text is announced by screen readers in place of image

Responsive Behavior

- **Srcset support:** Provides multiple image sources for different screen sizes and densities
- **Sizes attribute:** Specifies how much space image will occupy at different breakpoints
- **Object fit:** Controls how image fills its container (cover, contain, fill, scale-down)
- **Lazy loading:** Images below fold load only when user scrolls near them
- **Aspect ratio preservation:** Maintains image aspect ratio to prevent layout shifts

Advanced Features

- **Progressive loading:** Loads low-quality placeholder, then high-quality image (blur-up effect)
- **WebP/AVIF support:** Serves modern formats with fallbacks for older browsers
- **Retina support:** Serves 2x or 3x images for high-DPI displays
- **Image optimization:** Integrates with image CDNs or build-time optimization tools
- **Zoom/lightbox:** Optional click-to-zoom or lightbox functionality
- **Caption support:** Optional caption element associated with image

Test Scenarios

- **Image renders:** Image displays correctly from provided source URL
- **Alt text present:** Image has alt attribute with descriptive text
- **Decorative image:** When decorative, image has empty alt (`alt=""`)
- **Loading state:** Before image loads, loading placeholder or skeleton is visible

- **Error state:** When image fails to load, error fallback UI appears (broken image icon, fallback text, or alternative image)
- **Lazy loading:** Images outside viewport don't load until user scrolls near them
- **Srcset selection:** Browser selects appropriate image source based on viewport width and pixel density
- **Aspect ratio:** Container reserves correct space for image to prevent layout shift (CLS)
- **Object fit:** When object-fit is set, image scales according to specified value
- **Retina display:** On high-DPI screens, higher-resolution image is loaded
- **Progressive load:** Low-quality placeholder loads first, then replaced with high-quality image
- **Modern format:** Browser receives WebP or AVIF if supported, falls back to JPEG/PNG if not
- **Caption association:** When caption is provided, it's properly associated with image (e.g., using `<figure>` and `<figcaption>`)

Notes

- **Alt text best practices:** Describe what the image shows and its purpose, not “image of” or filename. For complex images (charts, diagrams), provide longer description nearby
- **Performance:** Images are often the largest assets on a page. Always optimize (compress, resize, use modern formats)
- **Lazy loading:** Use native `loading="lazy"` attribute for simplicity, or intersection observer for more control
- **Layout shift prevention:** Always specify width and height attributes (even if CSS overrides) to enable browsers to calculate aspect ratio
- **Responsive images:** Use `srcset` for art direction (different crops at different sizes) or resolution switching (same image, different sizes)
- **CDN usage:** Consider image CDNs (Cloudinary, Imgix, Cloudflare Images) for automatic optimization and transformation
- **Next-gen formats:** WebP typically 25-35% smaller than JPEG, AVIF even better. Always provide fallbacks for older browsers

#9 Avatar

Circular or rounded image representing a user or entity, with text/icon fallback when image is unavailable.

Specializations: Avatar Group (overlapping stack of multiple avatars)

Description

An avatar is a visual representation of a user, team, organization, or entity, typically displayed as a circular or rounded square image. Avatars help users quickly identify people or entities in lists, comments, profiles, and collaborative interfaces. They provide a consistent, recognizable visual identity and include fallback mechanisms when images aren't available.

Main Features

- **Image display:** Shows user's profile photo or entity logo in circular or rounded container
- **Fallback mechanisms:** When image unavailable, displays initials, icon, or placeholder
- **Size variants:** Small (16-24px), medium (32-48px), large (64-96px), extra-large (128px+)
- **Shape options:** Circle (most common) or rounded square
- **Status indicator:** Optional status badge (online, offline, busy, away)

Secondary Features

Accessibility

- **Alt text:** Includes meaningful alt text with person's name or entity identifier
- **Status announcement:** Status indicator (if present) is announced to screen readers

- **Non-interactive:** When avatar is purely display (not clickable), no keyboard focus needed
- **Interactive avatars:** When clickable (e.g., to view profile), has proper button/link semantics

Responsive Behavior

- **Consistent sizing:** Maintains circular/square shape at all breakpoints
- **Flexible scaling:** Sizes can adapt based on context (smaller in lists, larger on profile pages)
- **Image optimization:** Uses appropriately sized image for avatar dimensions (no need for huge source)

Variants & Features

- **Initials fallback:** Displays user's initials (e.g., "JD" for John Doe) when no image
- **Icon fallback:** Shows generic user icon when no image or initials available
- **Color generation:** Generates consistent background color from user ID/name for initials fallback
- **Border/ring:** Optional border or ring for emphasis or status indication
- **Avatar Group:** Multiple avatars displayed in overlapping stack with count of additional users

Test Scenarios

- **Image display:** Avatar displays user's image in correct shape (circle or rounded square)
- **Size variants:** Avatar renders at correct size (small, medium, large) based on prop
- **Initials fallback:** When image URL is unavailable, displays user's initials
- **Icon fallback:** When neither image nor initials available, displays generic icon
- **Alt text:** Avatar has alt attribute with user's name or entity identifier

- **Status indicator:** When status is provided (online, offline, etc.), badge displays in correct position
- **Consistent colors:** Initials fallback generates consistent color for same user across sessions
- **Shape rendering:** Circular avatars are perfectly round; rounded square has appropriate border radius
- **Image loading:** While image loads, displays fallback; swaps to image when loaded
- **Image error:** When image fails to load, reverts to initials or icon fallback
- **Interactive behavior:** When clickable, avatar has proper hover state and pointer cursor
- **Avatar Group:** Multiple avatars overlap correctly with proper z-index stacking
- **Avatar Group count:** When more users than displayed avatars, shows “+N” indicator

Avatar Group Specialization

Avatar Group displays multiple avatars in an overlapping horizontal stack, commonly used to show collaborators, participants, or team members.

Additional Features:

- **Overlap amount:** Avatars overlap by specified amount (typically 25-50% of avatar width)
- **Max display:** Limits number of visible avatars, shows “+N more” for remainder
- **Tooltip on hover:** Hovering avatar shows user’s full name
- **Z-index stacking:** Avatars stack with correct layering (first on top or last on top)

Additional Test Scenarios:

- **Overlap rendering:** Avatars overlap by correct amount without gaps
- **Max count:** When max is set to 3 and 5 users exist, shows 3 avatars + “+2”

- **Stacking order:** Avatars layer correctly (first on top, or last on top based on configuration)
- **Hover tooltip:** Hovering each avatar shows that user's name in tooltip
- **Responsive stacking:** Avatar group adjusts size appropriately on smaller screens

Notes

- **Image source:** Use optimized, square images (profile photos typically 200-400px square)
- **Initials logic:** Extract initials from full name (first letter of first name + first letter of last name)
- **Color generation:** Hash user ID or name to generate consistent color; ensure adequate contrast with white text
- **Accessibility of groups:** Avatar groups should have accessible label indicating how many users/members
- **Interaction patterns:** Clickable avatars typically open profile page or user menu
- **Status colors:** Common conventions—green (online), gray (offline), yellow (away), red (busy)
- **Large avatars:** On profile pages, avatars may be much larger (128-256px) and allow editing/uploading

#10 Badge

Small count or status overlay attached to another element (e.g., notification count on an icon or button). Sometimes called *Indicator* or *Pip* in other libraries.

Naming note: Not to be confused with **Chip/Tag**, which is a standalone element. Badge overlays on top of another element; Chip/Tag stands alone.

Description

A badge is a small visual indicator that displays counts, status, or notifications by overlaying on top of another UI element (typically icons, buttons, or avatars).

Badges draw attention to new activity, unread messages, pending tasks, or important status changes. They're essential for notification systems and activity indicators.

Main Features

- **Overlay positioning:** Attaches to corner or edge of parent element
- **Count display:** Shows numeric count (e.g., “5” for 5 unread messages)
- **Status indication:** Displays status without count (e.g., red dot for “has notifications”)
- **Size variants:** Small dot (just status), medium (single digit), large (multiple digits)
- **Color coding:** Different colors for different types (e.g., red for errors, blue for info)

Secondary Features

Accessibility

- **Screen reader announcement:** Badge content is announced (e.g., “5 unread notifications”)
- **Live region:** When count updates, change is announced to screen readers
- **Semantic meaning:** Color alone isn't sole indicator—count or status text provides meaning
- **Parent label:** Badge supplements parent element's label (e.g., button label includes badge count)

Responsive Behavior

- **Proportional sizing:** Badge scales appropriately with parent element size

- **Position stability:** Badge position remains anchored correctly across screen sizes
- **Overflow handling:** Very large counts display truncated (e.g., “99+” instead of “142”)

Variants & Features

- **Dot variant:** Simple colored dot with no text (indicates presence/absence of notifications)
- **Max count:** Displays maximum count (e.g., “99+”) when actual count exceeds threshold
- **Animation:** Optional subtle animation when badge appears or count changes
- **Positioning options:** Top-right (default), top-left, bottom-right, bottom-left

Test Scenarios

- **Badge renders:** Badge displays overlaid on parent element
- **Position:** Badge appears in correct corner (typically top-right by default)
- **Count display:** Numeric count displays correctly inside badge
- **Dot variant:** When configured as dot, badge appears as small circle without text
- **Color variants:** Badge displays in correct color based on type (error, info, success, etc.)
- **Max count:** When count exceeds max (e.g., 99), displays “99+” or configured max indicator
- **Zero count:** When count is zero, badge hides or remains visible based on configuration
- **Screen reader:** Badge count is announced to screen readers (e.g., “notification, 5 unread”)
- **Live update:** When count changes, screen readers announce the new count
- **Parent label:** Parent element’s accessible label includes badge information
- **Size scaling:** Badge size is appropriate for parent element size (small badge on small icon)
- **Overlap:** Badge doesn’t obscure critical parts of parent element (e.g., doesn’t cover icon’s visual meaning)

- **Animation:** When badge appears or count changes, optional animation plays smoothly

Notes

- **When to use badge vs chip:** Use badge as overlay on another element; use chip/tag as standalone label or filter
- **Count thresholds:** Consider when to show “99+” (or “9+”) to keep badge compact
- **Zero handling:** Typically hide badge when count is zero; sometimes keep visible for consistency
- **Real-time updates:** Badges often update in real-time via websockets or polling—ensure smooth transitions
- **Notification patterns:** Red badges typically indicate urgent/error; blue for info; green for success; gray for neutral
- **Multiple badges:** Avoid multiple badges on single element—consolidate or prioritize
- **Accessibility:** Never use color alone—include count or status text. “Red dot” means nothing to colorblind users without additional context

#11 Chip / Tag

Compact standalone label representing a value, category, or filter. Optionally removable or selectable. Also called *Token*, *Pill*, or *Label* in other libraries.

Specialization: Chip Group (collection of chips, useful for filter sets)

Description

A chip (or tag) is a compact, standalone element that represents an input, attribute, or action. Chips are commonly used for tags, categories, filters, selected items, or user-entered tokens. They can be static (display only), removable (with close button), or selectable (acting like toggles). Chips make complex information more scannable and allow users to manipulate multiple values easily.

Main Features

- **Compact display:** Small, rounded element containing text and/or icon
- **Standalone element:** Independent element, not overlaid on others (unlike badges)
- **Removable variant:** Includes close button (×) to remove chip
- **Selectable variant:** Acts like toggle button, can be selected/deselected
- **Icon support:** Optional leading or trailing icon

Secondary Features

Accessibility

- **Keyboard accessible:** Removable/selectable chips can be focused and activated via keyboard
- **Remove action:** Close button has clear label (e.g., “Remove tag: Design”)
- **Selection state:** Selected chips are announced as “selected” to screen readers
- **Focus indicator:** Clear focus ring when chip receives keyboard focus
- **Role semantics:** Appropriate ARIA role (button for removable/selectable, none for static)

Keyboard Navigation

- **Tab navigation:** Chips can be focused via Tab/Shift+Tab
- **Remove activation:** Backspace, Delete, or Enter removes chip (when removable)
- **Selection toggle:** Space or Enter toggles selection state (when selectable)
- **Arrow navigation:** In chip groups, arrow keys navigate between chips

Touch-screen

- **Touch target:** Adequate tap target size (minimum 44x44px including padding)
- **Close button size:** Remove button is large enough for easy tapping
- **Visual feedback:** Touch provides immediate visual feedback (press state)

Responsive Behavior

- **Text truncation:** Long labels truncate with ellipsis to maintain compact size
- **Wrapping:** Chip groups wrap to multiple lines when horizontal space is limited
- **Size variants:** Small, medium, large sizes for different contexts

Variants & Features

- **Static chip:** Display-only, no interaction
- **Removable chip:** Includes close button to delete
- **Selectable chip:** Toggle selection on/off (acts like checkbox button)
- **Avatar chip:** Includes avatar image before text (for person tags)
- **Color coding:** Different colors for different categories or states
- **Outlined vs filled:** Outlined (border only) or filled (background color) styles

Test Scenarios

- **Chip renders:** Chip displays with label text in compact, rounded container
- **Icon display:** When icon is provided, displays icon before or after text
- **Static chip:** Non-interactive chip has no focus state or hover effect
- **Removable chip:** Chip includes visible close button (×)
- **Remove action:** Clicking close button triggers onRemove callback and removes chip
- **Selectable chip:** Clicking chip toggles selected state
- **Selection state:** Selected chip shows distinct visual style (filled background, border, checkmark, etc.)
- **Keyboard focus:** Interactive chips (removable/selectable) can receive keyboard focus
- **Keyboard removal:** Pressing Delete or Backspace on focused removable chip removes it
- **Keyboard selection:** Pressing Space on focused selectable chip toggles selection
- **Screen reader label:** Remove button has accessible label including chip content (e.g., “Remove Design tag”)

- **Selection announcement:** Screen reader announces “selected” or “not selected” state
- **Truncation:** Long chip labels truncate with ellipsis and show full text on hover/focus
- **Color variants:** Chip displays in correct color based on category or state
- **Avatar chip:** When avatar is provided, displays small avatar image before text

Chip Group Specialization

Chip Group is a collection of chips displayed together, commonly used for filter sets, selected items, or tag lists.

Additional Features:

- **Horizontal/vertical layout:** Chips arranged in row or column
- **Wrapping:** Chips wrap to multiple lines when space is limited
- **Add button:** Optional button to add new chips
- **Clear all:** Optional action to remove all chips at once

Additional Test Scenarios:

- **Multiple chips render:** All chips in group display correctly
- **Wrapping behavior:** Chips wrap to new line when container width is exceeded
- **Consistent spacing:** Chips have consistent gap between them
- **Add new chip:** When add button is clicked, new chip creation flow initiates
- **Clear all:** When clear all action is triggered, all chips are removed
- **Keyboard navigation:** Arrow keys move focus between chips in group
- **Batch removal:** Multiple selected chips can be removed together

Notes

- **Chip vs Badge:** Chips are standalone; badges overlay other elements
- **Chip vs Button:** Chips represent data/attributes; buttons trigger actions

- **Common use cases:** Filter pills, selected search results, email recipients, category tags, skill badges
- **Multi-select patterns:** In filter contexts, chips often work like checkboxes—multiple can be selected
- **Input chips:** Some implementations allow typing to create new chips (common in email “To:” fields)
- **Max chips:** Consider limiting visible chips and showing “+N more” for long lists
- **Removal confirmation:** For destructive removals (e.g., removing team member), consider confirmation dialog

#12 Skeleton

Placeholder shape displayed while content is loading, mimicking the layout of the expected content. *Specialization:* Skeleton Layout (composed skeleton representing an entire section or page region)

Description

A skeleton (or skeleton screen) is an animated placeholder that mimics the structure of content while it loads. Unlike generic spinners, skeletons show users where content will appear and approximately what it will look like, reducing perceived loading time and preventing layout shifts. Skeletons provide visual continuity during loading states and set accurate expectations.

Main Features

- **Shape mimicry:** Placeholder shapes resemble the structure of loading content (rectangles for text, circles for avatars, etc.)
- **Shimmer animation:** Subtle wave or pulse animation indicates loading is in progress
- **Layout preservation:** Occupies same space as final content to prevent layout shifts

- **Component-level or page-level:** Can represent single component (card skeleton) or entire page section
- **Graceful transition:** Smoothly replaced by actual content when loaded

Secondary Features

Accessibility

- **Screen reader announcement:** Announces “loading” state to screen readers
- **Live region:** Loading state changes are announced when content loads
- **Hidden when loaded:** Skeleton is removed from accessibility tree when replaced by content
- **Loading indicator:** Includes aria-busy=“true” or aria-live=“polite” for status updates

Responsive Behavior

- **Proportional sizing:** Skeleton shapes scale with their containers across breakpoints
- **Layout adaptation:** Skeleton layout matches actual content layout at each breakpoint
- **Mobile optimization:** Simpler skeletons on mobile (fewer details) for faster perception

Animation & Transitions

- **Shimmer effect:** Animated gradient or wave that moves across skeleton
- **Pulse effect:** Alternative to shimmer—subtle opacity fade in/out
- **Reduced motion:** Respects prefers-reduced-motion by showing static skeleton
- **Smooth replacement:** Fade-in transition when real content replaces skeleton

Variants & Features

- **Text skeleton:** Horizontal bars representing text lines
- **Circle skeleton:** Circular shapes for avatars or profile images
- **Rectangle skeleton:** Boxes for images, cards, or content blocks
- **Custom shapes:** Specialized shapes for specific components (charts, icons, etc.)

Test Scenarios

- **Skeleton displays:** Skeleton placeholder appears before content loads
- **Shape accuracy:** Skeleton shapes approximate the layout of final content
- **Animation plays:** Shimmer or pulse animation is visible and smooth
- **Layout preservation:** Skeleton occupies same space as final content (no layout shift)
- **Screen reader announcement:** Loading state is announced to screen readers
- **Content replacement:** When content loads, skeleton is removed and content appears
- **Smooth transition:** Content fades in smoothly when replacing skeleton
- **Reduced motion:** When user prefers reduced motion, skeleton is static (no animation)
- **Multiple skeletons:** Multiple loading elements show synchronized or independent skeletons
- **Responsive layout:** Skeleton layout matches actual content layout at different breakpoints
- **Accessibility tree:** Skeleton is properly hidden from screen readers after content loads
- **Performance:** Skeleton renders quickly without causing additional delay

Skeleton Layout Specialization

Skeleton Layout is a composition of multiple skeleton elements representing an entire page section or complex component.

Additional Features:

- **Composed structure:** Multiple skeleton shapes arranged to mimic complete UI section
- **Reusable templates:** Common layouts (card grid, list, profile page) as reusable skeleton templates
- **Conditional elements:** Different skeleton layouts for different content types

Additional Test Scenarios:

- **Layout composition:** Multiple skeleton shapes render in correct positions
- **Template accuracy:** Skeleton layout closely matches actual content structure
- **Nested skeletons:** Skeleton components can be nested (e.g., card skeletons within grid skeleton)
- **Partial loading:** As content loads progressively, skeleton sections are replaced incrementally

Notes

- **When to use:** Use skeletons for content that takes >300ms to load; use spinners for faster loads or indeterminate actions
- **Skeleton vs spinner:** Skeletons for content loading (known structure); spinners for actions/processes (unknown duration)
- **Design accuracy:** Skeleton should closely match final content—users notice discrepancies
- **Animation subtlety:** Animation should be gentle and not draw excessive attention
- **Shimmer direction:** LTR interfaces typically shimmer left-to-right; RTL interfaces right-to-left
- **Color choices:** Light gray on white background (light mode), dark gray on black (dark mode)
- **Progressive loading:** Consider showing skeleton, then low-res content, then high-res content for images

- **Component libraries:** Many UI libraries provide skeleton components; consistent use improves UX

#13 Loader

Indicates an ongoing process or loading state. *Specialization:* Circular Loader (spinning circle, the most common form)

Naming note: Also called *Spinner* in many libraries. We use *Loader* as the base name to be more descriptive of purpose.

Description

A loader (commonly called spinner) is an animated indicator that communicates an ongoing process, data loading, or system activity. Unlike skeletons which show content structure, loaders indicate indeterminate progress or brief waiting periods. Loaders reassure users that the system is working and haven't frozen or failed.

Main Features

- **Animation:** Continuous rotating, pulsing, or bouncing animation
- **Size variants:** Small (inline with text), medium (standalone), large (full-page overlays)
- **Indeterminate state:** Indicates activity without showing specific progress
- **Optional label:** Text label explaining what's loading (e.g., "Loading data...")
- **Color customization:** Adapts to context (light/dark backgrounds, brand colors)

Secondary Features

Accessibility

- **Screen reader announcement:** Announces loading state to screen readers
- **Live region:** Changes in loading state are announced (aria-live="polite")

- **Loading text:** Includes visually hidden text describing the action (e.g., “Loading content”)
- **Role attribute:** Uses role=“status” or role=“alert” for ARIA semantics

Responsive Behavior

- **Proportional sizing:** Loader size scales appropriately for context (small in buttons, large in page centers)
- **Mobile considerations:** Not too large on mobile screens; doesn’t block entire viewport unnecessarily

Animation & Transitions

- **Smooth rotation:** Continuous, smooth animation without stuttering
- **Reduced motion:** Respects prefers-reduced-motion by showing static or subtle animation
- **Entrance/exit:** Fades in when appearing, fades out when disappearing
- **Frame rate:** Maintains 60fps for smooth visual experience

Variants & Features

- **Circular spinner:** Spinning circle or arc (most common)
- **Linear loader:** Horizontal progress bar with indeterminate animation
- **Dots loader:** Three bouncing or pulsing dots
- **Skeleton alternative:** For content loading, skeletons may be more appropriate
- **Overlay loader:** Full-page or container overlay with loader centered

Test Scenarios

- **Loader renders:** Loader animation displays correctly
- **Animation plays:** Loader continuously animates (spins, pulses, or bounces)
- **Size variants:** Loader renders at correct size (small, medium, large)
- **Label display:** When label is provided, text appears below or beside loader
- **Screen reader:** Loading state is announced to screen readers

- **Live region:** When loading state changes (starts/stops), screen readers announce the change
- **Reduced motion:** When user prefers reduced motion, animation is static or minimal
- **Color adaptation:** Loader color contrasts adequately with background
- **Smooth animation:** Animation is smooth at 60fps without stutter
- **Overlay behavior:** When used as overlay, loader centers correctly and backdrop prevents interaction
- **Multiple loaders:** Multiple simultaneous loaders don't conflict or overlap inappropriately
- **Removal:** Loader is fully removed from DOM when loading completes

Circular Loader Specialization (Default)

The circular loader is the most common form—a rotating circle or arc that indicates activity.

Additional Features:

- **Complete circle:** Full circle with rotating segment cut out
- **Partial arc:** Rotating arc (typically 270° or less) that spins
- **Stroke thickness:** Configurable line thickness for the circle
- **Determinate option:** Can show percentage progress (0-100%) instead of indeterminate spin

Additional Test Scenarios:

- **Circular shape:** Loader displays as perfect circle
- **Rotation direction:** Circle rotates clockwise (or counterclockwise if configured)
- **Stroke rendering:** Circle stroke is smooth and consistent thickness
- **Determinate mode:** When percentage is provided (e.g., 45%), loader shows 45% completion
- **Percentage label:** Determinate loader optionally displays percentage text inside or beside circle

Notes

- **When to use loader vs skeleton:** Loaders for actions/processes; skeletons for content loading with known structure
- **Duration considerations:** For loads <300ms, consider no indicator (feels instant). For >300ms, show loader. For >2 seconds, consider skeleton or progress bar
- **Blocking vs non-blocking:** Decide if loader should block interaction (overlay) or allow continued interaction
- **Button loaders:** Loaders inside buttons should be small and positioned where button text was
- **Error states:** When loading fails, replace loader with error message
- **Timeout handling:** Consider timeout for long-running loaders (show error or alternative after 30-60 seconds)
- **Accessibility timing:** Don't announce loading state immediately—wait 1-2 seconds to avoid spamming screen reader users for quick loads

#14 Color Swatch

Displays a single color sample, typically as a small colored square or circle. Used in color pickers and palette displays.

Description

A color swatch is a small visual representation of a single color, used to display, select, or preview colors in design tools, customization interfaces, or product configurators. Swatches make color selection intuitive and visual, allowing users to see and compare colors before making choices.

Main Features

- **Color display:** Shows specified color as filled shape (square, circle, or rounded square)

- **Size variants:** Small (16-24px), medium (32-48px), large (64px+)
- **Shape options:** Square, circle, or rounded square
- **Border/outline:** Optional border for visibility on white/light backgrounds
- **Selectable variant:** Can be clicked to select that color

Secondary Features

Accessibility

- **Color name label:** Provides text label with color name or hex value
- **Tooltip on hover:** Shows color value (hex, RGB) on hover
- **Selection state:** Selected swatch has clear visual indicator (border, checkmark, scale)
- **Keyboard accessible:** Selectable swatches can be focused and activated via keyboard
- **Not color-only:** When selectable, selection state isn't indicated by color alone (uses border, icon, etc.)

Keyboard Navigation

- **Tab navigation:** Selectable swatches can receive keyboard focus
- **Enter/Space selection:** Pressing Enter or Space selects the focused swatch
- **Arrow navigation:** In swatch groups, arrow keys navigate between swatches
- **Focus indicator:** Clear focus ring when swatch receives keyboard focus

Touch-screen

- **Touch target:** Adequate tap target size (minimum 44x44px including spacing)
- **Touch feedback:** Immediate visual feedback on tap (press state)
- **Tap to select:** Single tap selects color

Responsive Behavior

- **Flexible sizing:** Swatches scale appropriately for different screen sizes
- **Grid layout:** Multiple swatches arrange in responsive grid

- **Scrollable groups:** Large color sets scroll horizontally or vertically

Variants & Features

- **Named colors:** Common color names (red, blue, etc.) displayed as swatches
- **Custom colors:** User-defined or arbitrary hex/RGB colors
- **Transparent swatch:** Special pattern (checkered) to indicate transparency
- **Gradient swatch:** Can display gradient instead of solid color
- **Color picker integration:** Clicking swatch opens full color picker
- **Recently used colors:** Swatches showing user's recent color selections

Test Scenarios

- **Swatch renders:** Color swatch displays in correct shape and size
- **Color accuracy:** Swatch displays exact specified color (hex, RGB, HSL)
- **Shape rendering:** Square, circle, or rounded square renders correctly
- **Border visibility:** Border appears when configured or when color is too light
- **Size variants:** Swatch renders at correct size (small, medium, large)
- **Selection state:** When selected, swatch shows clear visual indicator (border, checkmark, scale)
- **Hover state:** Selectable swatch shows hover effect (border, scale, shadow)
- **Keyboard focus:** Selectable swatch can receive focus with visible focus ring
- **Keyboard selection:** Pressing Enter or Space selects focused swatch
- **Color label:** Tooltip or adjacent text shows color name/value on hover or focus
- **Transparent pattern:** Transparent colors show checkered pattern background
- **Multiple swatches:** Grid of swatches displays with consistent spacing
- **Touch feedback:** Tapping swatch provides immediate visual feedback
- **Callback execution:** When swatch is selected, onChange/onSelect callback fires with color value

Notes

- **Color contrast:** Very light colors may need border for visibility; very dark colors may need lighter border
- **Transparent swatch pattern:** Standard checkered pattern (alternating light/dark squares) indicates transparency
- **Color formats:** Support multiple color formats (hex, RGB, HSL, named colors) for flexibility
- **Recent colors:** Showing recently used colors improves efficiency in design tools
- **Swatch collections:** Common patterns include Material Design palette, Tailwind colors, brand color palette
- **Interactive vs display:** Distinguish between selectable swatches (color picker) and display-only swatches (showing current color)
- **Color picker integration:** Swatches often serve as quick-access for color picker—clicking opens full picker with current color pre-selected

#15 Link

Inline text navigation element with proper semantic meaning, visited state, and keyboard support.

Description

A link is a navigational element that allows users to move to different pages, sections, or external resources. Links are fundamental to web navigation and information architecture. They must be semantically correct, visually distinguishable from regular text, and accessible to all users including those using keyboards or assistive technologies.

Main Features

- **Navigation:** Navigates to specified URL or anchor when activated

- **Semantic HTML:** Uses `<a>` element with proper href attribute
- **Visual distinction:** Clearly distinguishable from regular text (typically underlined, colored, or both)
- **Visited state:** Shows different appearance for visited vs unvisited links
- **Hover indication:** Provides visual feedback on hover
- **Inline placement:** Works within text flow without breaking layout

Secondary Features

Accessibility

- **Keyboard accessible:** Can be activated via keyboard (Enter key)
- **Focus indicator:** Clear visual indicator when link has keyboard focus
- **Meaningful text:** Link text describes destination (avoid “click here” or “read more” without context)
- **External link indication:** External links indicate they open in new context (icon or text)
- **Skip to content:** Special links allow keyboard users to skip navigation (skip links)

Keyboard Navigation

- **Tab navigation:** Can be reached via Tab/Shift+Tab in document tab order
- **Enter activation:** Pressing Enter activates the link
- **Context menu:** Right-click or context menu key provides browser navigation options

Touch-screen

- **Touch target:** Adequate tap target size (minimum 44x44px including padding)
- **Touch feedback:** Provides visual feedback on tap (active state)
- **Long-press menu:** Long-press shows browser context menu (open in new tab, copy link, etc.)

Responsive Behavior

- **Text wrapping:** Link text wraps naturally within paragraphs
- **Breakable URLs:** Long URLs break at appropriate points to avoid overflow
- **Touch target on mobile:** Ensures adequate spacing between adjacent links on mobile

Internationalization

- **Bi-directional text:** Link text and surrounding content handle RTL languages correctly
- **Localized URLs:** Supports internationalized domain names and URL paths

Variants & Features

- **External links:** Links to external domains, often with icon indicator
- **Download links:** Links that download files instead of navigating
- **Anchor links:** Links to sections within same page (#anchor)
- **Button-styled links:** Links that look like buttons (for CTAs) but maintain link semantics
- **Disabled state:** Links that are temporarily non-functional (rare, usually avoid this)

Test Scenarios

- **Link renders:** Link displays with appropriate visual styling (color, underline)
- **Navigation works:** Clicking link navigates to specified URL
- **Semantic HTML:** Link renders as `<a>` element with href attribute
- **Visited state:** Previously visited links show different visual style
- **Hover state:** Hovering link shows distinct hover style (underline, color change, etc.)
- **Active state:** When clicked, link shows active/pressed state briefly
- **Keyboard focus:** Tab moves focus to link with visible focus indicator
- **Enter activation:** Pressing Enter on focused link activates navigation

- **External indicator:** External links show icon or text indicating they leave site
- **New tab behavior:** When configured to open in new tab, target=“_blank” with rel=“noopener noreferrer”
- **Download attribute:** Download links trigger download instead of navigation
- **Anchor navigation:** Clicking anchor link scrolls to target section on same page
- **Touch feedback:** Tapping link on touch device shows immediate visual feedback
- **Meaningful text:** Link text describes destination without relying solely on surrounding context
- **Skip link:** Skip-to-content link is first focusable element and jumps to main content

Notes

- **Link vs Button:** Links navigate; buttons perform actions. Use semantic element that matches intent
- **Underline convention:** Underlined text signals links to users. Removing underlines can hurt usability
- **Color alone insufficient:** Don't rely solely on color to distinguish links—use underline or other visual cue (WCAG)
- **Visited state privacy:** Modern browsers limit visited state styling to protect user privacy
- **External links:** Consider warning users that link opens externally, especially for government/financial sites
- **“Click here” problem:** Avoid generic link text. “Click here for documentation” → “Read the documentation”
- **Opening new tabs:** Generally let users decide (Ctrl+click). Only force new tab for PDFs, external tools, or specific contexts
- **Icon links:** When link is icon-only, include accessible text label (aria-label or visually-hidden text)

#16 Button

The primary action trigger. Executes an action when activated.

Specializations: Icon Button (icon only, no label), Close Button (specialized icon button for dismissal), Circular Button (round shape variant), Button Group (set of related buttons sharing a border)

Naming note: Distinguish from **Link** — buttons trigger actions, links navigate.

Description

A button is an interactive element that triggers an action when activated. Buttons are one of the most fundamental UI elements, used for form submission, opening modals, toggling states, confirming choices, and countless other user-initiated actions. Proper button implementation ensures users understand what will happen when they click, and that the button responds appropriately.

Main Features

- **Action trigger:** Executes specified action when clicked, tapped, or activated via keyboard
- **Clear labeling:** Text or icon clearly indicates the action that will be performed
- **Visual states:** Distinct appearance for idle, hover, active (pressed), focus, disabled, and loading states
- **Variant styles:** Primary (high emphasis), secondary (medium emphasis), tertiary/ghost (low emphasis), destructive (dangerous actions)
- **Size variants:** Small, medium, large, and sometimes extra-large
- **Loading state:** Can display loading indicator during async operations, preventing duplicate clicks

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<button>` element with `type` attribute (button, submit, reset)
- **Keyboard accessible:** Activates with Enter or Space key
- **Focus indicator:** Clear visual indicator when button has keyboard focus (WCAG 2.4.7)
- **Disabled communication:** Disabled buttons have `aria-disabled` and don't receive focus
- **Loading announcement:** Loading state changes announced to screen readers (`aria-live`)
- **Icon-only buttons:** Include `aria-label` when button contains only icon

Keyboard Navigation

- **Tab navigation:** Can be reached and exited via Tab/Shift+Tab
- **Enter and Space:** Both keys activate button (Space may be prevented on links styled as buttons)
- **Disabled state:** When disabled, button cannot receive keyboard focus

Touch-screen

- **Touch target:** Minimum 44x44px tap target (iOS HIG) or 48x48px (Material Design)
- **Touch feedback:** Immediate visual feedback on tap (active state, ripple effect)
- **No hover reliance:** Doesn't require hover to reveal essential information

Responsive Behavior

- **Flexible width:** Can be fixed-width, content-width, or full-width (100% of container)
- **Text wrapping:** Long button labels wrap or truncate based on configuration
- **Icon scaling:** Icons within buttons scale proportionally with button size

- **Stacking on mobile:** Button groups may stack vertically on narrow screens

Validation & Feedback

- **Loading state:** Shows spinner/loader during async actions, disables button to prevent multiple clicks
- **Success indication:** Optionally shows success state briefly after action completes
- **Error feedback:** Can show error state if action fails
- **Disabled state:** Visually de-emphasized (reduced opacity, gray color) when not actionable
- **Interaction callbacks:** Notifies when clicked (onClick), focused (onFocus), blurred (onBlur)

Variants & Features

- **Primary button:** Most visually prominent, solid fill with brand color, for main CTAs
- **Secondary button:** Less prominent, outlined or muted color, for secondary actions
- **Tertiary/Ghost button:** Minimal styling, text-only or subtle background, for low-priority actions
- **Destructive button:** Red or warning color for delete, remove, or destructive actions
- **Icon button:** Contains only icon, no text label (requires accessible label)
- **Button with icon:** Text label with leading or trailing icon
- **Full-width button:** Expands to fill container width (common on mobile)
- **Button group:** Multiple related buttons sharing borders, acting as segmented control

Test Scenarios

- **Button renders:** Button displays with correct label and styling
- **Click activation:** Clicking button executes onClick callback
- **Keyboard activation:** Pressing Enter or Space when focused executes action

- **Visual states:** Button shows distinct hover, active, focus, disabled, and loading states
- **Focus indicator:** When focused via keyboard, clear focus ring appears
- **Disabled state:** When disabled, button is visually de-emphasized and doesn't respond to clicks or keyboard
- **Loading state:** When loading, button shows loader, disabled state, and prevents additional clicks
- **Size variants:** Button renders at correct size (small, medium, large)
- **Style variants:** Primary, secondary, tertiary, and destructive styles display correctly
- **Icon rendering:** Icon buttons show icon; buttons with icons show both icon and text in correct order
- **Full-width mode:** When full-width enabled, button expands to container width
- **Touch target:** Button has adequate touch target size (minimum 44x44px)
- **Screen reader label:** Icon-only buttons have accessible label via aria-label or visually-hidden text
- **Loading announcement:** When button enters loading state, change is announced to screen readers
- **Form submission:** Submit buttons correctly submit parent form; button type="button" doesn't submit

Icon Button Specialization

Icon Button contains only an icon with no visible text label. Common for toolbars, compact interfaces, or actions where space is limited.

Additional Features:

- **Accessible label:** Requires aria-label or tooltip describing the action
- **Tooltip on hover:** Shows tooltip with action description on hover/focus
- **Consistent sizing:** Icon buttons typically square (same width and height)

Additional Test Scenarios:

- **Icon displays:** Button shows specified icon centered in button

- **Accessible label:** Button has aria-label or accessible text describing action
- **Tooltip appears:** Hovering or focusing button shows tooltip with action description
- **Square shape:** Icon button is square (equal width and height)

Close Button Specialization

Close Button is a specialized icon button (typically ×) used to dismiss modals, alerts, or panels.

Additional Features:

- **Standard icon:** Uses × (times) or similar close icon
- **Consistent placement:** Typically top-right corner of container
- **Secondary styling:** Less prominent than other buttons (ghost/tertiary style)

Additional Test Scenarios:

- **Close icon:** Button shows × or close icon
- **Accessible label:** Has aria-label like “Close” or “Dismiss”
- **Position:** Positioned in top-right corner of parent container
- **Dismissal action:** Clicking triggers close/dismiss action

Circular Button Specialization

Circular Button is round instead of rectangular. Often used for floating action buttons (FAB) or icon buttons.

Additional Features:

- **Perfect circle:** Equal width and height with 50% border radius
- **Centered content:** Icon or text perfectly centered in circle
- **Often elevated:** May include shadow to appear floating

Additional Test Scenarios:

- **Circular shape:** Button is perfectly round (equal dimensions, 50% border radius)

- **Centered content:** Icon is perfectly centered within circle
- **Shadow rendering:** If elevated, shadow renders correctly

Button Group Specialization

Button Group combines multiple related buttons into a single visual component with shared borders.

Additional Features:

- **Shared borders:** Adjacent buttons share borders (no double borders)
- **Consistent sizing:** All buttons same height
- **Single selection:** Can act like radio buttons (only one selected) or multi-select (like checkboxes)
- **Orientation:** Horizontal (default) or vertical

Additional Test Scenarios:

- **Buttons connected:** Adjacent buttons share borders visually
- **Consistent height:** All buttons in group are same height
- **Selection state:** Selected button shows active/selected styling
- **Single-select mode:** When single-select, only one button can be selected at a time
- **Multi-select mode:** When multi-select, multiple buttons can be selected simultaneously
- **Orientation:** Group displays horizontally or vertically based on configuration

Notes

- **Button vs Link:** Use buttons for actions (submit, open, delete). Use links for navigation (go to page, jump to section)
- **Primary button per page:** Generally one primary button per section/page. Too many primaries create confusion
- **Destructive actions:** Use red/warning styling and require confirmation for irreversible actions (delete, remove)

- **Loading state UX:** Show loading state for actions >300ms. Disable button during loading to prevent double-clicks
- **Icon-only accessibility:** Always include accessible label. Tooltip alone isn't sufficient (not accessible to keyboard-only users)
- **Full-width on mobile:** Primary action buttons often full-width on mobile for easier tapping
- **Button order:** Place primary action on the right (or bottom in vertical layouts) in Western contexts; reverse for RTL

#17 Label

Form field label with support for required/optional indicators and association with its input. Distinct from *Label Text* (Typography) in that it carries form-specific semantics.

Description

A label provides a text description for a form input, clearly identifying what information the user should enter. Labels are critical for accessibility and usability—they ensure all users understand the purpose of each form field. Proper label implementation includes programmatic association with inputs, required field indicators, and optional helper text.

Main Features

- **Text display:** Shows descriptive text identifying the form field's purpose
- **Programmatic association:** Linked to input via `for` attribute matching input's `id`
- **Required indicator:** Visual marker (typically asterisk `*`) when field is mandatory
- **Optional indicator:** Text like “(optional)” for optional fields in mostly-required forms
- **Clickable activation:** Clicking label focuses associated input

Secondary Features

Accessibility

- **Explicit association:** Uses `<label>` element with `for` attribute linking to `input id`
- **Screen reader support:** Screen readers announce label text when input receives focus
- **Required announcement:** Required state is conveyed to screen readers (via asterisk or `aria-required`)
- **Clear language:** Label text is concise and clearly describes expected input

Responsive Behavior

- **Text wrapping:** Long label text wraps to multiple lines if needed
- **Flexible positioning:** Can be positioned above input (default), inline, or to the side
- **Mobile considerations:** Labels above inputs work better on narrow screens than side-by-side

Internationalization

- **Localized text:** Label text can be translated to user's language
- **RTL support:** Label and input align correctly in RTL layouts
- **Cultural conventions:** Respects local conventions for required indicators

Variants & Features

- **Position variants:** Above input (default), inline (beside), floating (moves on focus)
- **Required styling:** Asterisk, “(required)” text, or visual treatment
- **Optional styling:** “(optional)” text for optional fields when most are required
- **Helper text:** Additional descriptive text below label (separate from label itself)

Test Scenarios

- **Label renders:** Label displays with correct text
- **Programmatic association:** Label has for attribute matching associated input's id
- **Clickable activation:** Clicking label focuses associated input
- **Required indicator:** When input is required, label shows asterisk or “(required)” indicator
- **Optional indicator:** When input is optional (and configured to show), label shows “(optional)” text
- **Screen reader association:** Screen reader announces label text when input receives focus
- **Required announcement:** Screen reader announces required state (via asterisk or aria-required on input)
- **Text wrapping:** Long label text wraps to next line without breaking association
- **Position variants:** Label renders above, beside, or inline with input based on configuration
- **Floating label:** When floating variant, label moves/shrinks when input has value or focus
- **Multiple inputs:** When label is associated with multiple inputs (radio group), structure is semantically correct
- **RTL layout:** Label and input align correctly in right-to-left layouts

Notes

- **Always include labels:** Every form input needs a label. Placeholders are NOT replacements for labels
- **Placeholder vs label:** Placeholders disappear on input; labels remain visible. Use both when helpful
- **Required indicators:** Asterisk (*) is most common but consider “(required)” text for clarity
- **Mostly required vs mostly optional:** If most fields required, mark optional ones. If most optional, mark required ones

- **Floating labels:** Popular in Material Design, but can have accessibility concerns. Ensure label remains visible and associated
- **Label-less inputs:** In rare cases (search box with adjacent button), placeholder + aria-label can replace visible label, but visible labels are strongly preferred
- **Form field wrapper:** Often labels are part of a Form Field wrapper component that also includes the input, helper text, and error messages

#18 Form Field

Wrapper component that groups a label, an input, helper text, and error/success messages into a single cohesive unit. Ensures consistent layout and accessibility associations across all form inputs.

Description

Form Field is a compositional wrapper that bundles a label, input control, optional helper text, and validation messages into a standardized, accessible unit. It ensures consistent spacing, layout, and accessibility associations across all form inputs in an application. By centralizing form field structure, it reduces implementation errors and maintains visual and semantic consistency.

Main Features

- **Component composition:** Contains and arranges label, input, helper text, error/success messages
- **Consistent layout:** Ensures uniform spacing and alignment across all form fields
- **Accessibility associations:** Properly links label to input, error messages to input (aria-describedby)
- **State management:** Coordinates visual states (idle, focus, error, success, disabled) across child components
- **Flexible input slot:** Accepts any form input type (text, select, checkbox, etc.)

Secondary Features

Accessibility

- **Proper ARIA:** Error messages connected via aria-describedby, required via aria-required
- **Error announcement:** Error messages in aria-live region for screen reader announcement
- **Label association:** Label programmatically linked to input via for/id
- **Help text description:** Helper text connected to input via aria-describedby
- **Focus management:** Focus indicator on input is clear and consistent

Responsive Behavior

- **Flexible layout:** Adapts from horizontal (label beside input) to vertical (label above) on narrow screens
- **Error message stacking:** Error messages stack vertically without breaking layout
- **Full-width inputs:** Inputs can fill container width on mobile

Validation & Feedback

- **Error state:** Displays error message below input with error styling on input and label
- **Success state:** Shows success indicator and optional success message
- **Helper text:** Shows additional guidance when field is idle (hidden when error appears)
- **Real-time validation:** Supports showing validation as user types or on blur
- **Required indicator:** Label shows asterisk or “(required)” when field is mandatory

Variants & Features

- **Layout variants:** Vertical (label above), horizontal (label beside), floating label
- **Size variants:** Small, medium, large spacing and input sizes

- **Inline errors:** Error messages inline vs below input
- **Character counter:** Shows character count / max length

Test Scenarios

- **Field renders:** Form field displays with label, input, and optional helper text
- **Layout structure:** Label, input, helper text, and error message positioned correctly
- **Label association:** Label's for attribute matches input's id
- **Helper text:** When provided, helper text displays below input
- **Error state:** When error is present, error message displays, helper text hides, input shows error styling
- **Error ARIA:** Error message is connected to input via aria-describedby
- **Success state:** When success is indicated, success styling appears on input and optional message displays
- **Required indicator:** When required, label shows required indicator (asterisk or text)
- **Disabled state:** When disabled, label, input, and helper text show disabled styling
- **Focus state:** When input receives focus, field shows focus state visually
- **Character counter:** When max length is set, character counter shows "X / Y" below input
- **Responsive layout:** On narrow screens, label moves above input (if configured for horizontal)
- **Multiple inputs:** When field contains radio group or checkbox group, layout handles multiple inputs
- **Screen reader flow:** Screen reader user can navigate through label → input → helper/error text logically
- **Error announcement:** When error appears or changes, screen reader announces the error

Notes

- **Centralized form styling:** Using Form Field wrapper ensures all forms look consistent without repeating structure

- **Accessibility by default:** Form Field handles ARIA associations, reducing errors in individual form implementations
- **Validation library integration:** Works well with validation libraries (Formik, React Hook Form, Zod) that provide error messages
- **Layout flexibility:** Some systems offer multiple Form Field variants (vertical, horizontal, floating) for different contexts
- **Error message patterns:** Consider showing errors on blur (not while typing) to reduce noise, unless field has specific real-time requirements
- **Helper text vs error:** Helper text disappears when error appears (less clutter). Some systems show both simultaneously
- **Form-level errors:** Form Field handles field-level errors; form-level errors (submission failures) shown separately

#19 Text Input

Single-line text entry field. The foundational form input component.

Specializations: Email Input (email keyboard hint + validation hint), Number Input (numeric keyboard, optional min/max/step), Money Input (currency formatting), Search Input (search icon, clear button, no drop pane)

***Naming note:** Search Input here is the simple input field only. For search with a results dropdown, see **Combobox** (41).*

Description

A single-line text input field that allows users to enter and edit text data. This is one of the most fundamental form controls, used for collecting short textual information like names, emails, search queries, and other single-line text content.

Main Features

- **Editable text field:** User can type, delete, and modify text content

- **Value state management:** Component maintains and exposes current text value
- **Focus management:** Can receive and lose focus through user interaction
- **Visual state feedback:** Clearly distinguishes between idle, focused, disabled, error, and success states
- **Label association:** Supports a visible label that clearly identifies the input's purpose
- **Placeholder text:** Optional hint text that disappears when user begins typing
- **Required field indication:** Visual marker when input is mandatory

Secondary Features

Accessibility

- **Programmatic label association:** Label is programmatically linked to input for screen reader announcement
- **ARIA attributes support:** Accepts description, error message, and required state attributes
- **Keyboard accessible:** Fully operable via keyboard alone
- **Focus indicator:** Clear visual indicator when input has keyboard focus (respects system preferences)

Keyboard Navigation

- **Tab navigation:** Can be reached and exited via Tab/Shift+Tab
- **Text selection shortcuts:** Standard selection shortcuts work (Ctrl+A, Shift+Arrow, etc.)
- **Enter key handling:** Component notifies when Enter is pressed (parent decides action)
- **Escape key handling:** Component notifies when Escape is pressed (can clear or blur field)

Touch-screen

- **Touch-friendly target:** Adequate size for touch interaction (minimum 44x44px or platform standard)
- **Native keyboard invocation:** Activates appropriate on-screen keyboard when focused
- **Input type hint:** Can specify input type to trigger appropriate keyboard (text, email, tel, url, number)

Responsive Behavior

- **Flexible width:** Adapts to container width constraints
- **Text overflow handling:** Long text scrolls horizontally or handles overflow gracefully
- **Readable font sizing:** Maintains legible text size across different screen sizes (minimum 16px to avoid auto-zoom on mobile)

Internationalization

- **Bi-directional text support:** Handles RTL (right-to-left) languages correctly
- **Multi-byte character support:** Properly handles characters from various language sets (emoji, accented characters, CJK)
- **Locale-appropriate placeholders:** Placeholder and label text can be localized

Validation & Feedback

- **Error state display:** Accepts and displays error state with associated error message
- **Success state display:** Accepts and displays success state with optional success message
- **Visual state feedback:** Changes appearance based on provided state (error, success, idle)

- **Interaction callbacks:** Notifies when user interactions occur (onChange, onBlur, onFocus) so external logic can respond
- **Character count display:** Can display current/maximum character count when limit information is provided

Test Scenarios

- **Basic input:** Typing characters updates the visible value and triggers onChange callback
- **Focus behavior:** Clicking/tapping into field applies focus state; clicking outside or tabbing away removes it
- **Disabled state:** When disabled, field does not accept input, focus, or trigger callbacks
- **Required validation display:** When marked required, displays visual indicator (typically asterisk or “required” label)
- **Placeholder visibility:** Placeholder text is visible when empty; disappears when user types first character
- **Character limit enforcement:** When max length is set, input prevents exceeding that length
- **Error state display:** When error state is true, error styling is applied and error message is displayed
- **Success state display:** When success state is true, success styling is applied
- **Label association:** Clicking associated label focuses the input
- **Keyboard navigation:** Tab key moves focus to/from input; text editing shortcuts work as expected
- **Value persistence:** Entered value is maintained and accessible via getValue or similar mechanism
- **Callback execution:** onChange fires on text change, onBlur fires on focus loss, onFocus fires on focus gain
- **RTL text handling:** Right-to-left text displays correctly when input direction is RTL

Notes

- **Auto-focus:** Whether inputs should auto-focus on page/screen load is context-dependent and should be decided at implementation level
- **Auto-complete:** Browser/system autocomplete behavior may vary; component should allow enabling/disabling autocomplete suggestions
- **Input masking/formatting:** Formatted inputs (phone numbers, credit cards, currency) are considered specialized variants that extend base text-input
- **Password inputs:** Password visibility toggle is a specialized variant
- **Search inputs:** Search with icon/clear button is a specialized variant
- **Numeric inputs:** Inputs with spinner controls or numeric keyboard are specialized variants

#20 Password Input

Text input with masked characters and a show/hide visibility toggle.

Description

Password Input is a specialized text input that masks entered characters for security and privacy, preventing shoulder-surfing and accidental exposure. It includes a visibility toggle allowing users to reveal their password temporarily to verify accuracy. This component is essential for login forms, registration flows, and password change interfaces.

Main Features

- **Masked input:** Entered characters display as bullets (•) or asterisks (*) instead of actual text
- **Visibility toggle:** Button or icon to temporarily show/hide password text
- **All text input features:** Inherits label, error states, validation from base text input

- **Paste support:** Allows pasting passwords (typically from password managers)
- **Auto-complete support:** Works with browser password managers

Secondary Features

Accessibility

- **Toggle label:** Visibility toggle has clear label (“Show password” / “Hide password”)
- **State announcement:** When toggled, screen reader announces new state
- **Input type:** Uses `type="password"` for semantic meaning and browser behavior
- **Auto-complete attribute:** Includes `autocomplete="current-password"` or `“new-password”` for password managers

Keyboard Navigation

- **Tab navigation:** Tab moves through input → visibility toggle → next field
- **Toggle activation:** Enter or Space on focused toggle button reveals/hides password
- **Keyboard shortcuts:** Ctrl+C (copy) works when password is visible

Touch-screen

- **Toggle button size:** Visibility toggle is large enough for easy tapping (44x44px minimum)
- **Toggle position:** Typically positioned at right edge of input, easily reachable
- **Native keyboard:** Triggers secure password keyboard on mobile devices

Responsive Behavior

- **Mobile keyboards:** Invokes secure keyboard with masked input on mobile
- **Toggle positioning:** Toggle remains accessible even on narrow screens
- **Text truncation:** Very long passwords scroll horizontally within input

Internationalization

- **Localized labels:** “Show password” / “Hide password” can be localized
- **Multi-byte support:** Handles passwords with non-ASCII characters (emoji, CJK, accented characters)

Validation & Feedback

- **Strength indicator:** Optional password strength meter (weak, medium, strong)
- **Requirements display:** Shows password requirements (length, special characters, etc.)
- **Real-time validation:** Validates requirements as user types
- **Error states:** Displays error for incorrect current password or invalid new password

Test Scenarios

- **Input masking:** Typed characters display as bullets or asterisks
- **Visibility toggle renders:** Toggle button/icon displays beside input
- **Toggle shows password:** Clicking toggle reveals password text (type changes to “text”)
- **Toggle hides password:** Clicking toggle again masks password (type changes to “password”)
- **Toggle label updates:** Toggle button label changes between “Show” and “Hide”
- **Screen reader announcement:** Toggling announces new state to screen readers
- **Password manager integration:** Browser password manager can autofill password
- **Paste support:** Pasting password works correctly (characters are masked)
- **Auto-complete attribute:** Input has appropriate autocomplete value
- **Keyboard toggle:** Pressing Enter/Space on focused toggle button reveals/hides password

- **Mobile keyboard:** On mobile, secure password keyboard appears when input is focused
- **Strength indicator:** When enabled, strength meter updates as user types
- **Requirements validation:** Password requirements (length, characters) are checked and displayed
- **Tab order:** Tab moves from input to toggle button to next field
- **Toggle icon:** Icon changes between eye (show) and eye-with-slash (hide)

Notes

- **Always include toggle:** Visibility toggle prevents typos and improves UX without compromising security
- **Default state:** Password should be masked by default; toggle reveals it temporarily
- **Security considerations:** Toggling doesn't reduce security—modern threats are digital, not shoulder-surfing
- **Password managers:** Support browser password managers with proper autocomplete attributes
- **Strength indicators:** Show strength for new password creation, not for login
- **Requirements:** Show password requirements before user types (not as errors). Check requirements in real-time
- **Current vs new:** Use autocomplete="current-password" for login, "new-password" for registration/change
- **Paste allowance:** Don't block pasting—it prevents use of password managers and strong generated passwords
- **Toggle position:** Typically at right edge of input. Ensure it doesn't obscure input text

#21 PIN / OTP Input

Multi-cell input for entering short numeric or alphanumeric codes (4–8 characters). Each character occupies its own cell. Also called *Code Input* or *OTP Input*.

Description

PIN/OTP Input is a specialized input for entering short verification codes, PINs, or one-time passwords. Instead of a single text field, it presents individual cells for each character, making it clear how many digits are expected and providing visual feedback for each entered character. Common in two-factor authentication, SMS verification, and security PIN entry.

Main Features

- **Multi-cell display:** Shows separate box/cell for each character (typically 4-6 cells)
- **Auto-advance:** Automatically moves focus to next cell after character entry
- **Numeric or alphanumeric:** Accepts numbers only or both numbers and letters
- **Visual feedback:** Each cell shows entered character or remains empty
- **Paste support:** Allows pasting complete code, distributing characters across cells

Secondary Features

Accessibility

- **Single input semantics:** Screen readers treat as single input with expected length announced
- **Cell grouping:** Cells grouped with role="group" and accessible label
- **Error announcement:** Errors announced to screen readers (invalid code, expired code)
- **Auto-submit:** Optional auto-submission when all cells filled (announced to screen readers)

Keyboard Navigation

- **Auto-advance on entry:** After entering character, focus moves to next empty cell

- **Backspace behavior:** Backspace deletes current cell and moves to previous cell
- **Arrow navigation:** Left/right arrows move between cells
- **Tab navigation:** Tab moves out of component to next field (not between cells)
- **Paste handling:** Ctrl+V pastes code, distributing characters across cells

Touch-screen

- **Large tap targets:** Each cell has adequate size for easy tapping (minimum 44x44px)
- **Numeric keyboard:** Invokes numeric keyboard when input is numeric-only
- **Visual feedback:** Tapped cell shows focus state immediately
- **Auto-complete:** Works with SMS auto-fill on mobile devices

Responsive Behavior

- **Cell sizing:** Cells scale appropriately for screen size (smaller on mobile)
- **Spacing:** Consistent gap between cells that adapts to container width
- **Font size:** Character size is large and readable (minimum 24px recommended)

Internationalization

- **RTL support:** Cells flow right-to-left in RTL languages
- **Localized errors:** Error messages can be localized
- **Numeric vs alphanumeric:** Configuration for numeric-only (most common) or alphanumeric codes

Validation & Feedback

- **Character validation:** Restricts input to allowed characters (numeric or alphanumeric)
- **Complete validation:** Validates entire code when all cells filled
- **Error state:** Shows error styling on all cells when code is invalid

- **Auto-submission:** Optionally submits form automatically when complete code entered
- **Expiration handling:** Can show countdown timer for time-limited codes (OTP)

Test Scenarios

- **Cells render:** Correct number of cells display (4, 5, 6, or custom)
- **Character entry:** Typing character fills current cell and advances to next
- **Auto-advance:** After entering character, focus automatically moves to next empty cell
- **Backspace delete:** Pressing Backspace deletes current cell and moves focus to previous cell
- **Arrow navigation:** Left arrow moves to previous cell, right arrow to next cell
- **Paste distribution:** Pasting complete code (e.g., “123456”) fills all cells correctly
- **Paste partial:** Pasting fewer characters than cells fills cells from current position
- **Numeric restriction:** When numeric-only, input rejects alphabetic characters
- **Alphanumeric support:** When alphanumeric, accepts both numbers and letters
- **Complete state:** When all cells filled, component triggers onComplete callback
- **Error state:** When validation fails, all cells show error styling
- **Error announcement:** Screen reader announces error when validation fails
- **Auto-submit:** When enabled, form submits automatically when last cell is filled
- **SMS auto-fill:** On mobile, SMS codes auto-populate cells when received
- **Mobile keyboard:** Numeric keyboard appears when input is numeric-only
- **Cell focus:** Clicking cell moves focus to that cell
- **Tab exit:** Tab key moves focus out of component, not between cells
- **RTL layout:** In RTL mode, cells flow right-to-left

Notes

- **Single input vs multiple:** Implement as single input with visual styling, not separate inputs (better for screen readers)
- **Code length:** Most common is 6 digits (SMS OTP). 4 digits for PINs. Rarely more than 8
- **Auto-submit:** Auto-submit is convenient but ensure users can review code before submission
- **Paste support essential:** Many users copy codes from SMS/email—blocking paste hurts UX
- **SMS auto-fill:** Use autocomplete=“one-time-code” for iOS SMS auto-fill
- **Expiration timers:** For OTP codes, show countdown timer. Provide “resend code” option
- **Character display:** Some implementations mask characters after brief delay (like password input)
- **Validation timing:** Validate on completion (all cells filled) rather than per-character
- **Error recovery:** When code is wrong, clear all cells and refocus first cell for easy retry

#22 Textarea

Multi-line text entry field with optional resize behavior.

Description

Textarea is a multi-line text input designed for longer content like comments, descriptions, messages, or any text that typically exceeds one line. Unlike single-line text inputs, textareas expand vertically to accommodate multiple lines and often include resizing controls. They're essential for forms that collect detailed user feedback, content creation, or any substantial text input.

Main Features

- **Multi-line input:** Allows text to wrap to multiple lines
- **Vertical scrolling:** Scrolls vertically when content exceeds visible area
- **Resize control:** User can drag corner/edge to resize (unless disabled)
- **All text input features:** Inherits label, placeholder, error states, validation from base text input patterns
- **Character counting:** Can display current/maximum character count
- **Minimum height:** Starts with configured minimum height (e.g., 3-4 lines visible)

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<textarea>` element
- **Label association:** Programmatically linked to label via `for/id`
- **ARIA attributes:** Supports `aria-describedby` for helper text and errors
- **Resize announcement:** Screen readers may announce resize capability
- **Character limit:** When max length is set, announced to screen readers

Keyboard Navigation

- **Tab navigation:** Tab moves to/from textarea
- **Enter creates new line:** Enter key inserts line break (unlike text input where it might submit form)
- **Text selection:** Standard selection shortcuts work (Ctrl+A, Shift+Arrow)
- **Tab key behavior:** Tab can be configured to insert tab character or move focus (typically moves focus)

Touch-screen

- **Touch target:** Entire textarea area is tappable (minimum height 44px)
- **Resize handle:** Resize handle is large enough for touch manipulation
- **Native keyboard:** Activates appropriate keyboard with multi-line support

- **Scroll support:** Touch scrolling works when content overflows

Responsive Behavior

- **Flexible width:** Adapts to container width
- **Minimum height:** Maintains readable minimum height (at least 3-4 lines)
- **Resize constraints:** Can constrain resizing to vertical-only, horizontal-only, both, or none
- **Auto-resize:** Optionally grows automatically as user types (eliminates need for scrolling)

Internationalization

- **Bi-directional text:** Handles RTL languages correctly
- **Multi-byte characters:** Properly handles emoji, accented characters, CJK
- **Line-breaking:** Respects language-specific line-breaking rules

Validation & Feedback

- **Error state:** Displays error message and error styling
- **Success state:** Shows success indicator when validation passes
- **Character count:** Shows current character count and/or remaining characters
- **Minimum length:** Can validate minimum character requirement
- **Maximum length:** Enforces maximum character limit
- **Word count:** Optionally displays word count instead of character count

Test Scenarios

- **Textarea renders:** Component displays as multi-line text field
- **Multi-line input:** Pressing Enter creates new line without submitting form
- **Vertical scrolling:** When content exceeds height, textarea scrolls vertically
- **Resize capability:** User can drag resize handle to change dimensions (unless disabled)

- **Resize constraints:** When resize is constrained (vertical-only, etc.), resizing respects constraints
- **Disabled resize:** When resize is disabled, resize handle doesn't appear
- **Auto-resize:** When auto-resize enabled, textarea grows vertically as user types
- **Character count display:** Current character count displays and updates as user types
- **Character limit enforcement:** When max length is set, input stops accepting characters at limit
- **Character count accuracy:** Character count correctly handles multi-byte characters (emoji count as 1, not more)
- **Minimum height:** Textarea starts with configured minimum height (shows 3-4 lines minimum)
- **Label association:** Clicking associated label focuses textarea
- **Placeholder text:** Placeholder displays when empty and disappears when user types
- **Error state:** Error styling applied and error message displays when validation fails
- **Tab behavior:** Tab key moves focus out of textarea (doesn't insert tab character by default)
- **Text selection:** Standard selection shortcuts work (Ctrl+A selects all, Shift+Arrow selects text)
- **Paste handling:** Pasting multi-line content preserves line breaks
- **RTL support:** Text aligns correctly in RTL mode
- **Mobile keyboard:** On mobile, appropriate keyboard appears with return/enter key for new lines

Notes

- **Resize default:** Default to vertical-only resize—horizontal resize often breaks layout
- **Auto-resize:** Auto-grow feature eliminates scrolling and makes form feel more natural. Ensure max-height to prevent excessive growth

- **Character counting:** For user-facing content (tweets, bio), count what users see (emoji = 1). For technical limits (database), count bytes accurately
- **Minimum height:** Start with at least 3-4 lines visible. Single-line textarea defeats the purpose
- **Form submission:** Unlike text input, Enter should NOT submit form—it creates new line
- **Tab key:** Default tab behavior (move focus) is usually correct. If you allow tab character insertion, provide escape hatch (Ctrl+Tab to move focus)
- **Accessibility of resize:** Some screen readers don't announce resize capability. Consider providing resize buttons (+/-) for accessibility
- **Max height:** Consider max-height for auto-resize textareas to prevent infinite growth

#23 Checkbox

Binary selection control (checked, unchecked, indeterminate).

Specialization: Checkbox Group (set of related checkboxes with shared label)

Description

A checkbox allows users to select or deselect an option, make multiple selections from a list, or indicate agreement with terms. Unlike radio buttons (where only one can be selected), checkboxes support independent selections. Checkboxes also support an indeterminate state for “partially checked” scenarios like nested selections where some but not all children are selected.

Main Features

- **Binary selection:** Can be checked (selected) or unchecked (unselected)
- **Indeterminate state:** Third visual state indicating partial selection (for parent items)
- **Label association:** Clickable label that toggles checkbox state

- **Independent selection:** Multiple checkboxes can be checked simultaneously
- **Keyboard accessible:** Can be toggled via keyboard

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<input type="checkbox">` element
- **Label association:** Programmatically linked to label via `for/id` or wrapped in `<label>`
- **State announcement:** Screen readers announce checked, unchecked, or indeterminate state
- **Group semantics:** Multiple related checkboxes grouped with `fieldset/legend` or `role="group"`
- **Keyboard accessible:** Can be focused and toggled via keyboard

Keyboard Navigation

- **Tab navigation:** Tab moves to/from checkbox in document order
- **Space toggle:** Pressing Space toggles checked state
- **Group navigation:** In checkbox group, Tab moves between checkboxes (not arrow keys like radio buttons)
- **Disabled state:** Disabled checkboxes cannot receive focus

Touch-screen

- **Touch target:** Checkbox and label together provide minimum 44x44px tap target
- **Entire label clickable:** Tapping label text toggles checkbox
- **Visual feedback:** Immediate visual feedback on tap

Responsive Behavior

- **Flexible sizing:** Checkbox size can scale (small, medium, large)

- **Label wrapping:** Long label text wraps without breaking checkbox-label association
- **Stacking:** Checkbox groups stack vertically by default

Internationalization

- **RTL support:** Checkbox position adjusts for RTL languages (appears on right side of label)
- **Localized labels:** Label text can be translated

Validation & Feedback

- **Required validation:** Can be marked required (must be checked to proceed)
- **Error state:** Shows error styling when validation fails
- **Checked state:** Provides checked state in form data
- **Interaction callbacks:** Notifies when state changes (onChange, onCheck, onUncheck)

Test Scenarios

- **Checkbox renders:** Checkbox displays with associated label
- **Unchecked state:** Initially unchecked checkbox is empty/hollow
- **Check action:** Clicking checkbox or label toggles from unchecked to checked
- **Uncheck action:** Clicking checked checkbox toggles back to unchecked
- **Visual states:** Checked shows checkmark/fill, unchecked is empty, indeterminate shows dash/minus
- **Indeterminate state:** When set to indeterminate, displays partial selection indicator
- **Label click:** Clicking label text toggles checkbox state
- **Keyboard focus:** Tab moves focus to checkbox with visible focus indicator
- **Space toggle:** Pressing Space on focused checkbox toggles state
- **Screen reader state:** Screen reader announces “checked”, “unchecked”, or “indeterminate”
- **Disabled state:** Disabled checkbox is visually de-emphasized and cannot be toggled

- **Required validation:** When required and unchecked, shows error state
- **Error styling:** Error state applies distinct visual treatment
- **Group association:** Related checkboxes grouped with accessible label
- **onChange callback:** Callback fires with new state when checkbox is toggled
- **Form submission:** Checked checkboxes include their value in form data
- **RTL layout:** Checkbox appears on right side of label in RTL mode

Checkbox Group Specialization

Checkbox Group presents multiple related checkboxes under a shared group label, allowing users to select zero, one, or multiple options from the set.

Additional Features:

- **Group label:** Shared label/legend for the entire group
- **Select all option:** Optional “select all” checkbox that controls all group members
- **Indeterminate parent:** Parent “select all” checkbox shows indeterminate when some (not all) children checked
- **Vertical/horizontal layout:** Checkboxes arranged vertically (default) or horizontally

Additional Test Scenarios:

- **Group label renders:** Group has accessible label (fieldset/legend or aria-labelledby)
- **Multiple selection:** Multiple checkboxes in group can be checked simultaneously
- **Select all functionality:** Clicking “select all” checkbox checks all children
- **Deselect all:** Unchecking “select all” when all are checked unchecks all children
- **Indeterminate parent:** When some (not all) children checked, parent shows indeterminate state
- **Parent click behavior:** Clicking indeterminate parent checks all children
- **Layout rendering:** Checkboxes stack vertically or arrange horizontally based on configuration

- **Group validation:** Validation can require minimum/maximum selections from group

Notes

- **Checkbox vs Radio:** Use checkbox for multiple selections or single on/off. Use radio for mutually exclusive single selection
- **Checkbox vs Switch:** Checkbox for form selections (saved on submit). Switch for immediate state changes (like settings)
- **Label is essential:** Every checkbox needs a label. Label-less checkboxes are inaccessible
- **Indeterminate state:** Cannot be set via HTML attribute—must be set via JavaScript. Used for tree views or “select all” scenarios
- **Required checkboxes:** Common for “I agree to terms” checkboxes. Clearly label as required
- **Group vs individual:** Related options should be grouped (fieldset/legend) for screen readers
- **Checkbox size:** Ensure checkbox itself is large enough (minimum 16x16px) in addition to overall touch target

#24 Radio Button

Single selection from a set of mutually exclusive options.

Specialization: Radio Group (set of radio buttons with shared label and single selection)

Description

Radio buttons allow users to select exactly one option from a set of mutually exclusive choices. When one radio button in a group is selected, any previously selected button is automatically deselected. Radio buttons are essential for forms where users must choose from predefined options like payment methods, shipping options, or preference selections.

Main Features

- **Mutually exclusive selection:** Only one radio button in a group can be selected at a time
- **Persistent selection:** Once a radio button is selected, one must always remain selected (no unselect all)
- **Label association:** Clickable label toggles radio button selection
- **Group behavior:** Radio buttons with same name attribute form a selection group
- **Visual distinction:** Selected shows filled circle, unselected shows empty circle

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<input type="radio">` element
- **Label association:** Programmatically linked to label via `for/id` or wrapped in `<label>`
- **Group semantics:** Radio groups wrapped in `<fieldset>` with `<legend>` or `role="radiogroup"`
- **State announcement:** Screen readers announce checked/unchecked and position in group
- **Keyboard accessible:** Can be navigated and selected via keyboard

Keyboard Navigation

- **Tab navigation:** Tab moves to selected radio in group (or first if none selected), then out of group
- **Arrow key selection:** Arrow keys (`↑↓` or `←→`) navigate and select within group
- **Space selection:** Space selects focused radio button
- **Circular navigation:** Arrow keys wrap from last to first option

Touch-screen

- **Touch target:** Radio button and label together provide minimum 44x44px tap target
- **Entire label clickable:** Tapping label text selects that radio button
- **Visual feedback:** Immediate visual feedback on tap
- **Adequate spacing:** Sufficient space between adjacent radio buttons to prevent mis-taps

Responsive Behavior

- **Flexible sizing:** Radio button size can scale (small, medium, large)
- **Label wrapping:** Long label text wraps without breaking radio-label association
- **Stacking:** Radio groups typically stack vertically (horizontal layout optional)

Internationalization

- **RTL support:** Radio button position adjusts for RTL languages (appears on right side of label)
- **Localized labels:** Label text can be translated

Validation & Feedback

- **Required validation:** Radio group can be marked required (one option must be selected)
- **Error state:** Shows error styling when validation fails (no selection made)
- **Selected state:** Provides selected value in form data
- **Interaction callbacks:** Notifies when selection changes (onChange, onSelect)

Test Scenarios

- **Radio renders:** Radio button displays with associated label
- **Unselected state:** Initially unselected radio is empty circle
- **Selection:** Clicking radio button or label selects that option

- **Mutual exclusivity:** Selecting one radio deselects previously selected radio in same group
- **Visual distinction:** Selected radio shows filled/checked appearance
- **Label click:** Clicking label text selects associated radio button
- **Keyboard focus:** Tab moves focus to selected radio (or first if none selected)
- **Arrow navigation:** Arrow keys move selection through radio group
- **Arrow wrapping:** Arrows wrap from last option to first and vice versa
- **Space selection:** Pressing Space selects focused radio button
- **Screen reader announcement:** Screen reader announces option label, selected state, and position (e.g., “2 of 4”)
- **Disabled option:** Disabled radio is visually de-emphasized and cannot be selected or focused
- **Required validation:** When required and no selection made, shows error state
- **onChange callback:** Callback fires with new value when selection changes
- **Form submission:** Selected radio button value is included in form data
- **RTL layout:** Radio button appears on right side of label in RTL mode

Radio Group Specialization (Default Usage)

Radio Group is the standard implementation—multiple radio buttons grouped together for single selection. Individual radio buttons rarely exist in isolation.

Additional Features:

- **Group label:** Shared label/legend for entire group (e.g., “Shipping method:”)
- **Default selection:** Can specify which option is selected by default
- **Horizontal/vertical layout:** Options arranged vertically (default) or horizontally
- **Disabled options:** Individual options within group can be disabled

Additional Test Scenarios:

- **Group label renders:** Group has accessible label (fieldset/legend or aria-labelledby)

- **Single selection enforced:** Selecting any option deselects all others in group
- **Default selection:** When default value provided, correct radio is initially selected
- **Group validation:** Required validation applies to group (at least one must be selected)
- **Mixed disabled state:** Some options can be disabled while others remain enabled
- **Layout rendering:** Radios stack vertically or arrange horizontally based on configuration
- **Name attribute grouping:** All radio buttons in group share same name attribute

Notes

- **Radio vs Checkbox:** Radio for mutually exclusive single choice. Checkbox for independent multiple selections
- **Radio vs Dropdown:** Use radio when options are few ($\leq 5-7$) and should all be visible. Use dropdown for longer lists
- **Always use groups:** Individual radio buttons are rarely useful—they exist in groups
- **Pre-select default:** Unlike checkboxes, radio groups should typically have a default selection (prevents validation errors)
- **Cannot deselect:** Once selected, a radio group always has one selected option. If “none” is valid, include “None” as an explicit option
- **Fieldset/legend:** Use `<fieldset>` and `<legend>` for radio groups for proper accessibility
- **Arrow keys vs Tab:** Arrow keys navigate within group; Tab moves between groups
- **Horizontal layouts:** Use cautiously—vertical is more scannable and accessible

#25 Switch

Binary on/off control styled as a sliding toggle.

Specializations: Two-Way Button (binary choice styled as two adjacent buttons), Toggle Group (select one or more from a set of toggle buttons)

Naming note: Also called *Toggle* in many libraries. We use *Switch* to avoid confusion with the broader concept of toggling.

Description

A switch is a binary control that toggles between on/off states, visually represented as a sliding toggle similar to physical light switches. Unlike checkboxes (which represent selections in forms), switches trigger immediate state changes—they're commonly used for settings, preferences, or feature toggles. The visual metaphor makes the current state and available action immediately clear.

Main Features

- **Binary state:** On (active) or off (inactive)
- **Immediate effect:** Change takes effect instantly (not on form submission)
- **Visual metaphor:** Sliding toggle or pill shape with thumb that moves between positions
- **Clear state indication:** Visual distinction between on and off states
- **Label association:** Descriptive label explains what the switch controls

Secondary Features

Accessibility

- **Semantic role:** Uses `role="switch"` with `aria-checked` attribute
- **State announcement:** Screen readers announce “on” or “off” state
- **Label association:** Programmatically linked to descriptive label
- **Keyboard accessible:** Can be toggled via keyboard

- **Focus indicator:** Clear visual indicator when switch has keyboard focus

Keyboard Navigation

- **Tab navigation:** Tab moves to/from switch in document order
- **Space toggle:** Pressing Space toggles between on and off
- **Enter toggle:** Enter also toggles state (platform-dependent)
- **Disabled state:** Disabled switches cannot receive focus

Touch-screen

- **Touch target:** Entire switch provides minimum 44x44px tap target
- **Drag support:** Optional: user can drag thumb between positions
- **Tap anywhere:** Tapping anywhere on switch toggles state
- **Visual feedback:** Immediate visual feedback on interaction

Responsive Behavior

- **Flexible sizing:** Switch can scale (small, medium, large)
- **Label positioning:** Label can be positioned before or after switch
- **Compact mode:** Smaller switches for dense interfaces

Internationalization

- **RTL support:** Thumb position mirrors for RTL (on = left in RTL)
- **Localized labels:** Label text can be translated
- **On/off indicators:** Optional on/off text within switch can be localized

Animation & Transitions

- **Smooth transition:** Thumb slides smoothly between positions
- **State change animation:** Background color transitions smoothly
- **Reduced motion:** Respects prefers-reduced-motion with instant state change
- **Tactile feedback:** Optional haptic feedback on mobile devices

Validation & Feedback

- **Loading state:** Optional loading indicator during async state changes
- **Error handling:** If state change fails, reverts to previous state with error message
- **Interaction callbacks:** Notifies when state changes (onChange, onToggle)
- **Disabled state:** Visually de-emphasized when not interactive

Test Scenarios

- **Switch renders:** Switch displays in off state by default (unless configured otherwise)
- **Off state visual:** Off state shows thumb on left, typically gray or muted color
- **On state visual:** On state shows thumb on right, typically brand or accent color
- **Toggle to on:** Clicking/tapping switch toggles from off to on
- **Toggle to off:** Clicking on switch toggles back to off
- **Smooth animation:** Thumb slides smoothly between positions (unless reduced motion)
- **Label click:** Clicking label toggles switch state
- **Keyboard focus:** Tab moves focus to switch with visible focus indicator
- **Space toggle:** Pressing Space on focused switch toggles state
- **Screen reader state:** Screen reader announces “on” or “off” and control name
- **Disabled state:** Disabled switch is visually de-emphasized and cannot be toggled
- **Loading state:** When loading, switch shows loading indicator and prevents interaction
- **onChange callback:** Callback fires with new state when switch is toggled
- **RTL layout:** In RTL mode, on position is on left, off position on right
- **Drag interaction:** If drag is supported, user can drag thumb to change state
- **Reduced motion:** When user prefers reduced motion, state changes instantly without animation

Two-Way Button Specialization

Two-Way Button presents binary choice as two adjacent buttons (like Yes/No, Light/Dark, List/Grid). Only one can be selected at a time.

Additional Features:

- **Button-styled options:** Two buttons side-by-side, one selected at a time
- **Equal sizing:** Both options typically same width for visual balance
- **Text labels:** Each option has clear text label (not just icons)
- **Icon support:** Optional icons alongside or instead of text

Additional Test Scenarios:

- **Both buttons render:** Two options display side-by-side
- **Initial selection:** One option is selected by default
- **Selection toggle:** Clicking unselected option selects it and deselects the other
- **Visual distinction:** Selected option has distinct styling (filled, bold, or highlighted)
- **Equal width:** Both buttons are same width (visual balance)
- **Keyboard navigation:** Tab moves to selected button; Space/Enter toggles to other option

Toggle Group Specialization

Toggle Group allows selection of one or more options from a set of toggle-style buttons. Can function as radio buttons (single select) or checkboxes (multi-select).

Additional Features:

- **Multiple options:** More than two options (unlike Two-Way Button)
- **Single or multi-select:** Can restrict to single selection or allow multiple
- **Icon-only variant:** Options can be icon-only (with accessible labels)
- **Size and spacing:** Consistent button sizing and spacing

Additional Test Scenarios:

- **All options render:** All toggle options display
- **Single-select mode:** When single-select, selecting one deselects others
- **Multi-select mode:** When multi-select, multiple options can be selected simultaneously
- **No selection:** In multi-select mode, all options can be deselected
- **Visual states:** Selected and unselected options have clear visual distinction
- **Keyboard navigation:** Arrow keys navigate between options in group
- **Icon-only labels:** Icon-only toggles have accessible text labels

Notes

- **Switch vs Checkbox:** Switch for immediate settings (dark mode, notifications). Checkbox for form selections saved on submit
- **State clarity:** Switch's physical metaphor makes current state obvious. Checkbox state can be ambiguous visually
- **Undo difficulty:** Since switches take effect immediately, consider confirmations for destructive toggles
- **Loading states:** For switches that trigger server calls, show loading state during operation
- **Label text:** Write labels that clearly describe what “on” means. “Enable notifications” not “Notifications”
- **On/off text:** Optional on/off labels within switch can help clarity but aren't required with good label
- **Mobile usage:** Switches are touch-friendly and common on mobile settings screens
- **Accessibility roles:** Use role=“switch”, not checkbox, for proper semantics

#26 Segmented Control

Compact control for selecting one option from a small, fixed set. Similar to Tabs but used for option selection rather than content switching. Can be oriented horizontally (default) or vertically (common on mobile).

***Naming note:** Also called *Button Group* or *Toggle Group* in some libraries. Distinct from *Tabs* (content switching) and *Radio Group* (form selection).*

Description

Segmented Control presents a small set of mutually exclusive options (typically 2-5) in a compact, connected button group. Unlike radio buttons (which save selection on form submit) or tabs (which switch content panels), segmented controls typically apply changes immediately and control view modes, filters, or preferences. Common uses include list/grid view toggles, time period selectors (day/week/month), and chart type selectors.

Main Features

- **Mutually exclusive selection:** Only one option can be selected at a time
- **Compact presentation:** All options visible and accessible simultaneously
- **Connected appearance:** Options visually connected (shared borders) as single unit
- **Immediate effect:** Selection typically applies instantly (not on form submit)
- **Equal sizing:** Options typically equal width for visual harmony
- **Small option count:** Works best with 2-5 options

Secondary Features

Accessibility

- **Role semantics:** Can use role="radiogroup" or tab-like semantics depending on use case

- **State announcement:** Screen readers announce selected option
- **Keyboard accessible:** Can navigate and select options via keyboard
- **Label:** Group has accessible label describing the choice being made

Keyboard Navigation

- **Tab to group:** Tab moves focus into segmented control
- **Arrow navigation:** Left/Right (or Up/Down for vertical) arrows navigate between options
- **Space/Enter selection:** Space or Enter selects focused option
- **Tab out:** Tab moves focus out of control to next element

Touch-screen

- **Touch targets:** Each segment has minimum 44x44px tap target
- **Adequate spacing:** Clear visual separation between segments for accurate tapping
- **Full segment clickable:** Entire segment area is tappable
- **Visual feedback:** Immediate feedback on tap

Responsive Behavior

- **Orientation:** Can be horizontal (default) or vertical
- **Mobile stacking:** Horizontal controls may become vertical on narrow screens
- **Equal width:** Segments typically equal width in horizontal orientation
- **Text truncation:** Long labels truncate with ellipsis to maintain compact size
- **Icon support:** Can use icons instead of or alongside text

Variants & Features

- **Size variants:** Small, medium, large
- **Icon-only:** Segments contain only icons (with accessible labels)
- **Icon + text:** Segments contain both icon and label
- **Disabled segments:** Individual segments can be disabled
- **Full-width:** Control can expand to fill container width

Test Scenarios

- **Control renders:** All segments display in connected group
- **Initial selection:** One segment is selected by default
- **Selection change:** Clicking/tapping unselected segment selects it and deselects previous
- **Visual distinction:** Selected segment has clear visual distinction (filled, outlined, or elevated)
- **Equal width:** In horizontal orientation, all segments are equal width
- **Connected appearance:** Segments share borders (no gaps or double borders between segments)
- **Keyboard focus:** Tab moves focus into control with visible focus indicator
- **Arrow navigation:** Arrow keys navigate between segments
- **Arrow wrapping:** Arrows wrap from last segment to first and vice versa
- **Space/Enter selection:** Space or Enter on focused segment selects it
- **Screen reader announcement:** Screen reader announces option label and selected state
- **Disabled segment:** Disabled segment is visually de-emphasized and cannot be selected
- **onChange callback:** Callback fires with new value when selection changes
- **Orientation:** Vertical variant stacks segments vertically with appropriate styling
- **Icon rendering:** Icon-only segments display icons with accessible text labels
- **Text truncation:** Long segment labels truncate appropriately
- **Mobile touch:** Segments have adequate touch target size on mobile

Notes

- **Segmented Control vs Tabs:** Segmented control changes a setting/filter. Tabs switch between content panels
- **Segmented Control vs Radio Group:** Segmented control for immediate changes (view mode, filter). Radio group for form selections saved on submit
- **Segmented Control vs Dropdown:** Use segmented control when options are few (2-5) and should all be visible. Use dropdown for more options

- **Option limit:** Works best with 2-5 options. More than 5 becomes cramped—consider dropdown or other control
- **Equal width advantage:** Equal-width segments create visual harmony and balanced appearance
- **Common patterns:** List/Grid toggle, Map/Satellite view, Day/Week/Month selector, Chart type selector
- **Disabled options:** Disabled segments indicate unavailable but related options (e.g., “Year” view disabled when insufficient data)
- **Mobile orientation:** Vertical orientation often better on mobile for larger touch targets

#27 Range Slider

Allows selection of a value or a range along a continuous track.

Specialization: Range Slider with two handles (min/max range selection)

Description

A range slider allows users to select a numeric value (single slider) or a range of values (dual slider) by dragging a handle along a track. Sliders are ideal for settings where the exact value is less important than relative position (volume, brightness, price ranges) or when users need visual feedback during adjustment. They provide an intuitive, visual way to input numeric values within a defined range.

Main Features

- **Track display:** Horizontal or vertical bar showing value range
- **Draggable handle:** Thumb/handle users drag to select value
- **Value display:** Shows current value (tooltip, label, or adjacent text)
- **Min/max bounds:** Defined minimum and maximum values
- **Step increments:** Optional discrete steps between values (e.g., multiples of 5)
- **Visual fill:** Track portion before handle typically highlighted

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<input type="range">` or custom implementation with `role="slider"`
- **ARIA attributes:** `aria-valuemin`, `aria-valuemax`, `aria-valuenow`, `aria-valuetext`
- **Keyboard accessible:** Can adjust value via keyboard
- **Value announcement:** Screen readers announce value as it changes
- **Label association:** Labeled to describe what value represents

Keyboard Navigation

- **Tab navigation:** Tab moves to/from slider
- **Arrow keys:** Left/Down decreases, Right/Up increases value
- **Page Up/Down:** Larger increments (e.g., 10% of range)
- **Home/End:** Jump to minimum/maximum value
- **Step precision:** Arrow keys move by defined step amount

Touch-screen

- **Touch target:** Handle is large enough for touch interaction (minimum 44x44px)
- **Drag support:** Users can drag handle along track
- **Tap to position:** Tapping track moves handle to that position
- **Touch feedback:** Visual feedback during drag

Responsive Behavior

- **Flexible width:** Slider stretches to container width (horizontal) or height (vertical)
- **Mobile-friendly:** Larger handles on touch devices
- **Vertical orientation:** Can be oriented vertically for some use cases

Validation & Feedback

- **Real-time value:** Value updates continuously as handle moves
- **Discrete steps:** Can snap to specific values (e.g., integers only)
- **Value constraints:** Enforces min/max boundaries
- **onChange callback:** Notifies of value changes during drag and on release

Variants & Features

- **Dual-handle slider:** Two handles for selecting a range (min and max)
- **Labeled track:** Tick marks or labels at intervals along track
- **Tooltips:** Value tooltip appears when dragging
- **Color gradients:** Track can show gradient (e.g., cold to hot, cheap to expensive)

Test Scenarios

- **Slider renders:** Track and handle display correctly
- **Initial value:** Handle positioned at initial value
- **Drag interaction:** Dragging handle updates value and handle position
- **Track click:** Clicking track moves handle to that position
- **Value display:** Current value displayed and updates in real-time
- **Min/max boundaries:** Handle cannot move beyond min or max values
- **Step increments:** When steps defined, handle snaps to step values
- **Keyboard increment:** Arrow keys adjust value by step amount
- **Keyboard large jump:** Page Up/Down adjusts by larger increment
- **Keyboard min/max:** Home goes to min, End goes to max
- **Screen reader:** Value changes announced as handle moves
- **onChange callback:** Callback fires during drag and on release with current value
- **Disabled state:** Disabled slider is visually de-emphasized and cannot be adjusted
- **Visual fill:** Track portion before handle shows filled styling
- **Tooltip:** When configured, tooltip shows current value during drag
- **RTL layout:** Slider direction mirrors for RTL (right is min, left is max)

Dual-Handle Range Slider Specialization

Additional Features:

- **Two handles:** Min handle and max handle on same track
- **Range selection:** User selects a range between two values
- **Handle independence:** Each handle can be moved independently
- **Handle collision:** Handles cannot pass each other
- **Range display:** Visual indication of selected range between handles

Additional Test Scenarios:

- **Both handles render:** Min and max handles display on track
- **Independent movement:** Each handle can be dragged independently
- **Collision prevention:** Handles stop when they reach each other
- **Range highlighting:** Track between handles shows selected range styling
- **Min handle boundary:** Min handle cannot go below minimum or above max handle
- **Max handle boundary:** Max handle cannot go above maximum or below min handle
- **Keyboard navigation:** Tab moves between min handle, max handle, then out
- **Value announcement:** Screen reader announces both min and max values

Notes

- **Slider vs Input:** Slider for approximate values or relative adjustments. Text input for precise values
- **Step size:** Consider domain—volume might be continuous, star rating is discrete (1-5)
- **Value visibility:** Always show current value somewhere (on handle, in label, or adjacent)
- **Mobile considerations:** Ensure handle is large enough for touch (minimum 44x44px)

- **Vertical sliders:** Less common but useful for volume, elevation, or spatial controls
- **Accessibility:** Native `<input type="range">` provides good accessibility. Custom implementations need careful ARIA
- **Range slider complexity:** Dual-handle sliders are significantly more complex—ensure value of feature before implementing

#28 Rating

Input for selecting a rating, typically using stars or icons.

Description

A rating component allows users to provide feedback or evaluation by selecting a number of items (typically stars) from a fixed set. Ratings are commonly used for product reviews, service feedback, content quality, and satisfaction surveys. The visual metaphor of filled vs empty icons makes the rating scale immediately understandable and provides clear visual feedback.

Main Features

- **Icon-based selection:** Typically 5 stars (or other icons like hearts, thumbs)
- **Click to rate:** Clicking an icon sets rating to that value
- **Visual feedback:** Selected items filled/colored, unselected items empty/gray
- **Read-only mode:** Can display existing rating without allowing changes
- **Half-star support:** Optional half-increment ratings (2.5 stars)
- **Customizable scale:** Configurable number of items (3, 5, 10, etc.)

Secondary Features

Accessibility

- **Semantic structure:** Implemented as fieldset with radio buttons or ARIA slider

- **Keyboard accessible:** Can select rating via keyboard
- **Screen reader labels:** Each rating level has clear label (“1 star”, “2 stars”, etc.)
- **Current rating announcement:** Screen reader announces selected rating
- **Label association:** Clear label explaining what is being rated

Keyboard Navigation

- **Tab to rating:** Tab moves focus into rating control
- **Arrow keys:** Left/Right or Up/Down arrows navigate between rating values
- **Number keys:** Pressing 1-5 (or 0-10) selects that rating directly
- **Home/End:** Jump to lowest/highest rating
- **Space/Enter:** Confirms focused rating value

Touch-screen

- **Touch targets:** Each icon has minimum 44x44px tap target
- **Tap to rate:** Tapping any icon sets rating to that level
- **Visual feedback:** Immediate visual response to tap
- **Adequate spacing:** Sufficient space between icons for accurate tapping

Responsive Behavior

- **Flexible sizing:** Icons scale based on component size (small, medium, large)
- **Icon spacing:** Consistent spacing between icons
- **Mobile sizing:** Larger icons on mobile for easier tapping

Validation & Feedback

- **Required validation:** Can require rating before form submission
- **Error state:** Shows error when required rating not provided
- **onChange callback:** Notifies when rating value changes
- **onHover feedback:** Shows preview of rating on hover before selection

Variants & Features

- **Star icon:** Traditional 5-star rating (most common)
- **Custom icons:** Hearts, thumbs up/down, emoji, or custom symbols
- **Half ratings:** Support for half-increments (2.5, 3.5, etc.)
- **Labels:** Optional text labels for each rating level (“Poor”, “Fair”, “Good”, “Excellent”)
- **Read-only display:** Shows rating value without interaction (for displaying existing ratings)

Test Scenarios

- **Rating renders:** All icons display in unselected state initially
- **Click to rate:** Clicking third icon selects 3-star rating
- **Visual feedback:** Icons up to and including selected icon show filled state
- **Hover preview:** Hovering shows preview of that rating (icons fill temporarily)
- **Read-only mode:** In read-only mode, icons show rating but are not interactive
- **Half-star support:** When enabled, half-stars can be selected and display correctly
- **Custom icon:** When custom icon specified, uses that icon instead of star
- **Scale configuration:** When max rating is 10, displays 10 icons (not 5)
- **Keyboard focus:** Tab moves focus to rating with visible focus indicator
- **Arrow navigation:** Arrow keys navigate between rating values (0-5)
- **Number key:** Pressing “4” selects 4-star rating
- **Screen reader:** Screen reader announces “3 out of 5 stars selected”
- **Required validation:** When required and not rated, shows error state
- **onChange callback:** Callback fires with new rating value when selection changes
- **Clear rating:** Optional clear/reset button returns to zero rating
- **Disabled state:** Disabled rating is visually de-emphasized and non-interactive

- **Icon labels:** When hover labels enabled, shows text for each level (“Poor”, “Good”, etc.)

Notes

- **5-star standard:** 5-star rating is most recognizable and widely used. Consider carefully before deviating
- **Half-stars:** Add complexity for both users and implementation. Use only if precision matters
- **Read-only vs interactive:** Clearly distinguish between rating input and rating display (read-only)
- **Zero rating:** Decide if “no rating” (0 stars) is allowed or if minimum is 1 star
- **Hover behavior:** Hover preview helps users understand what will happen when they click
- **Icon choice:** Stars are universal. Other icons (hearts, thumbs) may have cultural or contextual meaning
- **Label clarity:** “Rate this product” is clearer than “Rating” for the control label
- **Rating display:** When showing average ratings, support decimal values (4.3 stars) visually
- **Accessible alternatives:** For screen reader users, radio buttons with labels may be clearer than icon-based rating

#29 File Uploader

Allows users to select and upload one or more files. Supports drag-and-drop and click-to-browse interactions, with upload progress feedback.

Description

File Uploader enables users to select files from their device for upload to the application. It combines file selection (via browse dialog or drag-and-drop), preview, validation, and upload progress feedback. File uploaders are essential for

applications handling documents, images, media, or any user-provided files. A well-designed uploader provides clear feedback at every step and handles errors gracefully.

Main Features

- **File selection:** Click to open browser's file picker dialog
- **Drag-and-drop:** Drag files from desktop onto drop zone
- **Multiple file support:** Can accept single file or multiple files
- **File type restrictions:** Limits accepted file types (images, PDFs, etc.)
- **File size validation:** Enforces maximum file size limits
- **Upload progress:** Shows progress bar or percentage during upload
- **Preview:** Displays thumbnail or filename of selected files

Secondary Features

Accessibility

- **Keyboard accessible:** Can be activated via keyboard (Enter/Space opens file dialog)
- **Screen reader support:** Announces file selection and upload status
- **Error announcements:** Validation errors and upload failures announced to screen readers
- **Focus management:** Maintains logical focus flow during interaction

Keyboard Navigation

- **Tab to upload:** Tab moves focus to upload trigger
- **Enter/Space activation:** Opens file browser dialog
- **Delete key:** Removes selected file from upload queue
- **Arrow navigation:** Navigate between multiple selected files

Touch-screen

- **Touch target:** Upload trigger has minimum 44x44px tap target

- **Drag-and-drop:** Mobile browsers may not support drag-and-drop (show browse button prominently)
- **Camera access:** On mobile, can trigger camera for photo capture
- **Native file picker:** Uses device's native file picker

Responsive Behavior

- **Flexible drop zone:** Drop zone scales to container
- **Mobile optimization:** Prominent browse button on mobile (drag-and-drop less common)
- **Preview sizing:** File previews scale appropriately
- **Progress display:** Progress indicators adapt to screen size

Validation & Feedback

- **File type validation:** Rejects files not matching accepted types
- **File size validation:** Rejects files exceeding size limit
- **Maximum file count:** Enforces limit on number of files
- **Upload progress:** Real-time progress indicator (percentage, bar, or spinner)
- **Success confirmation:** Clear indication when upload completes
- **Error messaging:** Specific error messages for validation failures or upload errors
- **Retry mechanism:** Allows retrying failed uploads

Advanced Features

- **Image preview:** Thumbnail preview for image files
- **Multiple file queue:** Shows list of all selected files before upload
- **Remove files:** Ability to remove files from queue before upload
- **Paste support:** Paste images from clipboard
- **Chunked upload:** Large files uploaded in chunks for reliability
- **Resume capability:** Resume interrupted uploads

Test Scenarios

- **Uploader renders:** Upload trigger (button or drop zone) displays
- **Click to browse:** Clicking trigger opens file browser dialog
- **File selection:** Selecting file from browser dialog loads file into component
- **Drag-and-drop:** Dragging file over drop zone highlights it
- **Drop to upload:** Dropping file onto zone adds it to upload queue
- **Multiple files:** When multiple files allowed, selecting multiple files loads all
- **File type validation:** Selecting invalid file type shows error message
- **File size validation:** Selecting oversized file shows error message
- **Max file count:** Attempting to exceed max files shows error or prevents selection
- **Preview display:** Selected file shows preview (thumbnail for images, icon+name for others)
- **Upload initiation:** Triggering upload starts file transfer
- **Progress indicator:** Progress bar or percentage displays during upload
- **Upload completion:** Successful upload shows success state
- **Upload failure:** Failed upload shows error message with retry option
- **Remove file:** Removing file from queue deletes it before upload
- **Keyboard activation:** Pressing Enter/Space on focused trigger opens file dialog
- **Screen reader:** File selection and upload status announced to screen readers
- **Camera access:** On mobile devices with camera, camera option appears in file picker
- **Paste support:** When enabled, pasting image from clipboard adds it to queue

Notes

- **File type restrictions:** Use accept attribute (`accept="image/*, application/pdf"`) to filter file picker
- **Size limits:** Validate both client-side (immediate feedback) and server-side (security)
- **Multiple files:** Consider UX carefully—multiple files can overwhelm users if not well-designed

- **Progress feedback:** Essential for large files or slow connections. Show estimated time remaining
- **Error recovery:** Allow users to retry failed uploads without re-selecting files
- **Thumbnail generation:** Generate thumbnails client-side before upload for better UX
- **Security:** Never trust client-side validation alone. Always validate server-side
- **Chunked uploads:** For very large files (>100MB), consider chunked uploads with resume capability
- **Mobile camera:** On mobile, file picker should include camera option for photos
- **Drag-and-drop browser support:** Not all mobile browsers support drag-and-drop. Provide browse button

#30 Color Picker

Full color selection tool combining hue, saturation, brightness, and opacity controls. Builds on Color Swatch.

Description

A color picker allows users to select any color from the color spectrum using visual controls. It typically includes a hue selector, saturation/brightness surface, optional opacity slider, and displays the selected color in multiple formats (hex, RGB, HSL). Color pickers are essential for design tools, customization interfaces, theming systems, and any application where users need precise color control.

Main Features

- **Hue selector:** Slider or strip showing full color spectrum (0-360°)
- **Saturation/brightness surface:** 2D area for selecting color saturation and brightness
- **Color preview:** Shows currently selected color
- **Multiple formats:** Displays color as hex, RGB, HSL, or HSV

- **Format switcher:** Toggle between color format displays
- **Color input:** Text input for directly entering color values
- **Opacity control:** Optional alpha/opacity slider for transparency

Secondary Features

Accessibility

- **Keyboard accessible:** All controls operable via keyboard
- **Color value announcement:** Screen readers announce selected color value
- **Format labels:** Each control labeled clearly (Hue, Saturation, etc.)
- **High contrast:** Controls remain visible against colored backgrounds

Keyboard Navigation

- **Tab navigation:** Tab moves between controls (hue, surface, opacity, inputs)
- **Arrow keys on surface:** Arrow keys adjust saturation/brightness
- **Arrow keys on sliders:** Adjust hue/opacity values
- **Text input editing:** Standard text input for direct color entry
- **Enter confirmation:** Enter applies selected color

Touch-screen

- **Touch targets:** Sliders and controls have adequate touch targets
- **Drag support:** Smooth dragging on hue slider and color surface
- **Pinpoint accuracy:** Precise color selection even on small screens
- **Touch feedback:** Visual feedback during interaction

Responsive Behavior

- **Flexible sizing:** Color picker scales to available space
- **Mobile layout:** May use different layout on mobile (sliders stack vertically)
- **Surface size:** Saturation/brightness surface maintains usable size

Validation & Feedback

- **Real-time updates:** Color value updates as user interacts with controls
- **Format validation:** Text input validates color format (hex, RGB, etc.)
- **Invalid input:** Shows error for invalid color values
- **onChange callback:** Notifies when color changes
- **Preset colors:** Optional predefined color swatches for quick selection

Advanced Features

- **Eyedropper tool:** Pick color from anywhere on screen
- **Color history:** Shows recently selected colors
- **Preset palettes:** Predefined color sets (brand colors, material colors, etc.)
- **Gradient support:** Advanced pickers support gradient creation
- **Alpha channel:** Transparency/opacity control (RGBA, HSLA)

Test Scenarios

- **Picker renders:** All controls (hue, surface, preview) display correctly
- **Hue selection:** Dragging hue slider changes color hue
- **Saturation/brightness:** Clicking or dragging on surface adjusts saturation and brightness
- **Color preview:** Preview box shows currently selected color in real-time
- **Opacity slider:** When enabled, opacity slider adjusts color transparency
- **Format display:** Selected color displays in current format (hex, RGB, HSL)
- **Format switcher:** Switching format updates display without changing color
- **Text input:** Typing valid color value into input updates picker
- **Invalid input:** Entering invalid color shows error and doesn't update picker
- **Preset swatches:** Clicking preset swatch selects that color
- **Keyboard hue:** Arrow keys on hue slider adjust hue value
- **Keyboard surface:** Arrow keys on surface adjust saturation/brightness
- **Tab order:** Tab moves through controls in logical order
- **Screen reader:** Color values announced when changed
- **onChange callback:** Callback fires with new color value when color changes
- **Color history:** When enabled, recently used colors appear in history section

- **Eyedropper:** When supported, eyedropper tool picks color from screen
- **Alpha transparency:** Opacity slider adjusts alpha channel (0-100%)
- **Format conversion:** Color value correctly converts between formats

Notes

- **Color formats:** Support hex (#RRGGBB), RGB, HSL at minimum. RGBA/HSLA for transparency
- **Default color:** Always have a selected color (don't start empty). Black or white are common defaults
- **Hex input:** Accept both 3-digit (#RGB) and 6-digit (#RRGGBB) hex codes
- **Alpha channel:** Transparency adds significant complexity. Include only if needed
- **Eyedropper:** Browser support varies. Check compatibility before implementing
- **Color space:** Most pickers use RGB/HSL. Advanced tools may need LAB or CMYK
- **Recent colors:** Store in localStorage for persistence across sessions
- **Accessibility challenge:** Color pickers are inherently visual. Provide text input as accessible alternative
- **Mobile UX:** Color pickers can be complex on small screens. Consider simplified mobile version
- **Preset palettes:** Include brand colors or common palettes for faster selection

#31 Card

Content grouping container with optional header, body, footer, and action areas.

Specializations: Multirow Card (structured rows of label/value pairs), Multirow Card List (list of multirow cards, useful for mobile data display)

Description

A card is a container that groups related information and actions into a cohesive, visually distinct unit. Cards make content scannable and digestible by creating clear boundaries between different topics or items. They're versatile building blocks used for product listings, user profiles, article previews, dashboard widgets, and countless other content displays.

Main Features

- **Content container:** Groups related content with distinct visual boundaries
- **Flexible structure:** Supports header, body, footer sections
- **Visual elevation:** Typically uses shadow, border, or background to separate from page
- **Optional sections:** Header, footer, and actions are optional based on content needs
- **Clickable area:** Entire card can be clickable or specific areas can be interactive

Secondary Features

Accessibility

- **Semantic structure:** Uses appropriate semantic HTML (article, section, or div with role)
- **Heading hierarchy:** Card titles use proper heading levels
- **Clickable cards:** When entire card is clickable, has proper button/link semantics
- **Action labels:** Card actions have clear, descriptive labels

Keyboard Navigation

- **Focusable elements:** Interactive elements within card are keyboard accessible

- **Tab order:** Logical tab order through card elements (title → actions → body links)
- **Card navigation:** If card itself is clickable, can be activated via keyboard

Touch-screen

- **Touch targets:** Interactive elements have minimum 44x44px tap targets
- **Card tap:** If entire card is interactive, entire area is tappable
- **Action separation:** Distinct actions within card have clear tap boundaries

Responsive Behavior

- **Flexible width:** Adapts to container width
- **Grid layouts:** Multiple cards typically arranged in responsive grid
- **Stacking:** Card sections stack vertically on narrow screens
- **Image scaling:** Images within cards scale responsively

Variants & Features

- **Header section:** Optional title, subtitle, and actions
- **Media section:** Image, video, or illustration
- **Body section:** Main content area
- **Footer section:** Metadata, timestamps, or secondary actions
- **Horizontal variant:** Content arranged horizontally instead of vertically
- **Interactive card:** Entire card is clickable/tappable
- **Elevation levels:** Different shadow depths for visual hierarchy

Test Scenarios

- **Card renders:** Card displays with border, shadow, or background separation
- **Header section:** When header provided, displays at top of card
- **Body content:** Card body displays main content
- **Footer section:** When footer provided, displays at bottom of card
- **Image display:** Media section displays image with proper scaling

- **Action buttons:** Card actions render in appropriate location (header or footer)
- **Clickable card:** When entire card is interactive, clicking anywhere triggers action
- **Hover state:** Interactive cards show hover effect
- **Elevation:** Card shows appropriate shadow or elevation styling
- **Responsive layout:** Card width adapts to container
- **Grid arrangement:** Multiple cards arrange in responsive grid
- **Semantic HTML:** Card uses appropriate semantic element or ARIA role
- **Heading levels:** Card title uses correct heading level in document hierarchy
- **Tab order:** Tab moves through interactive elements in logical order
- **Touch feedback:** Tapping interactive card provides visual feedback

Multirow Card Specialization

Multirow Card displays data as structured label/value rows, useful for displaying entity details or form-like data.

Additional Features:

- **Row structure:** Each row contains a label and value
- **Consistent layout:** Rows have consistent styling and spacing
- **Label alignment:** Labels align (typically left or right)
- **Dividers:** Optional dividers between rows

Additional Test Scenarios:

- **Rows render:** All label/value pairs display as rows
- **Label alignment:** Labels align consistently
- **Row spacing:** Consistent spacing between rows
- **Divider display:** When enabled, dividers appear between rows
- **Responsive behavior:** Rows stack or maintain side-by-side layout based on screen size

Multirow Card List Specialization

List of Multirow Cards, optimized for mobile data display (contacts, transactions, orders).

Additional Features:

- **Vertical stacking:** Cards stack vertically
- **Consistent spacing:** Uniform gaps between cards
- **Infinite scroll:** Often used with pagination or infinite loading
- **Swipe actions:** On mobile, swipe reveals actions (delete, archive)

Additional Test Scenarios:

- **Cards stack:** Multiple cards display in vertical list
- **Spacing:** Consistent gaps between list items
- **Swipe actions:** Swiping card reveals action buttons
- **Loading states:** Loading indicator appears when fetching more cards

Notes

- **Card vs Section:** Cards group related content. Sections organize page structure. Cards can exist within sections
- **Elevation consistency:** Use consistent elevation levels across card types (e.g., level 1 for standard cards, level 2 for interactive)
- **Clickable area:** If entire card is clickable, ensure there's clear visual affordance (hover state, cursor change)
- **Action placement:** Primary actions in header or footer. Overflow menu for additional actions
- **Image aspect ratios:** Maintain consistent image ratios across cards in a grid for visual harmony
- **Content length:** Design for variable content lengths—cards with different amounts of text should still align in grids
- **Mobile optimization:** Cards work well on mobile. Consider full-width cards on small screens

#32 Alert / Banner

Displays an important message to the user.

Specializations: Inline Alert (appears within content flow), Actionable Alert (includes buttons for user response), Banner (full-width page-level alert, typically at top of page)

***Naming note:** Toast / Snackbar (45) are temporary and auto-dismiss. Alerts/Banners are persistent until dismissed by the user.*

Description

An alert communicates important information that requires user awareness or action. Alerts appear inline within content or as prominent banners, remaining visible until the user dismisses them or takes action. They're used for errors, warnings, success confirmations, and informational messages. Unlike toasts (which auto-dismiss), alerts persist until explicitly handled.

Main Features

- **Message display:** Shows title and/or description text
- **Severity levels:** Error (critical), warning (caution), success (confirmation), info (neutral)
- **Visual distinction:** Color coding and icon indicate severity
- **Dismissible:** Close button allows user to dismiss alert
- **Persistent:** Remains visible until dismissed (unlike auto-dismissing toasts)
- **Optional actions:** Can include action buttons (retry, undo, learn more)

Secondary Features

Accessibility

- **ARIA role:** Uses role="alert" or role="status" for screen reader announcement

- **Live region:** Alert appearance is announced to screen readers
- **Severity indication:** Severity communicated via text and ARIA, not color alone
- **Close button:** Dismiss button has clear label (“Close alert”, “Dismiss”)
- **Action accessibility:** Action buttons are keyboard accessible

Keyboard Navigation

- **Tab to close:** Close button can be focused via Tab
- **Tab to actions:** Action buttons are in tab order
- **Escape to dismiss:** Pressing Escape closes dismissible alert
- **Focus management:** When alert appears, focus may move to alert or remain in context

Touch-screen

- **Touch targets:** Close button and actions have minimum 44x44px tap targets
- **Swipe to dismiss:** Optional swipe gesture to dismiss on mobile

Responsive Behavior

- **Flexible width:** Alert adapts to container width
- **Text wrapping:** Long messages wrap without breaking layout
- **Action stacking:** Action buttons may stack vertically on narrow screens
- **Banner full-width:** Banner variant spans full viewport width

Variants & Features

- **Inline alert:** Appears within content flow, typically in forms
- **Banner alert:** Full-width at top of page, sticky or static
- **Actionable alert:** Includes action buttons (Retry, Undo, etc.)
- **With icon:** Severity icon provides visual cue
- **With title:** Optional title/heading for alert
- **Auto-dismiss:** Uncommon, but some systems support timed dismissal

Test Scenarios

- **Alert renders:** Alert displays with appropriate severity styling
- **Severity styling:** Error is red, warning is yellow/orange, success is green, info is blue
- **Icon display:** Severity icon displays (X for error, ! for warning, checkmark for success, i for info)
- **Message text:** Alert message displays clearly
- **Title display:** When title provided, displays prominently
- **Close button:** Dismiss button displays and is clearly labeled
- **Close action:** Clicking close button dismisses alert
- **Keyboard close:** Pressing Escape dismisses dismissible alert
- **Screen reader announcement:** Alert appearance is announced to screen readers
- **Severity announcement:** Severity is communicated to screen readers (not just visually)
- **Action buttons:** When actions provided, buttons display and are functional
- **Action callback:** Clicking action button triggers appropriate callback
- **Persistent display:** Alert remains visible until explicitly dismissed
- **Focus management:** Focus handling is logical when alert appears
- **RTL support:** Alert layout mirrors correctly in RTL mode

Inline Alert Specialization

Inline Alert appears within content flow, typically near the element it relates to (e.g., form validation errors).

Additional Features:

- **Contextual placement:** Appears near related content
- **Compact sizing:** More compact than page-level banners
- **Field association:** Can be associated with specific form field

Additional Test Scenarios:

- **Inline positioning:** Alert appears inline within content flow

- **Field proximity:** Displays near associated form field or content
- **Compact styling:** Uses more compact padding and sizing

Actionable Alert Specialization

Alert with action buttons allowing user to respond (Retry, Undo, Learn More, etc.).

Additional Features:

- **Action buttons:** One or more buttons for user response
- **Primary/secondary actions:** Visual distinction between action types
- **Button positioning:** Actions in footer or alongside message

Additional Test Scenarios:

- **Actions render:** Action buttons display clearly
- **Action distinction:** Primary and secondary actions visually distinguished
- **Action callbacks:** Each action button triggers correct callback
- **Keyboard actions:** Actions are keyboard accessible

Banner Specialization

Full-width alert at top (or bottom) of page, often sticky.

Additional Features:

- **Full viewport width:** Spans entire page width
- **Sticky positioning:** Often remains visible during scroll
- **Dismissible state:** Remembers dismissal (localStorage) to avoid repeated displays
- **System-level:** Used for system-wide messages (maintenance, announcements)

Additional Test Scenarios:

- **Full width:** Banner spans entire viewport width
- **Sticky behavior:** When sticky, remains visible during page scroll
- **Dismissal persistence:** Dismissed banner doesn't reappear on page reload

- **Positioning:** Appears at configured location (top or bottom of page)

Notes

- **Alert vs Toast:** Alert is persistent, requires dismissal. Toast auto-dismisses after brief period
- **Error alerts:** For errors, explain what happened and how to fix it. “Invalid email” → “Email must include @”
- **Success alerts:** Confirm action completion. “Changes saved”, “Email sent successfully”
- **Warning alerts:** Prevent potential issues. “Your session will expire in 5 minutes”
- **Info alerts:** Provide helpful context. “This beta feature may have limitations”
- **Alert fatigue:** Too many alerts cause users to ignore them. Use sparingly
- **Placement:** Inline alerts near affected content. Banners for system-wide messages
- **Don’t block actions:** Avoid modal-like blocking alerts unless action is critical

#33 Progress

Communicates the status of an ongoing operation.

Specializations: Progress Bar (linear, 0–100%), Progress Circle (circular, 0–100%), Indeterminate Progress (no known completion percentage)

Description

Progress indicators show users that a process is underway and, when possible, how much has been completed and how much remains. They reduce perceived wait time, prevent user frustration, and help users decide whether to wait or abandon. Determinate progress shows percentage (0-100%), while indeterminate progress shows ongoing activity without a specific completion estimate.

Main Features

- **Visual progress representation:** Bar fills, circle completes, or animation shows activity
- **Determinate vs indeterminate:** Shows specific percentage or generic “working” animation
- **Percentage display:** Optional text showing completion percentage (45%, 3 of 10 items, etc.)
- **Label:** Describes what is in progress (“Uploading file...”, “Processing payment...”)
- **Accessible status:** Progress updates announced to screen readers

Secondary Features

Accessibility

- **ARIA role:** role=“progressbar” with aria-valuenow, aria-valuemin, aria-valuemax
- **Live region:** Progress updates announced via aria-live=“polite”
- **Status text:** Text description of current state, not just visual
- **Screen reader updates:** Not too frequent (avoid spamming every 1% change)

Responsive Behavior

- **Flexible width:** Progress bars adapt to container width
- **Size variants:** Small, medium, large sizes for different contexts
- **Mobile optimization:** Adequate size for visibility on small screens

Animation & Transitions

- **Smooth progression:** Progress updates animate smoothly, not jump abruptly
- **Indeterminate animation:** Continuous motion (sliding, pulsing, or rotating)

- **Reduced motion:** Respects prefers-reduced-motion preference
- **Frame rate:** Maintains smooth 60fps animation

Variants & Features

- **Progress bar (linear):** Horizontal bar that fills left to right (or right to left in RTL)
- **Progress circle:** Circular progress with arc completing around circle
- **Color coding:** Optional color changes (red for errors, green for success)
- **Striped pattern:** Animated stripes for visual interest
- **Buffer indicator:** Shows buffered/loaded amount separately from active progress (video players)

Test Scenarios

- **Progress renders:** Progress indicator displays
- **Determinate display:** Shows current percentage (e.g., 45% complete)
- **Progress updates:** Visual progress updates as value changes
- **Percentage text:** Optional percentage label displays and updates
- **Label text:** Descriptive label shows what's in progress
- **Min/max bounds:** Progress constrained to 0-100% range
- **Indeterminate mode:** Shows continuous animation when percentage unknown
- **ARIA attributes:** Has correct role, aria-valuenow, aria-valuemin, aria-valuemax
- **Screen reader updates:** Progress changes announced to screen readers appropriately
- **Color variants:** When configured, progress color changes based on state
- **Completion state:** At 100%, shows completed state (may change color or icon)
- **Smooth animation:** Progress transitions smoothly without jumping
- **Reduced motion:** Animation is minimal or static when user prefers reduced motion

Progress Bar Specialization (Linear)

Additional Features:

- **Horizontal orientation:** Bar fills from left to right
- **Height variants:** Thin, medium, thick bars
- **Striped animation:** Animated diagonal stripes move across bar
- **Background track:** Shows empty portion of bar

Additional Test Scenarios:

- **Bar fills:** Filled portion extends from left to right
- **Height:** Bar renders at correct height (thin, medium, thick)
- **Striped animation:** When enabled, diagonal stripes animate across bar
- **RTL:** In RTL mode, fills from right to left

Progress Circle Specialization

Additional Features:

- **Circular display:** Progress arc completes around circle
- **Stroke width:** Configurable thickness of circular stroke
- **Center content:** Optional content in center (percentage, icon, text)
- **Clockwise progression:** Arc typically progresses clockwise

Additional Test Scenarios:

- **Circle renders:** Circular progress indicator displays
- **Arc progression:** Arc completes clockwise around circle
- **Center display:** When provided, center content displays (percentage, text)
- **Stroke width:** Circle stroke renders at correct thickness

Indeterminate Progress Specialization

Additional Features:

- **Continuous animation:** Never shows percentage, always animating

- **Multiple patterns:** Sliding bar, pulsing, rotating spinner
- **Duration unknown:** Used when completion time is unpredictable

Additional Test Scenarios:

- **Indeterminate animation:** Continuous motion without showing percentage
- **Pattern type:** Animation matches configured pattern (slide, pulse, spin)
- **ARIA:** aria-valuenow is not set (or set to indeterminate value)

Notes

- **When to use determinate:** Use when progress can be calculated (file uploads, step-by-step processes)
- **When to use indeterminate:** Use when duration is unknown (server processing, initial loading)
- **Update frequency:** Update progress visually every 100-200ms. Update screen readers every 10-20%
- **Completion feedback:** When reaching 100%, consider showing success state before hiding
- **Estimated time:** For long processes, show estimated time remaining (“About 2 minutes left”)
- **Cancellation:** For cancellable processes, provide cancel button
- **Errors:** If process fails, show error state on progress indicator

#34 Scroll Area

Custom scrollable container with styled, consistent scrollbars across platforms.

Description

Scroll Area provides a scrollable container with customized, consistent scrollbar styling across different browsers and operating systems. Native scrollbars vary significantly between platforms, which can disrupt visual design. A custom scroll

area maintains design consistency while preserving expected scroll behavior and accessibility.

Main Features

- **Scrollable content:** Container scrolls when content exceeds dimensions
- **Custom scrollbar styling:** Consistent appearance across browsers/platforms
- **Vertical and horizontal:** Supports both scroll directions
- **Drag scrolling:** Users can drag scrollbar thumb to scroll
- **Wheel/touch scrolling:** Native scroll gestures work normally

Secondary Features

Accessibility

- **Keyboard scrolling:** Arrow keys, Page Up/Down, Home/End work
- **Focus scrolling:** Focused elements scroll into view automatically
- **Screen reader compatibility:** Works with screen reader scroll commands
- **Scrollbar visibility:** Scrollbars don't hide critical content

Keyboard Navigation

- **Arrow keys:** Scroll content up/down/left/right
- **Page Up/Down:** Scroll by page height
- **Home/End:** Jump to top/bottom of content
- **Tab navigation:** Tabbing through focusable elements scrolls them into view

Touch-screen

- **Touch scrolling:** Standard touch scroll gestures work
- **Momentum scrolling:** Inertial scrolling on mobile
- **Pull-to-refresh:** Optional pull-to-refresh on mobile (if applicable)

Responsive Behavior

- **Flexible dimensions:** Scrollable area adapts to container
- **Auto-hide scrollbars:** Scrollbars fade when not in use (optional)
- **Mobile optimization:** Thinner scrollbars or hidden scrollbars on mobile

Variants & Features

- **Always visible:** Scrollbars always shown
- **Auto-hide:** Scrollbars fade when idle, appear on hover/scroll
- **Overlay scrollbars:** Float over content (don't take layout space)
- **Gutter scrollbars:** Take layout space (push content)

Test Scenarios

- **Content scrolls:** Overflowing content is scrollable
- **Custom styling:** Scrollbars use custom styling (not default browser)
- **Drag thumb:** Dragging scrollbar thumb scrolls content
- **Wheel scroll:** Mouse wheel scrolls content
- **Touch scroll:** Touch dragging scrolls content on touch devices
- **Arrow keys:** Arrow keys scroll content when container focused
- **Page Up/Down:** Page keys scroll by viewport height
- **Auto-hide:** When enabled, scrollbars fade when idle
- **Hover reveal:** Scrollbars appear on container hover (if auto-hide)
- **Scrollbar size:** Scrollbar thumb size represents content proportion
- **Scroll position:** Scroll position indicator accurate
- **Focus scrolling:** Focused element scrolls into view automatically
- **RTL support:** Horizontal scrolling direction reversed in RTL

Notes

- **Browser inconsistency:** Native scrollbars vary greatly (Windows vs Mac vs mobile). Custom scrollbars solve this
- **Accessibility first:** Ensure keyboard and screen reader scrolling still work with custom scrollbars

- **Performance:** Custom scrollbars can impact performance with large content. Test thoroughly
- **Overlay vs gutter:** Overlay scrollbars don't affect layout but may obscure content. Gutter scrollbars are safer
- **Mobile scrollbars:** On mobile, scrollbars are typically hidden. Consider if custom scrollbars are needed
- **Implementation:** CSS `scrollbar - *` properties (WebKit only) or JavaScript libraries like SimpleBar, OverlayScrollbars

#35 Collapsible

A single section with a trigger that expands or collapses its content. Base primitive used by Accordion.

Description

Collapsible (also called disclosure or expander) shows or hides content based on user interaction with a trigger. It's a fundamental progressive disclosure pattern that reduces cognitive load by hiding secondary information until needed.

Collapsible is the base component for Accordion—a single expandable section versus multiple coordinated sections.

Main Features

- **Trigger control:** Button or heading that toggles expanded/collapsed state
- **Expandable content:** Content area that shows when expanded, hides when collapsed
- **Visual indicator:** Icon (chevron, plus/minus) shows current state
- **Smooth transition:** Content expands/collapses with animation
- **Initial state:** Can start expanded or collapsed

Secondary Features

Accessibility

- **ARIA pattern:** Uses button with aria-expanded and aria-controls
- **Keyboard accessible:** Trigger is keyboard operable
- **State announcement:** Screen readers announce expanded/collapsed state
- **Focus management:** Focus remains on trigger after expansion/collapse

Keyboard Navigation

- **Tab to trigger:** Tab moves focus to trigger button
- **Enter/Space toggle:** Enter or Space expands/collapses content
- **Focus persistence:** Focus stays on trigger after toggling

Touch-screen

- **Touch target:** Trigger has minimum 44x44px tap target
- **Entire header tappable:** Entire trigger area is tappable
- **Touch feedback:** Immediate visual feedback on tap

Responsive Behavior

- **Flexible width:** Adapts to container width
- **Content overflow:** Long content scrolls or wraps within expanded area
- **Icon positioning:** Expand/collapse icon positioned consistently

Animation & Transitions

- **Smooth expansion:** Content animates smoothly when expanding
- **Height transition:** Transitions from 0 to auto height (or vice versa)
- **Reduced motion:** Respects prefers-reduced-motion preference
- **Icon rotation:** Indicator icon rotates (chevron down → up, plus → minus)

Test Scenarios

- **Collapsible renders:** Trigger and collapsed content area display
- **Initial collapsed:** Content starts collapsed (if configured)
- **Initial expanded:** Content starts expanded (if configured)
- **Toggle expands:** Clicking trigger expands collapsed content
- **Toggle collapses:** Clicking trigger collapses expanded content
- **Icon indicator:** Icon shows correct state (down when collapsed, up when expanded)
- **Icon animation:** Icon rotates smoothly during transition
- **Content animation:** Content expands/collapses with smooth height transition
- **ARIA expanded:** aria-expanded attribute updates (true/false)
- **Keyboard toggle:** Enter/Space on focused trigger toggles state
- **Screen reader:** State change announced to screen readers
- **Focus management:** Focus remains on trigger after toggling
- **Reduced motion:** When user prefers reduced motion, transition is instant
- **Multiple instances:** Multiple collapsibles can be expanded simultaneously (unlike accordion)

Notes

- **Collapsible vs Accordion:** Collapsible is a single section. Accordion is multiple collapsibles with coordinated behavior (typically only one open at a time)
- **Trigger labeling:** Use clear labels that describe what will be revealed (“Show more details”, “Expand settings”)
- **Initial state:** Choose based on content importance—critical content should start expanded
- **Animation duration:** 200-300ms is typical. Too fast feels jarring, too slow feels sluggish
- **Height transitions:** CSS transition from height: 0 to height: auto can be tricky. Use max-height or JavaScript for smooth transitions

- **Icon conventions:** Chevron down/up is universal. Plus/minus also common. Be consistent across application
- **Content lazy loading:** For heavy content, consider loading it only when expanded (performance optimization)

#36 Resizable

A container with one or more draggable handles allowing the user to resize panels. Also called *Split Pane* or *Resizable Panel*.

Description

Resizable allows users to adjust the size of one or more panels within a container by dragging divider handles. Common in code editors (sidebar vs main content), email clients (inbox vs message), and dashboards (adjustable widgets). Resizable panels give users control over their workspace layout, allowing them to allocate screen space based on their current task.

Main Features

- **Draggable handle:** Visual divider users can drag to resize panels
- **Multiple panels:** Typically two or more resizable sections
- **Flexible sizing:** Panels adjust size as handle is dragged
- **Constraints:** Minimum and maximum sizes prevent panels from collapsing entirely
- **Layout preservation:** Maintains panel proportions on window resize (optional)

Secondary Features

Accessibility

- **Keyboard resizing:** Handle can be operated via keyboard
- **Focus indicator:** Clear focus ring on resize handle

- **ARIA semantics:** Handle has appropriate role and label
- **State announcement:** Size changes announced to screen readers (optional)

Keyboard Navigation

- **Tab to handle:** Tab moves focus to resize handle
- **Arrow keys:** Arrow keys adjust panel size incrementally
- **Page Up/Down:** Larger size adjustments
- **Home/End:** Jump to min/max sizes

Touch-screen

- **Touch-friendly handle:** Handle large enough for touch (minimum 44x44px tap area)
- **Drag support:** Smooth dragging on touch devices
- **Visual feedback:** Handle shows active state during drag

Responsive Behavior

- **Horizontal/vertical:** Panels can be arranged horizontally or vertically
- **Nested resizing:** Can have nested resizable containers
- **Collapse behavior:** Panels can collapse to minimum size or hide completely
- **Responsive defaults:** Initial sizes adapt to viewport

Variants & Features

- **Two-panel split:** Most common (left/right or top/bottom)
- **Multi-panel:** Three or more panels with multiple handles
- **Collapsible panels:** Panels can collapse to zero width/height
- **Size persistence:** Remembers user's sizing preferences (localStorage)

Test Scenarios

- **Panels render:** Multiple panels display with resize handle between them
- **Handle visibility:** Resize handle is visible and indicates drag capability
- **Drag resize:** Dragging handle adjusts panel sizes

- **Panel constraints:** Panels cannot resize below minimum or above maximum
- **Smooth dragging:** Resizing is smooth without lag or jumping
- **Cursor change:** Cursor changes to resize indicator on handle hover
- **Keyboard focus:** Handle can receive keyboard focus
- **Keyboard resize:** Arrow keys adjust panel size when handle focused
- **Touch drag:** Handle can be dragged on touch devices
- **Layout preservation:** Panel proportions maintained on window resize (if configured)
- **Collapse behavior:** Panels collapse when dragged to minimum (if configured)
- **Multi-handle:** With 3+ panels, each handle independently controls adjacent panels
- **Screen reader:** Handle labeled appropriately for screen readers
- **Size persistence:** Panel sizes restored from previous session (if configured)

Notes

- **Handle affordance:** Make handle visually obvious—use hover effects, grab cursor, or visible divider
- **Minimum sizes:** Always enforce minimum sizes to prevent panels from disappearing entirely
- **Keyboard support:** Essential for accessibility. Arrow keys for fine control, Page Up/Down for coarse
- **Touch considerations:** Wider handle areas for touch. Consider if dragging is the best interaction on mobile
- **Performance:** Resizing can be expensive with complex content. Consider debouncing or using CSS transforms
- **Persistence:** Store user's preferred sizes in localStorage for better UX across sessions
- **Snap points:** Optional snap-to sizes for common layouts (50/50, 70/30, etc.)
- **Implementation:** Libraries like react-resizable-panels, split.js handle the complexity

#37 Empty State

Placeholder displayed when a content area has no data to show. Typically includes an illustration, a message, and an optional action.

Description

Empty State communicates that there's no content currently available and helps users understand why and what to do next. Instead of showing a blank screen (which feels broken), empty states provide context, guidance, and often a path forward. They turn potentially frustrating moments into opportunities for engagement.

Main Features

- **Visual element:** Illustration, icon, or image representing the empty state
- **Message:** Clear explanation of why content is missing
- **Optional action:** Button or link to help user populate the area
- **Friendly tone:** Encouraging rather than negative messaging

Secondary Features

Accessibility

- **Semantic structure:** Uses appropriate HTML structure
- **Alt text:** Image/illustration has descriptive alt text
- **Action accessibility:** Call-to-action button is keyboard accessible
- **Clear messaging:** Text clearly explains the state without relying on visuals alone

Responsive Behavior

- **Flexible sizing:** Adapts to container size
- **Image scaling:** Illustration scales appropriately

- **Mobile optimization:** Simpler or smaller illustrations on mobile

Variants & Features

- **First-use empty:** User hasn't created content yet ("Get started by creating your first project")
- **No results:** Search or filter returned nothing ("No results for 'widget'. Try different keywords")
- **Error empty:** Error prevented content from loading ("Unable to load messages. Try again")
- **Permission empty:** User lacks access ("You don't have permission to view this content")
- **Completed empty:** All items completed/cleared ("Inbox zero! You're all caught up")

Test Scenarios

- **Empty state renders:** When no content exists, empty state displays
- **Illustration display:** Visual element (image, icon, illustration) shows
- **Message text:** Clear, helpful message explains why area is empty
- **Action button:** When provided, call-to-action button displays
- **Action trigger:** Clicking action button triggers appropriate callback
- **Alt text:** Illustration has descriptive alt text for screen readers
- **Semantic HTML:** Uses appropriate HTML structure (not just styled divs)
- **Responsive sizing:** Empty state scales appropriately for screen size
- **Tone:** Message tone is friendly and helpful, not negative
- **State-specific messaging:** Different messages for different empty scenarios

Notes

- **Don't show blank screens:** Always show empty state rather than blank white space
- **Be specific:** "No projects yet" is better than "Empty". Explain the context
- **Provide action:** Include "Create project" or "Import data" button when applicable

- **Tone matters:** Avoid negative words. “No data” → “Ready to add your first item?”
- **Different scenarios:** First use vs no results vs errors need different messages and actions
- **Illustration style:** Match your brand. Simple icons work for functional apps, friendly illustrations for consumer apps
- **Loading vs empty:** Show loading state while fetching. Show empty state only when confirmed no data exists
- **Search results:** For no search results, suggest checking spelling or trying different keywords
- **Zero state:** Another term for empty state, especially for first-use scenarios

#38 Breadcrumb

Displays the user’s current location within a navigational hierarchy as a trail of links.

Description

Breadcrumbs show users where they are in the site’s hierarchy and provide one-click access to parent pages. They’re especially valuable in deep content structures (e-commerce categories, documentation, file systems) where users need context about their location and easy navigation back up the hierarchy.

Main Features

- **Hierarchical trail:** Shows path from root to current page (Home > Category > Subcategory > Page)
- **Clickable links:** Each level (except current) is a link to that page
- **Current page indicator:** Last item represents current page (not linked)
- **Separators:** Visual separators between items (>, /, •)

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<nav>` with `<ol>` (ordered list)
- **ARIA landmark:** `aria-label="Breadcrumb"` identifies navigation type
- **Current page:** `aria-current="page"` on current item
- **Keyboard accessible:** All links are keyboard navigable

Keyboard Navigation

- **Tab navigation:** Tab moves through breadcrumb links
- **Enter activation:** Enter follows link
- **Skip links:** Screen reader users can skip breadcrumb navigation

Touch-screen

- **Touch targets:** Each link has adequate tap target (minimum 44x44px including padding)
- **Link spacing:** Sufficient space between links to prevent mis-taps

Responsive Behavior

- **Truncation:** Long breadcrumbs truncate or show ellipsis on narrow screens
- **Wrapping:** May wrap to multiple lines or condense to show only last few items
- **Mobile collapse:** On mobile, may show only “< Back” or last 2-3 items

Variants & Features

- **Separator styles:** Chevron (`>`), slash (`/`), bullet (`•`), or custom
- **Truncation:** Long middle items can be truncated with ellipsis
- **Dropdown overflow:** Collapsed items accessible via dropdown menu
- **Icon support:** Items can include icons (home icon for root, etc.)

Test Scenarios

- **Breadcrumb renders:** Full path displays from root to current page
- **Links functional:** Clicking breadcrumb link navigates to that page
- **Current page:** Last item is current page and not clickable
- **Separators:** Separators display between all items
- **Semantic HTML:** Uses `<nav>` with `<ol>` structure
- **ARIA label:** nav element has `aria-label="Breadcrumb"`
- **Current indicator:** Current page has `aria-current="page"`
- **Keyboard navigation:** Tab moves through links, Enter activates
- **Touch targets:** Links have adequate tap target size
- **Responsive truncation:** Long breadcrumbs truncate or collapse on narrow screens
- **Home link:** First item (home/root) links to homepage
- **Screen reader:** Breadcrumb trail is announced appropriately
- **Separator semantics:** Separators are decorative (`aria-hidden`) not part of link text

Notes

- **Breadcrumb vs Pagination:** Breadcrumbs show hierarchy/location. Pagination shows sequence (page 1 of 10)
- **Don't replace primary nav:** Breadcrumbs supplement main navigation, don't replace it
- **Always start at root:** First item should be homepage or app root
- **Current page:** Never link the current page—it's redundant and confusing
- **Mobile considerations:** On mobile, consider showing just "< Category" back link instead of full breadcrumb
- **Separator choice:** > (chevron) is most common and universally understood
- **Truncation strategy:** When truncating, keep first and last items, collapse middle
- **E-commerce pattern:** Category > Subcategory > Product is the classic breadcrumb use case

- **Don't use for flat sites:** If your site is only 2 levels deep, breadcrumbs add clutter without value

#39 Menu

A simple list of links or action buttons. Serves as a building block for navigation components. Items can be grouped and include icons or descriptions.

Description

Menu is a primitive navigation and action component that renders a vertical list of clickable items. Unlike Dropdown Menu (which is anchored to a trigger and floats), Menu is typically an always-visible structural element—used as a building block for sidebars, navigation rails, context panels, and other persistent navigation structures. Items can represent links (navigation) or actions (commands), and can be grouped under labeled sections with dividers.

Main Features

- **Item list:** Vertical list of clickable items (links or action buttons)
- **Active/selected state:** Visual highlight for the currently active item
- **Item grouping:** Items can be organized into labeled sections with separators
- **Icon support:** Items can include a leading icon for visual scanning
- **Disabled items:** Individual items can be marked as non-interactable
- **Semantic rendering:** Links render as `<a>` elements; actions render as `<button>` elements

Secondary Features

Accessibility

- **Semantic list structure:** Uses `<ul>` and `<li>` for proper list semantics
- **Role attributes:** Menu role with `menuitem`, `menuitemcheckbox`, or `menuitemradio` for items as appropriate

- **Active indicator:** `aria-current="page"` (for navigation links) or `aria-selected` for selected items
- **Disabled semantics:** Disabled items use `aria-disabled="true"` and are not keyboard-focusable
- **Group labels:** Section headings use `aria-label` or `role="group"` to associate group title with items

Keyboard Navigation

- **Tab to reach menu:** The menu container is navigable via Tab
- **Arrow key navigation:** Up/Down arrow keys move focus between items within the menu
- **Home/End:** Jump to first/last enabled item
- **Enter/Space to activate:** Activates the focused item (follows link or fires action)
- **Letter key jump:** Typing a letter moves focus to the next item starting with that character

Touch-screen

- **Touch-friendly targets:** Each item meets minimum tap target size (44×44px)
- **Spacing:** Adequate vertical padding between items to prevent mis-taps
- **Active feedback:** Visual press feedback on touch (active state)

Responsive Behavior

- **Flexible width:** Fills its container or respects a defined max-width
- **Text truncation:** Long item labels truncate with ellipsis rather than wrapping (configurable)
- **Compact mode:** Reduced padding mode for dense navigation panels (icon-only or smaller targets)
- **Collapsible groups:** On narrow layouts, groups can be collapsible to save space

Internationalization

- **RTL support:** Icons and indicators appear on the correct side for right-to-left languages
- **Multi-byte character support:** Item labels render correctly across all character sets
- **Localizable content:** All visible strings (labels, group headings) are consumer-provided and localizable

Variants & Features

- **Navigation menu:** Items are links; one item is marked as current page
- **Action menu:** Items are buttons that fire callbacks; no persistent selection
- **Single-select menu:** One item can be selected at a time (like a settings option group)
- **Multi-select menu:** Multiple items can be toggled on/off (checkmark items)
- **Icon-only / collapsed:** Shows only icons with tooltips on hover (sidebar collapsed state)
- **Item badge/count:** Optional count badge on items (unread messages, notifications)
- **Nested sub-menus:** Items can expand to reveal a nested menu (tree-style navigation)

Test Scenarios

- **Menu renders:** All items display in correct order
- **Link items:** Link items render as `<a>` tags and navigate on click/Enter
- **Action items:** Action items render as `<button>` tags and fire callbacks on click/Enter
- **Active item:** The active/current item is visually highlighted and has correct ARIA attribute
- **Disabled items:** Disabled items are visually distinct and cannot be clicked or keyboard-focused
- **Group headings:** Section labels appear above their group of items
- **Group separators:** Dividers render between groups

- **Icon rendering:** Icons display to the left (or right in RTL) of item labels
- **Keyboard arrow navigation:** Up/Down arrows move focus through items
- **Home/End keys:** Focus jumps to first/last enabled item
- **Letter key jump:** Typing a letter moves focus to matching item
- **Badge display:** When configured, badge/count displays on item
- **Single-select behavior:** Selecting one item deselects the previously selected item
- **Multi-select behavior:** Items toggle independently without affecting other items
- **RTL layout:** Icons and active indicators flip correctly in RTL
- **Callback execution:** onItemClick fires with item identifier when item is activated
- **Screen reader:** Item labels and states (active, disabled) are announced correctly

Notes

- **Menu vs Dropdown Menu:** Menu is typically always visible (structural navigation). Dropdown Menu is floating and triggered by a button click
- **Menu vs Navbar:** Navbar is horizontal and typically contains the Menu as a vertical list within a mobile drawer or sidebar
- **Active state management:** Active state should be driven by the parent (router location, selected state) not managed internally
- **Icon consistency:** Use same icon size and style throughout a menu for visual consistency
- **Sub-menus on mobile:** Nested sub-menus are difficult on touch. Consider flat navigation with breadcrumb for mobile
- **Performance:** For very long menus (50+ items), consider virtualization
- **Keyboard vs mouse:** Arrow key navigation is essential for menus (ARIA menu pattern requires it)

#40 Select

Allows the user to choose one option from a predefined list. Form input equivalent of a native `<select>`.

Specializations: Multi-Select (multiple selections with checkboxes), Searchable Select (filterable list), Grouped Select (options organized under labeled groups)

Naming note: Also called *Dropdown* or *Picker* in other libraries. Distinct from *Dropdown Menu*, which triggers actions rather than selecting a form value.

Description

Select is a form input component that allows users to choose one option (or multiple, in the multi-select variant) from a predefined list. Unlike *Dropdown Menu* (which triggers actions), Select captures a form value. It replaces or enhances the native `<select>` element, providing consistent cross-browser styling, richer option content, search/filter capability, and grouped options while maintaining all expected form behaviors.

Main Features

- **Single selection:** User picks one option from the list
- **Selected value display:** Shows the current selection (or placeholder) when closed
- **Toggle open/closed:** Clicking the trigger opens/closes the option list
- **Option list:** Full list of selectable options presented in a floating panel
- **Selection closes list:** Choosing an option selects it and collapses the list
- **Placeholder:** Text shown when no value is selected (“Select a country...”)
- **Label association:** Visible label linked to the select for identification
- **Form integration:** Works as a controlled or uncontrolled form element; participates in form submit

Secondary Features

Accessibility

- **ARIA combobox pattern:** Uses role="combobox" on trigger, role="listbox" on option list, role="option" on items
- **Expanded state:** aria-expanded reflects open/closed state
- **Selected option:** aria-selected="true" on the selected option; selected value referenced via aria-activedescendant
- **Label linkage:** Label linked to trigger via htmlFor / aria-labelledby
- **Required indication:** aria-required="true" and visual required marker when field is mandatory
- **Error state:** aria-invalid="true" and aria-describedby linking to error message
- **Screen reader announcements:** Option list open/close and selection changes are announced

Keyboard Navigation

- **Tab to reach:** Select trigger receives focus via Tab; Tab again exits
- **Space/Enter to open:** Opens option list when trigger is focused
- **Arrow keys navigate options:** Up/Down arrows move through options (works both open and closed for quick navigation)
- **Home/End keys:** Jump to first/last option
- **Type-ahead:** Typing characters moves focus to matching option
- **Enter to select:** Selects focused option and closes list
- **Escape to close:** Closes list without changing selection; returns focus to trigger

Touch-screen

- **Adequate touch targets:** Trigger and options meet 44×44px minimum
- **Native picker option:** On mobile, can delegate to native OS picker (better UX for simple selects)
- **Scrollable list:** Long option lists scroll smoothly within the floating panel

- **No hover dependency:** All interactions work without hover

Responsive Behavior

- **Flexible width:** Trigger adapts to container width; option panel matches or exceeds trigger width
- **Viewport-aware positioning:** Option panel opens below or above trigger based on available space
- **Maximum list height:** Long option lists constrain to a max height with internal scroll
- **Text truncation:** Long option labels truncate gracefully in the trigger display

Internationalization

- **RTL layout:** Icon, label, and option alignment flip correctly for right-to-left languages
- **Multi-byte characters:** Handles all character sets in option labels and search input
- **Locale-aware sorting:** Option lists can be sorted using locale-appropriate collation
- **Placeholder localization:** Placeholder text is consumer-provided and localizable

Validation & Feedback

- **Error state:** Accepts `hasError` / `errorMessage` props; renders error styling and message below
- **Success state:** Accepts `isValid` prop; renders success styling (e.g., green border, checkmark)
- **Required field:** Visual marker (e.g., asterisk) and semantic `required` attribute
- **Callbacks:** `onChange` fires with new value on selection; `onOpen`/`onClose` fire on panel toggle; `onBlur`/`onFocus` fire on focus changes

Test Scenarios

- **Renders closed:** Shows trigger with placeholder or selected value
- **Opens on click:** Clicking trigger shows option list
- **Opens on keyboard:** Space or Enter while focused opens list
- **Options display:** All options render in the list
- **Option selection:** Clicking an option selects it, updates trigger, closes list
- **Keyboard selection:** Arrow keys navigate; Enter selects focused option
- **Escape closes:** Closes list without changing selection
- **Placeholder:** When no value, placeholder text is shown
- **Disabled state:** Disabled select cannot be opened or interacted with
- **Disabled options:** Individual disabled options are skipped in keyboard navigation and cannot be selected
- **Error state:** Error styling and message display when `hasError` is true
- **Required indicator:** Required marker shows when field is mandatory
- **Outside click closes:** Clicking outside the component closes the list
- **Viewport positioning:** Panel opens above trigger when insufficient space below
- **Type-ahead:** Typing letters focuses matching options
- **Label association:** Clicking label focuses the select trigger
- **Callback execution:** `onChange` fires with correct value; `onOpen/onClose` fire correctly
- **Multi-select:** Multiple options can be selected simultaneously; trigger shows count or tags
- **Searchable:** Typing in search input filters option list in real time
- **Grouped:** Group headings render above their options; group headings are not selectable
- **RTL:** Layout and icons flip correctly in RTL
- **Form integration:** Selected value submitted with form; participates in form validation

Notes

- **Native vs custom:** On mobile, native `<select>` often provides better OS-native UX. Consider offering a native fallback
- **Search threshold:** Add search for lists with 10+ options; it quickly becomes essential
- **Multi-select complexity:** Multi-select is significantly more complex. Treat it as a distinct component variant with separate testing
- **Positioning library:** Viewport-aware floating panel positioning (flipping, overflow detection) benefits from libraries like Floating UI or Popper.js
- **Controlled vs uncontrolled:** Support both patterns—controlled (value + onChange) and uncontrolled (defaultValue)
- **Virtualization:** For very long option lists (100+), virtualize the rendered options for performance

Multi-Select Specialization

Additional Features:

- Checkboxes in option list allow multiple simultaneous selections
- Trigger displays selected count (“3 selected”) or inline tags for each selected item
- “Select all” / “Clear all” controls available within the option panel
- Individual selected items removable via tag × button without opening the list
- `aria-multiselectable=“true”` on listbox; each selected option has `aria-selected=“true”`

Additional Test Scenarios:

- **Multiple selections:** Multiple options can be selected without closing the list
- **Trigger count display:** Trigger shows correct selected count as selections change
- **Tag display:** When tags are shown, each selected item renders as a removable tag in the trigger
- **Tag removal:** Clicking × on a tag deselects that item without opening the list

- **Select all:** “Select all” control selects every non-disabled option
- **Clear all:** “Clear all” control deselects all selected options
- **aria-multiselectable:** Listbox has `aria-multiselectable=“true”`
- **aria-selected per item:** Each selected option has `aria-selected=“true”`; deselected options have `aria-selected=“false”`

Searchable Select Specialization

Additional Features:

- Text input appears inside the open panel, allowing users to filter visible options in real time
- “No results” empty state renders when the filter query matches no options
- Search input clears automatically on close or on selection
- Optional minimum character threshold before filtering begins

Additional Test Scenarios:

- **Search input renders:** When panel opens, a text input for filtering is present
- **Real-time filtering:** Typing in the search input narrows the visible option list immediately
- **No results state:** When filter matches nothing, a “No results” message displays
- **Search clears on close:** Closing the panel without selecting resets the search input
- **Search clears on select:** Selecting an option resets the search input
- **Minimum threshold:** If configured, filtering does not begin until the minimum character count is reached

Grouped Select Specialization

Additional Features:

- Options organized under non-selectable, labeled group headings
- Group headings use `role=“group”` with `aria-label` for accessibility

- Visual separator between groups for clear delineation
- Type-ahead navigation skips over group heading labels and moves directly to options

Additional Test Scenarios:

- **Group headings render:** Group label text appears above its group of options
- **Group headings not selectable:** Clicking or keyboard-focusing a group heading does not select it
- **Options within groups:** All options within a group are selectable as normal
- **Separators between groups:** Visual dividers render between each group
- **Type-ahead skips headings:** Typing a character jumps to matching options, not group labels
- **ARIA group role:** Group container has role=“group” with correct aria-label

#41 Combobox / Autocomplete

Text input combined with a suggestion dropdown. User can type freely or select from suggestions.

Specializations: Search Bar (search-optimized combobox with results pane), Autocomplete (suggestions based on input, completes the value)

Naming note: The distinction between Combobox and Autocomplete is subtle — Combobox allows free input + list selection, Autocomplete completes the typed value. Both share the same base pattern.

Description

Combobox combines a free-text input with a suggestions dropdown. Unlike Select, the user can type any value in addition to choosing from the list. The suggestions filter or populate dynamically based on what the user types. This pattern is ideal for large option sets (country names, user search, address lookup) where a static list would be impractical and type-to-filter is the primary UX.

Main Features

- **Free text input:** User can type any value (not restricted to predefined options)
- **Dynamic suggestions:** Dropdown appears and filters as user types
- **Suggestion selection:** User can select from suggestions to populate the input
- **Dismiss on outside click:** Suggestion list closes when focus leaves the component
- **Clearable:** Input can be cleared with a clear button or Escape
- **Minimum character trigger:** Suggestions appear after a configurable number of characters (often 1–3)

Secondary Features

Accessibility

- **ARIA combobox pattern:** role=“combobox” on input, aria-expanded, aria-autocomplete, aria-haspopup=“listbox”
- **Listbox:** Suggestions container uses role=“listbox”; items use role=“option”
- **Active descendant:** aria-activedescendant references the currently highlighted suggestion
- **Live region:** Changes to the suggestion list announced via aria-live or status announcements (“5 results available”)
- **Label association:** Input linked to a visible label via htmlFor / aria-labelledby
- **Error/success states:** aria-invalid and aria-describedby for error messages

Keyboard Navigation

- **Type to filter:** Typing in the input updates suggestions
- **Down Arrow to enter list:** Pressing Down Arrow while input is focused moves focus into the suggestion list
- **Arrow keys in list:** Up/Down navigate through suggestions
- **Enter to select:** Selects highlighted suggestion, populates input, closes list

- **Escape:** Clears suggestion highlight, closes list (may clear input depending on variant)
- **Tab:** Accepts suggestion and moves focus to next element (configurable), or closes list and moves focus
- **Home/End in input:** Moves cursor within the text input (standard text input behavior)

Touch-screen

- **Touch-friendly suggestions:** Suggestion items meet 44×44px tap target
- **Soft keyboard handling:** On mobile, soft keyboard opening does not obscure suggestions (suggestions scroll into view)
- **Tap to select:** Tapping a suggestion selects it and dismisses keyboard appropriately

Responsive Behavior

- **Suggestion panel width:** Panel matches or exceeds input width
- **Viewport-aware:** Panel flips above input when insufficient space below
- **Maximum panel height:** Suggestion list constrained with internal scroll for long results
- **Loading state:** Shows spinner or skeleton within panel while fetching async suggestions

Internationalization

- **RTL input and suggestions:** Text direction and icon placement adapt for RTL languages
- **Locale-aware filtering:** Filtering handles accented characters and locale-specific matching
- **Multi-byte support:** Input and suggestions handle all character sets

Validation & Feedback

- **Error state:** hasError + errorMessage for invalid input
- **Loading indicator:** Visual indicator while suggestions are being fetched

- **No results state:** “No results found” message when suggestions list is empty
- **Callbacks:** onChange fires on input change; onSelect fires when suggestion is chosen; onBlur/onFocus for focus events; onSearch for debounced search queries

Test Scenarios

- **Input renders:** Text input displays with label and optional placeholder
- **No suggestions initially:** Suggestion list is hidden before minimum character threshold is reached
- **Suggestions appear:** Typing triggers suggestion list to appear
- **Filtering:** Suggestions narrow as user continues typing
- **Suggestion selection via click:** Clicking a suggestion populates the input and closes the list
- **Suggestion selection via keyboard:** Arrow keys navigate list; Enter selects highlighted suggestion
- **Down Arrow enters list:** Down Arrow from input moves focus into suggestions
- **Escape closes:** Escape closes suggestion list; focus returns to input
- **Outside click closes:** Clicking outside the component closes the suggestion list
- **Clear button:** Clear button removes input value and closes suggestions
- **No results state:** When no suggestions match, “No results” message displays
- **Loading state:** While fetching async suggestions, loading indicator displays
- **ARIA expanded:** aria-expanded is true when list is open, false when closed
- **Active descendant:** aria-activedescendant updates as user arrows through suggestions
- **Live region:** Screen reader is informed of suggestion count changes
- **Error state:** Error styling and message display when hasError is true
- **Minimum threshold:** Suggestions do not appear until minimum characters are typed
- **Callback execution:** onSelect fires with selected suggestion value; onSearch fires with debounced query
- **RTL:** Input direction and icon placement correct in RTL

Notes

- **Debouncing:** Async suggestion fetching should be debounced (200–400ms) to avoid excessive network requests
- **Combobox vs Select:** If the user must always choose from a list (no free entry), use Select. If they can type any value, use Combobox
- **Inline autocomplete:** Inline completion (ghost text) is a distinct interaction from list-based suggestions—both can coexist
- **Async suggestions:** For remote data (e.g., address lookup), handle loading, error, and empty states explicitly
- **Accessibility complexity:** The ARIA combobox pattern is nuanced. Reference the ARIA Authoring Practices Guide (APG) combobox pattern carefully
- **Mobile keyboards:** On mobile, suggestion list may be obscured by keyboard. Use `scroll into view` and test thoroughly
- **Clear vs Escape:** Be explicit about whether Escape clears the input or just closes the list. Document the behavior clearly

Search Bar Specialization

Additional Features:

- Input styled as a dedicated search field with leading magnifier icon and trailing clear button
- Suggestions panel may show categorized result groups (recent searches, popular queries, matched items) rather than a flat list
- Pressing Enter submits the search query (navigates to results page or fires search callback) rather than selecting a suggestion
- Results pane may replace or update the main content area rather than floating over it
- Optional recent search history shown before the user begins typing

Additional Test Scenarios:

- **Search icon:** Magnifier icon renders in the input field

- **Clear button:** Clear (×) button appears when input has a value and clears on click
- **Enter submits:** Pressing Enter fires the search/submit callback with the current query value
- **Categorized suggestions:** When configured, suggestion panel groups results under labeled categories
- **Recent searches:** When no query is entered, recent search history displays (if configured)
- **Results update on selection:** Selecting a suggestion triggers a search/navigation, not just input population

Autocomplete Specialization

Additional Features:

- Completes the user's input inline with ghost (greyed-out) text after the cursor, based on the top-ranked suggestion
- User accepts the inline completion by pressing Tab or Right Arrow
- Pressing any other character replaces the ghost text with the next matching completion
- More constrained than base Combobox—user is guided toward completing a single known value rather than free entry
- Ghost text is visually distinct from typed text (lower contrast, italic, or muted color)

Additional Test Scenarios:

- **Ghost text renders:** Top suggestion appears as inline ghost text after the cursor as user types
- **Tab accepts completion:** Pressing Tab replaces ghost text with the full suggestion and moves focus
- **Right Arrow accepts completion:** Pressing Right Arrow at end of typed text accepts the ghost text
- **Other keys replace ghost:** Typing a different character updates ghost text to next matching suggestion

- **Ghost text styling:** Ghost text is visually distinct (muted color) from user-typed text
- **No ghost when no match:** Ghost text does not appear when no suggestion matches current input

#42 Dropdown Menu

A trigger element (button, icon, or text) that opens a floating list of actions or navigation options.

Specialization: Context Menu (same pattern but triggered by right-click or long-press instead of a button)

Naming note: Distinct from Select (form value selection). Dropdown Menu triggers actions or navigation; Select captures a form value.

Description

A dropdown menu (also called select, picker, or combobox) allows users to choose one option from a list of predefined choices. When closed, it displays the currently selected value. When opened, it shows all available options for selection.

Main Features

- **Single selection:** User can select one option from the list
- **Selected value display:** Shows currently selected option when closed
- **Toggle open/closed:** User can open and close the dropdown
- **Option list presentation:** Displays all available options when open
- **Selection action:** Clicking an option selects it and closes the dropdown
- **Visual state feedback:** Clearly distinguishes between idle, focused, open, disabled, error, and success states
- **Label association:** Supports a visible label that identifies the dropdown's purpose

- **Placeholder option:** Optional default text when no selection is made (e.g., “Select an option...”)

Secondary Features

Accessibility

- **Programmatic label association:** Label is linked to dropdown for screen reader announcement
- **ARIA attributes:** Proper roles (combobox, listbox, option) and states (expanded, selected)
- **Keyboard accessible:** Fully operable via keyboard
- **Focus indicator:** Clear visual indicator when dropdown has keyboard focus
- **Screen reader announcements:** Selected option and list state changes are announced
- **Option descriptions:** Support for additional descriptive text on options

Keyboard Navigation

- **Tab navigation:** Can be reached and exited via Tab/Shift+Tab
- **Space/Enter to open:** Space or Enter key opens the dropdown when focused
- **Arrow key navigation:** Up/Down arrows navigate through options (can work when closed or open)
- **Home/End keys:** Jump to first/last option
- **Type-ahead/search:** Typing letters jumps to matching options
- **Escape to close:** Escape key closes dropdown without changing selection
- **Enter to select:** Enter key selects currently focused option and closes dropdown

Touch-screen

- **Touch-friendly target:** Adequate size for touch interaction on trigger and options

- **Native picker support:** Option to use native OS picker on mobile devices (better UX on mobile)
- **Scroll support:** Option list scrolls smoothly when it contains many items
- **Touch-friendly option spacing:** Options have sufficient height/padding for easy touch selection

Responsive Behavior

- **Flexible width:** Adapts to container width constraints
- **Dropdown positioning:** Intelligently positions dropdown (above/below) based on available viewport space
- **Option list sizing:** Limits visible options with scroll when list is very long
- **Text truncation:** Long option text truncates gracefully with ellipsis or wrapping

Internationalization

- **Bi-directional text support:** Handles RTL languages correctly in trigger and options
- **Multi-byte character support:** Properly displays characters from various language sets
- **Locale-appropriate sorting:** Options can be sorted according to locale rules

Validation & Feedback

- **Error state display:** Accepts and displays error state with associated error message
- **Success state display:** Accepts and displays success state
- **Required field indication:** Visual marker when selection is mandatory
- **Interaction callbacks:** Notifies when selection changes (onChange), when opened/closed (onOpen/onClose), when focused/blurred

Advanced Features

- **Multi-select variant:** Allow selection of multiple options (checkboxes in list)

- **Searchable/filterable:** Include search input to filter long option lists
- **Option grouping:** Support for grouped options with group labels
- **Custom option rendering:** Allow custom content in options (icons, descriptions, badges)
- **Disabled options:** Individual options can be disabled/unselectable

Test Scenarios

- **Open/close toggle:** Clicking trigger opens dropdown; clicking again (or outside) closes it
- **Option selection:** Clicking an option selects it, updates displayed value, and closes dropdown
- **Keyboard open:** Pressing Space or Enter when focused opens the dropdown
- **Keyboard navigation:** Arrow keys move focus through options
- **Keyboard selection:** Pressing Enter selects focused option and closes dropdown
- **Escape closes:** Pressing Escape closes dropdown without changing selection
- **Type-ahead:** Typing letters focuses matching option
- **Disabled state:** When disabled, dropdown cannot be opened or interacted with
- **Required validation display:** When marked required and no selection made, displays appropriate state
- **Error state display:** When error state is true, error styling is applied and message displayed
- **Outside click closes:** Clicking outside the dropdown closes it
- **Focus management:** When opened, focus moves to first option or selected option; when closed, focus returns to trigger
- **Viewport positioning:** Dropdown opens above trigger if insufficient space below
- **Callback execution:** onChange fires when selection changes, onOpen/onClose fire appropriately
- **Label association:** Clicking label focuses the dropdown trigger

Notes

- **Native vs custom:** On mobile, consider using native `<select>` for better UX; on desktop, custom implementations often provide better control
- **Search/filter:** For lists with 10+ options, consider adding search functionality (this is a specialized variant)
- **Multi-select:** Completely different interaction pattern; should be treated as separate component
- **Autocomplete:** Dropdown with free-text input is a different component (combobox/autocomplete)
- **Positioning library:** Complex positioning logic (handling viewport bounds, scroll containers) often benefits from dedicated libraries

#43 Tooltip

A small, non-interactive informational overlay triggered by hover or focus. Disappears when the user moves away. Does not contain interactive elements.

***Naming note:** Distinct from Popover — Tooltip is hover/focus-only, non-interactive, and ephemeral. Popover is click-triggered and can contain interactive content.*

Description

Tooltip provides brief, supplementary text that appears when a user hovers over or focuses on a UI element. It is always non-interactive—users cannot click, tab to, or interact with a tooltip’s content. Tooltips are used to label icon buttons, explain truncated text, or surface keyboard shortcuts. Because they are inaccessible to touch users by default, they should never contain information required to complete a task.

Main Features

- **Hover trigger:** Tooltip appears when the user hovers over the trigger element

- **Focus trigger:** Tooltip also appears when the trigger receives keyboard focus (ensuring keyboard accessibility)
- **Auto-dismiss:** Tooltip disappears when the cursor leaves the trigger or focus moves away
- **Text content only:** Tooltips contain only short text (no buttons, links, or complex content)
- **Anchored positioning:** Tooltip positions itself relative to the trigger element
- **Delay:** Short delay before appearing prevents flicker on accidental hover

Secondary Features

Accessibility

- **ARIA tooltip pattern:** Trigger has aria-describedby referencing the tooltip's id
- **role="tooltip":** Tooltip container has role="tooltip"
- **Focus trigger mandatory:** Tooltip must appear on focus (not just hover) for keyboard accessibility
- **No keyboard trap:** Users never tab to the tooltip; it is not focusable
- **Do not use for required info:** Tooltips are supplementary only—never the sole source of critical information
- **Screen reader announcement:** Content announced when trigger is focused (via aria-describedby)

Keyboard Navigation

- **Trigger receives focus:** Tab moves focus to the trigger element
- **Tooltip shows on focus:** Tooltip appears when the trigger is focused
- **Escape to dismiss:** Pressing Escape while trigger is focused dismisses the tooltip (leaves trigger focused)
- **No tooltip focus:** Users cannot tab into or interact with the tooltip itself

Touch-screen

- **Long-press to show (optional):** On touch devices, long-press on trigger can show tooltip (then dismiss on next tap elsewhere)
- **No hover on touch:** Tooltips are inherently inaccessible on touch-only devices without a long-press fallback
- **Alternative:** Consider replacing tooltip with visible helper text or Popover for touch-critical information

Responsive Behavior

- **Viewport-aware positioning:** Tooltip repositions (flips to opposite side) if it would overflow the viewport
- **Preferred placement:** Configurable preferred position (top, bottom, left, right) with automatic fallback
- **Max width:** Tooltip text wraps at a configurable maximum width to prevent very long single-line tooltips

Animation & Transitions

- **Entrance delay:** Short delay (100–300ms) before appearance prevents flicker on accidental hover
- **Fade in:** Brief fade-in animation on appear
- **Immediate dismiss:** No delay on disappear—tooltip hides immediately when trigger loses hover/focus
- **Reduced motion:** Respects prefers-reduced-motion; skips animation or uses instant show/hide

Internationalization

- **RTL positioning:** Preferred side adjusts for RTL (left-positioned tooltips flip to right)
- **Multi-byte text:** Tooltip text renders correctly for all character sets
- **Consumer-provided text:** Tooltip label is a string prop; fully localizable by the consumer

Variants & Features

- **Placement:** top, bottom, left, right (with auto-flip fallback)
- **With arrow:** Optional visual arrow pointing to trigger
- **Delay control:** Configurable show delay (shorter for icon labels, longer for hints)
- **Controlled mode:** Tooltip open/close state can be controlled externally (for testing or guided tours)

Test Scenarios

- **Tooltip hidden initially:** Tooltip is not visible before hover or focus
- **Appears on hover:** Tooltip shows after configured delay when mouse enters trigger
- **Disappears on mouse leave:** Tooltip hides immediately when cursor leaves trigger
- **Appears on focus:** Tooltip shows when trigger receives keyboard focus
- **Disappears on blur:** Tooltip hides when trigger loses focus
- **Escape dismisses:** Pressing Escape while trigger is focused hides tooltip
- **role="tooltip":** Tooltip container has correct ARIA role
- **aria-describedby:** Trigger has aria-describedby pointing to tooltip id
- **Viewport flipping:** When tooltip would overflow viewport, it opens on opposite side
- **Max width:** Long tooltip text wraps rather than extending off-screen
- **No focus on tooltip:** Users cannot tab to the tooltip itself
- **Delay:** Tooltip does not appear on momentary hover (below delay threshold)
- **Reduced motion:** When user prefers reduced motion, animation is skipped
- **RTL:** Tooltip placement adjusts correctly for RTL layout
- **Callback:** onShow/onHide callbacks fire when tooltip appears/disappears

Notes

- **Never gate required info behind tooltip:** If information is needed to complete the task, show it inline—not in a tooltip

- **Touch devices:** Tooltips are inaccessible to touch-only users. Always ensure icon buttons have visible labels or use a Popover instead
- **Tooltip vs Popover:** Tooltip = hover/focus, non-interactive, text only.
Popover = click-triggered, interactive content
- **Delay tuning:** Icon button labels benefit from shorter delays (50–100ms).
Informational hints benefit from longer delays (300–500ms)
- **Arrow pointer:** An arrow visually connecting tooltip to trigger reduces confusion about what it describes
- **Positioning library:** Viewport-aware floating positioning is complex.
Libraries like Floating UI handle this reliably
- **Do not nest interactive content:** Buttons, links, or form inputs inside a tooltip violate ARIA tooltip semantics and should use Popover instead

#44 Popover

A click-triggered interactive overlay anchored to a trigger element. Can contain rich content including buttons, forms, or other components.

***Naming note:** Distinct from Tooltip (see 43) and Modal (Popover is anchored to a trigger and doesn't block the rest of the page).*

Description

Popover is a floating overlay anchored to a trigger element that opens on explicit user interaction (click or keyboard activation). Unlike a Tooltip, a Popover can contain any interactive content—buttons, form inputs, links, or complex layouts. Unlike a Modal, it does not block the rest of the page and does not require a full-page backdrop. Popovers are ideal for date pickers, filter panels, color pickers, user profile previews, and any other rich supplementary content attached to a specific trigger.

Main Features

- **Click-triggered:** Opens when the trigger element is clicked (or activated via keyboard)
- **Anchored to trigger:** Positioned relative to the trigger element, not centered on screen
- **Interactive content:** Can contain any UI including buttons, inputs, lists, and media
- **Dismiss on outside click:** Clicking outside the popover closes it
- **Focus management:** Focus moves into popover when opened; returns to trigger when closed
- **No page-blocking backdrop:** Underlying page remains interactive (though clicking outside closes the popover)

Main Features

- **Configurable trigger:** Trigger can be any focusable element (button, icon, text link)
- **Rich content area:** Content slot accepts arbitrary components
- **Controlled/uncontrolled modes:** Can manage its own open state or be controlled externally
- **Header/footer sections:** Optional header with title and close button; optional footer with action buttons

Secondary Features

Accessibility

- **ARIA dialog or region:** Uses role="dialog" when containing interactive content, or role="region" for simpler content
- **aria-haspopup:** Trigger has aria-haspopup="dialog" (or "true")
- **aria-expanded:** Trigger has aria-expanded reflecting open/closed state
- **Focus trap (optional):** For complex interactive popovers (forms), focus can be trapped within
- **Escape to close:** Escape key always closes the popover

- **Focus return:** When closed, focus returns to the trigger element
- **Title accessibility:** If present, title linked to popover via `aria-labelledby`

Keyboard Navigation

- **Trigger activation:** Enter or Space activates the trigger and opens the popover
- **Tab through content:** Tab navigates through interactive elements within the popover
- **Escape to close:** Escape closes the popover from anywhere within it
- **Focus return:** Closing returns focus to the original trigger
- **Tab out to close (optional):** Tabbing past the last item in a non-trapping popover closes it and moves focus forward

Touch-screen

- **Tap to open:** Tapping trigger opens popover
- **Tap outside to close:** Tapping anywhere outside the popover closes it
- **Scroll within popover:** Content inside popover scrolls if it exceeds maximum height
- **Touch-friendly controls:** All interactive elements within meet 44×44px minimum

Responsive Behavior

- **Viewport-aware positioning:** Popover flips sides if it would overflow the viewport
- **Preferred placement:** Configurable preferred position (top, bottom, left, right) with automatic fallback
- **Maximum dimensions:** Configurable max-width and max-height; content scrolls if exceeded
- **Full-screen on mobile:** On very small screens, popover can expand to full-width or bottom-sheet style

Animation & Transitions

- **Entrance animation:** Subtle fade or scale-in on open
- **Exit animation:** Fade or scale-out on close
- **Reduced motion:** Respects prefers-reduced-motion; uses instant show/hide
- **Origin-anchored:** Animation origin point aligns with the trigger element

Internationalization

- **RTL positioning:** Preferred opening side adjusts for RTL layouts
- **Text direction:** Content inside popover inherits correct text direction
- **Consumer-provided content:** All visible text is provided by the consumer and is fully localizable

Variants & Features

- **Simple info popover:** Non-interactive content (closer to an enhanced tooltip with rich text)
- **Form popover:** Contains form inputs (filter panel, date picker, settings)
- **Menu popover:** Contains a list of actions (this pattern is similar to Dropdown Menu)
- **Profile/card popover:** Shows a user or entity card preview
- **Confirmation popover:** Inline “are you sure?” confirmation (lighter than a Modal confirmation dialog)
- **Close button:** Optional explicit close (×) button in the header

Test Scenarios

- **Closed initially:** Popover is not visible before trigger interaction
- **Opens on click:** Clicking trigger opens the popover
- **Opens on keyboard:** Enter/Space on focused trigger opens popover
- **Content renders:** Popover content displays correctly when open
- **Close on outside click:** Clicking outside the popover closes it
- **Escape closes:** Pressing Escape while popover is open closes it

- **Focus moves in:** When opened, focus moves to the first focusable element inside the popover
- **Focus returns:** When closed, focus returns to the trigger element
- **Tab navigation in popover:** Tab cycles through interactive elements inside the popover
- **Aria attributes:** aria-expanded on trigger is true when open, false when closed
- **Viewport positioning:** Popover flips to opposite side when insufficient space on preferred side
- **Scroll in content:** Tall popover content scrolls internally without page scrolling
- **Close button:** When present, clicking close button closes popover
- **Callback execution:** onOpen/onClose callbacks fire correctly
- **Controlled mode:** When open state is controlled externally, component reflects and uses that state
- **RTL layout:** Popover positioning and content direction correct in RTL

Notes

- **Popover vs Tooltip:** Tooltip is hover/non-interactive/text-only. Popover is click-triggered/interactive/rich content
- **Popover vs Modal:** Modal blocks the page and requires explicit dismissal. Popover is anchored, non-blocking, and dismisses on outside click
- **Focus trap decision:** Trap focus for complex forms inside popovers. For simple popovers, let Tab flow naturally
- **Scroll and portals:** Render popovers in a portal (outside normal DOM flow) to avoid z-index and overflow clipping issues
- **Positioning library:** Use a library like Floating UI for robust viewport-aware positioning
- **Mobile full-screen pattern:** For popovers with significant content, consider rendering as a bottom sheet on mobile (see Drawer component)
- **Nested popovers:** Avoid deeply nested popovers; they are hard to manage and confusing for users

#45 Toast / Notification

A temporary, non-blocking message that appears briefly and auto-dismisses. Also called *Snackbar* in Material Design.

Naming note: *Distinct from Alert/Banner (persistent) and Modal (blocking). Toast is ephemeral and non-blocking.*

Description

Toast provides brief, non-blocking feedback about an operation—confirming an action completed, warning of a minor issue, or reporting an error that doesn't prevent the user from continuing. Toasts appear overlaid on the content (typically at the bottom or top of the screen), auto-dismiss after a short time, and do not require user acknowledgment to proceed. Because they are transient, they are only suitable for supplementary information; anything critical must be communicated through persistent UI elements.

Main Features

- **Auto-dismissal:** Toast disappears automatically after a configurable duration (typically 3–6 seconds)
- **Non-blocking:** Underlying page and interface remain fully interactive while toast is visible
- **Message content:** Brief text message describing the event or outcome
- **Semantic variants:** Success, error, warning, and info variants with corresponding visual styling
- **Stacking:** Multiple toasts can queue or stack when triggered in rapid succession
- **Manual dismiss:** User can dismiss toast early via a close (×) button

Secondary Features

Accessibility

- **Live region:** Toast container uses `aria-live="polite"` (or `"assertive"` for errors/critical alerts) so screen readers announce new messages
- **role="status" or role="alert":** status for informational toasts; alert for error toasts requiring immediate attention
- **Not keyboard-focusable (typically):** Toast close button should be reachable if present; toast itself is not a focus destination
- **Do not auto-dismiss errors:** Error toasts that communicate failure should persist longer or not auto-dismiss, as users may need to read and act on them
- **Contrast:** Toast messages must meet WCAG contrast ratios, especially when overlaid on images or dynamic backgrounds

Keyboard Navigation

- **Close button focus:** If a close button is present, it must be keyboard-accessible (Tab, then Enter/Space)
- **Action button focus:** If toast contains an action link or button (e.g., "Undo"), it must be keyboard-accessible
- **No forced focus:** Focus is never moved to the toast on appearance; it appears passively
- **Escape to dismiss (optional):** Pressing Escape can dismiss the currently visible toast

Touch-screen

- **Touch to dismiss:** Swiping toast (left/right or down) dismisses it
- **Touch-friendly close:** Close button meets 44×44px tap target
- **Readable duration:** Auto-dismiss duration accounts for slower reading on mobile
- **Non-interfering position:** Toast positioned to avoid bottom navigation bars or fixed CTAs

Responsive Behavior

- **Fixed positioning:** Toast renders in a fixed overlay position regardless of scroll
- **Mobile positioning:** Full-width or near-full-width at bottom on mobile; compact at bottom-right on desktop
- **Stacking direction:** Multiple toasts stack vertically; newer toasts appear below (or above, configurable) existing ones
- **Safe areas:** Respects device safe areas (notch, bottom home indicator)

Animation & Transitions

- **Slide in:** Toast slides in from the edge of the screen on appear
- **Fade out:** Toast fades or slides out on dismiss
- **Stagger:** When multiple toasts appear, they animate with slight delay for visual clarity
- **Reduced motion:** Respects prefers-reduced-motion; uses simpler or instant show/hide
- **Progress indicator:** Optional visual timer bar showing remaining display time

Internationalization

- **RTL positioning:** Slide direction and position adjust for right-to-left layouts
- **Text direction:** Toast message inherits correct text direction
- **Consumer-provided messages:** All toast text is provided by the consumer; fully localizable
- **Long messages:** Toast layout handles longer translated strings without breaking layout

Variants & Features

- **Success toast:** Confirms successful action (saved, created, deleted)
- **Error toast:** Reports failure (form submission failed, request error)

- **Warning toast:** Non-critical caution (approaching limit, slower than expected)
- **Info toast:** Neutral information (new feature available, background sync started)
- **With action:** Toast includes an action button (e.g., “Undo”, “View”, “Retry”)
- **Persistent (no auto-dismiss):** Toast remains until explicitly dismissed (for errors or important warnings)
- **Queue vs replace:** When multiple toasts are triggered rapidly, either queue them or replace with latest

Test Scenarios

- **Toast appears:** Triggering a toast causes it to display
- **Auto-dismiss:** Toast disappears after the configured duration without user interaction
- **Close button:** Clicking close button dismisses toast immediately
- **Success variant:** Success toast renders with correct icon and color styling
- **Error variant:** Error toast renders with correct icon and color styling
- **Warning variant:** Warning toast renders with warning icon and styling
- **Info variant:** Info toast renders with info icon and styling
- **Live region:** Screen reader announces toast message when it appears
- **role attribute:** role=“status” for info/success; role=“alert” for errors
- **Action button:** When action is provided, button renders and fires callback on click
- **Stacking:** When multiple toasts are triggered, they stack vertically
- **Queue behavior:** When configured to queue, toasts appear sequentially
- **Swipe to dismiss:** Swiping toast dismisses it on touch devices
- **Progress bar:** When configured, visual timer bar depletes over the auto-dismiss duration
- **Reduced motion:** Animation is skipped when user prefers reduced motion
- **RTL layout:** Toast slide direction and text direction correct in RTL
- **Persistent toast:** When auto-dismiss is disabled, toast remains until manually dismissed

- **Callback execution:** onDismiss fires when toast is dismissed (by timer or user action)
- **Non-blocking:** Underlying page remains fully interactive while toast is visible

Notes

- **Toast vs Alert/Banner:** Toasts are temporary and non-blocking. Alerts/Banners are persistent and inline. Do not use toasts for critical or persistent information
- **Error communication:** For errors that affect the user's current task, prefer inline error messages or persistent alerts over toasts. A toast error can be missed
- **Auto-dismiss duration:** 3–4 seconds for short messages; 5–6 seconds for longer ones. Error toasts should be 6+ seconds or persistent
- **Undo pattern:** Success toasts for destructive actions (delete, archive) benefit from an “Undo” action button
- **Queue vs. replace:** Queueing is safer but can delay feedback. Replacing with latest is faster but loses intermediate messages. Choose based on use case
- **ARIA live assertive:** Use assertive only for genuine errors requiring immediate attention; polite for all other toasts. assertive interrupts screen reader speech
- **Toast manager/service:** Toasts are typically managed by a centralized service or context (not rendered inline) so they can be triggered from anywhere in the app
- **Avoid stale messages:** Clear toast queue on route changes or significant app state transitions to avoid confusing stale messages

#46 Modal / Dialog

A blocking overlay window that requires user interaction before returning to the main interface.

Specialization: Prompt / Confirmation Dialog (simplified modal for yes/no or destructive action confirmation)

Naming note: Also called *Dialog* in WAI-ARIA and many libraries. *Prompt* is a common specialization for confirmation flows.

Description

A modal (or dialog) is an overlay window that appears on top of the main content, requiring user interaction before returning to the main interface. Modals focus user attention on a specific task, decision, or information while temporarily blocking interaction with the underlying page.

Main Features

- **Overlay/backdrop:** Semi-transparent layer covers underlying content
- **Content container:** Visible panel/card containing the modal content
- **Focus trapping:** Focus remains within modal while open; cannot tab to underlying page
- **Close mechanism:** User can close modal via close button, cancel action, or completion of task
- **Block underlying interaction:** Clicks on backdrop or underlying page do not interact with page content
- **Visual hierarchy:** Modal appears above all other content with clear visual separation
- **Title/header:** Clear identification of modal purpose or current task

Secondary Features

Accessibility

- **Focus management:** Focus moves to modal when opened; returns to trigger element when closed
- **ARIA attributes:** Proper role (dialog or alertdialog), aria-labelledby, aria-describedby

- **Keyboard trap:** Tab/Shift+Tab cycle through focusable elements within modal only
- **Screen reader announcements:** Modal opening is announced to screen readers
- **Focus on first element:** When opened, focus moves to first focusable element (or close button)

Keyboard Navigation

- **Escape to close:** Escape key closes modal (unless it's a critical confirmation)
- **Tab navigation:** Tab cycles through focusable elements within modal; Shift+Tab cycles backwards
- **Enter for primary action:** Enter key triggers primary action when appropriate
- **Focus cycling:** When reaching last element, Tab moves to first element (and vice versa)

Touch-screen

- **Touch-friendly controls:** Close button and action buttons are adequately sized
- **Scroll support:** Modal content scrolls if it exceeds viewport height
- **Backdrop interaction:** Tapping backdrop closes modal (configurable behavior)
- **Prevent body scroll:** Underlying page does not scroll while modal is open

Responsive Behavior

- **Flexible sizing:** Modal size adapts to viewport (full-screen on small screens, centered on large screens)
- **Content scrolling:** Long content scrolls within modal without scrolling page
- **Safe area handling:** Respects device safe areas (notches, bottom bars on mobile)
- **Maximum width:** On large screens, modal doesn't exceed readable width (typically 600-800px)

Internationalization

- **Bi-directional text support:** Modal content and controls handle RTL languages
- **Localizable buttons:** Action buttons (OK, Cancel, Close) can be localized
- **Multi-byte character support:** Properly displays all character sets

Animation & Transitions

- **Entrance animation:** Modal fades/scales in when opened (respects prefers-reduced-motion)
- **Exit animation:** Modal fades/scales out when closed
- **Backdrop animation:** Backdrop fades in/out with modal
- **Smooth state changes:** State transitions are visually smooth, not jarring

Modal Variants & Features

- **Size variants:** Small, medium, large, full-screen options
- **Dismissible control:** Configure whether clicking backdrop or pressing Escape closes modal
- **Action buttons:** Primary and secondary action buttons (OK/Cancel, Save/Discard, etc.)
- **Close button:** Optional close button (X) in header
- **Footer actions:** Action buttons typically in footer for consistency
- **Confirmation modals:** Specialized variant for destructive actions (different styling, clearer buttons)

Test Scenarios

- **Modal opens:** When triggered, modal and backdrop appear
- **Focus moves to modal:** When opened, focus automatically moves into modal
- **Backdrop blocks interaction:** Clicking underlying page content has no effect
- **Backdrop click closes:** Clicking backdrop closes modal (if dismissible)

- **Escape closes:** Pressing Escape closes modal (if dismissible)
- **Focus trap active:** Tab navigation cycles only within modal; cannot tab to underlying page
- **Close button works:** Clicking close button closes modal
- **Action buttons work:** Clicking primary/secondary action buttons triggers appropriate callbacks
- **Focus returns:** When closed, focus returns to element that triggered modal
- **Body scroll prevented:** Underlying page does not scroll while modal is open
- **Responsive sizing:** Modal adapts size appropriately on different screen sizes
- **Animation plays:** Modal animates in when opened, out when closed (unless prefers-reduced-motion)
- **Multiple modals:** If second modal opens over first, focus trap applies to top modal
- **Screen reader announcement:** Modal opening is announced to assistive technologies
- **Callback execution:** onClose fires when modal closes, onOpen fires when it opens

Notes

- **Critical confirmations:** For destructive actions, consider making modal non-dismissible (no backdrop close, no Escape) and require explicit button click
- **Nested modals:** Generally avoid, but if needed, ensure focus management works correctly
- **Scroll locking:** Prevent body scroll requires platform-specific techniques; be aware of scroll position jumping
- **Portals/layers:** Modals often need to render outside normal DOM hierarchy to avoid z-index issues
- **Drawer variant:** Side drawer/panel is a related pattern with different behavior (slides from edge, may allow backdrop interaction)
- **Toast/Snackbar:** Non-blocking, auto-dismissing notifications are different from modals; don't confuse the two

- **Loading states:** Modals can contain loading states; consider disabling close during critical operations

#47 Drawer

A panel that slides in from the edge of the screen, containing navigation or supplementary content.

Specializations: Side Drawer (slides from left or right), Bottom Drawer (slides from bottom, common on mobile)

Naming note: Also called *Sheet*, *Side Panel*, or *Off-canvas* in other libraries. *Bottom Drawer* is often called *Bottom Sheet* in mobile design systems.

Description

Drawer is a panel that slides in from one edge of the screen (left, right, or bottom) and overlays the main content. It is used to surface navigation menus, settings panels, filter controls, and supplementary content without navigating away from the current page. Unlike Modal, Drawer is anchored to an edge and typically implies a spatial relationship—content slides in from where it logically “lives.” A semi-transparent backdrop is usually shown over the main content while the Drawer is open.

Main Features

- **Slide-in panel:** Panel animates in from a screen edge (left, right, or bottom)
- **Backdrop overlay:** Semi-transparent backdrop covers the main content behind the Drawer
- **Focus trapping:** Focus is contained within the Drawer while it is open
- **Dismiss on backdrop click:** Clicking or tapping the backdrop closes the Drawer
- **Escape to close:** Pressing Escape dismisses the Drawer

- **Scrollable content:** Drawer content scrolls independently if it exceeds the viewport height

Secondary Features

Accessibility

- **ARIA dialog:** Drawer uses role=“dialog” with aria-modal=“true”
- **Label:** aria-labelledby references a visible title within the Drawer
- **Focus management:** Focus moves to the first focusable element inside the Drawer when opened; returns to the trigger element when closed
- **Focus trap:** Tab and Shift+Tab cycle only through focusable elements within the Drawer
- **Screen reader announcement:** Opening is announced to assistive technologies
- **Body scroll lock:** Underlying page scroll is locked while Drawer is open

Keyboard Navigation

- **Escape to close:** Escape key closes the Drawer from anywhere within it
- **Tab cycle:** Tab moves forward through focusable elements; Shift+Tab moves backward
- **Focus cycling:** Tab past the last element wraps to the first; Shift+Tab before the first wraps to the last
- **Close button:** Explicit close button (×) is keyboard-accessible

Touch-screen

- **Swipe to dismiss:** Swiping in the opposite direction of the Drawer (e.g., swipe left on a right Drawer) dismisses it
- **Drag handle:** Bottom Drawers often include a visible drag handle for intuitive swipe interaction
- **Touch backdrop dismiss:** Tapping the backdrop closes the Drawer
- **Scroll within Drawer:** Content inside the Drawer scrolls without dismissing it

Responsive Behavior

- **Full height for side Drawers:** Side Drawers span the full viewport height
- **Configurable width:** Side Drawer width is configurable (typically 240–400px or a percentage)
- **Full width for Bottom Drawers:** Bottom Drawer spans the full viewport width
- **Configurable height:** Bottom Drawer height is configurable or determined by content
- **Body scroll lock:** Prevents the underlying page from scrolling behind the open Drawer

Animation & Transitions

- **Slide in:** Panel slides in from its anchor edge on open
- **Slide out:** Panel slides back out on close
- **Backdrop fade:** Backdrop fades in and out with the Drawer
- **Reduced motion:** Respects prefers-reduced-motion; uses instant show/hide or minimal transition

Internationalization

- **RTL side flipping:** In RTL layouts, a “left” Drawer typically becomes a “right” Drawer (mirrored)
- **Content text direction:** All content inside the Drawer inherits the correct text direction
- **Localizable controls:** Close button and any action labels are consumer-provided and fully localizable

Variants & Features

- **Persistent Drawer:** Always visible (no backdrop, no dismiss); pushes page content rather than overlaying it—common in desktop sidebars
- **Temporary Drawer:** Overlays content with backdrop; closes on outside interaction—common on mobile

- **Mini/collapsed variant:** Drawer collapses to a narrow icon-only rail; expands on hover or click
- **Controlled open state:** Open/close state can be driven externally (router state, user action)

Test Scenarios

- **Drawer hidden initially:** Drawer is not visible before being triggered
- **Opens on trigger:** Clicking trigger causes Drawer to slide in from the correct edge
- **Backdrop renders:** Semi-transparent backdrop appears behind the open Drawer
- **Backdrop click closes:** Clicking the backdrop closes the Drawer
- **Escape closes:** Pressing Escape closes the Drawer
- **Close button closes:** Clicking the explicit close button dismisses the Drawer
- **Focus moves in:** When opened, focus moves to the first focusable element inside the Drawer
- **Focus trap active:** Tab navigation is contained within the Drawer; cannot reach underlying page
- **Focus returns:** When closed, focus returns to the element that triggered the Drawer
- **Body scroll locked:** Underlying page does not scroll while Drawer is open
- **Content scrolls:** If Drawer content exceeds height, it scrolls internally without closing the Drawer
- **Slide animation:** Drawer animates in/out from correct edge
- **Reduced motion:** Animation is skipped when user prefers reduced motion
- **Swipe to dismiss:** Swiping in the dismiss direction closes the Drawer on touch devices
- **Callback execution:** onOpen/onClose callbacks fire at correct moments
- **RTL layout:** Side Drawer opens from the mirrored edge in RTL mode

Side Drawer Specialization

Additional Features:

- Anchored to the left or right edge of the viewport
- Spans full viewport height
- Width configurable (fixed px or percentage of viewport)
- Commonly used for: primary navigation (left), settings/details panel (right)
- On desktop, may operate in persistent (push) mode rather than overlay mode

Additional Test Scenarios:

- **Left/right placement:** Drawer slides in from the configured edge (left or right)
- **Full height:** Drawer occupies full viewport height
- **Width constraint:** Drawer renders at the configured width
- **Persistent mode:** In persistent mode, main content shifts to accommodate the Drawer rather than being overlaid

Bottom Drawer Specialization

Additional Features:

- Anchored to the bottom edge of the viewport
- Spans full viewport width
- Height determined by content or configurable maximum height
- Drag handle visible for swipe-up/swipe-down interaction
- Commonly used on mobile for: action sheets, filter panels, contextual menus

Additional Test Scenarios:

- **Bottom placement:** Drawer slides in from the bottom edge
- **Full width:** Drawer spans full viewport width
- **Height adapts to content:** Drawer height matches content height up to the configured maximum
- **Drag handle visible:** Drag handle indicator renders at the top of the Bottom Drawer

- **Swipe down to dismiss:** Swiping downward on the Drawer or its drag handle dismisses it

#48 Adaptive Menu

A context-aware menu component that renders differently based on device or context. Typically renders as a Dropdown Menu on desktop and a Drawer (bottom or side) on mobile.

***Naming note:** This is a behavioral pattern rather than a distinct visual component. It combines Dropdown Menu and Drawer under a single interface.*

Description

Adaptive Menu is a behavioral pattern that presents the same set of actions or options using the most contextually appropriate UI component based on the current device or viewport. On desktop, actions are shown in a floating Dropdown Menu anchored to the trigger. On mobile or narrow viewports, the same actions are shown in a Bottom Drawer or Side Drawer—providing larger touch targets and a more native-feeling experience. The consumer defines one set of menu items; the component decides how to render them.

Main Features

- **Single interface, multiple renderings:** One set of items and configuration drives both the desktop and mobile presentations
- **Automatic rendering mode:** Switches between Dropdown and Drawer based on viewport width or device capability
- **Override control:** Consumer can force a specific rendering mode (always Dropdown or always Drawer)
- **Shared item model:** Items, groups, icons, disabled states, and callbacks are defined once and apply to both renderings
- **Trigger consistency:** The same trigger element activates both Dropdown and Drawer modes

Secondary Features

Accessibility

- **Mode-appropriate ARIA:** Applies correct ARIA pattern for the active rendering—Dropdown Menu ARIA when in menu mode, dialog ARIA when in Drawer mode
- **Focus management:** Follows focus management rules of the active rendering (focus trap in Drawer mode; no trap in Dropdown mode)
- **Escape to close:** Escape closes the active presentation in both modes
- **Screen reader announcement:** Opening behavior is announced correctly for each rendering mode

Keyboard Navigation

- **Dropdown mode keyboard:** Follows Dropdown Menu keyboard pattern (arrow keys, Enter, Escape)
- **Drawer mode keyboard:** Follows Drawer keyboard pattern (Tab through items, Escape to close)
- **Trigger activation:** Enter or Space on trigger opens the component in whichever mode is active

Touch-screen

- **Drawer mode on touch:** On touch devices, renders as Bottom Drawer for larger tap targets and swipe-to-dismiss
- **Swipe to dismiss:** In Drawer mode, swipe down dismisses the menu
- **Touch-friendly item sizing:** In Drawer mode, items have adequate height and padding for touch

Responsive Behavior

- **Breakpoint-based switching:** Switches rendering mode at a configurable viewport width breakpoint
- **Recalculates on resize:** If viewport is resized while component is closed, next open uses correct mode

- **No rerender while open:** If viewport is resized while the component is open, it remains in the current mode until closed and reopened

Internationalization

- **RTL consistent across modes:** Both Dropdown and Drawer modes apply correct RTL layout
- **Localizable items:** All item labels are consumer-provided and localizable regardless of rendering mode

Variants & Features

- **Force desktop mode:** Prop to always render as Dropdown regardless of viewport
- **Force mobile mode:** Prop to always render as Bottom/Side Drawer regardless of viewport
- **Custom breakpoint:** Configurable pixel breakpoint at which the switch occurs
- **Side Drawer mode:** Can use Side Drawer instead of Bottom Drawer for the mobile rendering

Test Scenarios

- **Desktop rendering:** On wide viewport, component renders as Dropdown Menu
- **Mobile rendering:** On narrow viewport, component renders as Drawer (Bottom or Side)
- **Same items in both modes:** All menu items, groups, and icons appear identically in both rendering modes
- **Trigger opens both:** Clicking the trigger opens whichever rendering is active
- **Dropdown keyboard pattern:** In Dropdown mode, arrow keys navigate, Escape closes
- **Drawer keyboard pattern:** In Drawer mode, Tab navigates items, Escape closes

- **Focus trap in Drawer mode:** In Drawer mode, focus is trapped within the Drawer
- **No focus trap in Dropdown mode:** In Dropdown mode, focus is not trapped
- **Backdrop in Drawer mode:** In Drawer mode, backdrop renders and clicking it closes the menu
- **Item callback:** onItemClick fires with the same item identifier regardless of rendering mode
- **Force desktop mode:** When forceDesktop prop is set, Dropdown renders even on narrow viewport
- **Force mobile mode:** When forceMobile prop is set, Drawer renders even on wide viewport
- **Breakpoint switch:** Switching viewport width across the breakpoint changes next-open rendering mode
- **RTL in both modes:** RTL layout is correct in both Dropdown and Drawer rendering modes

Notes

- **Pattern, not primitive:** Adaptive Menu is a wrapper pattern over Dropdown Menu and Drawer. Implement those components first, then compose them here
- **Breakpoint decision:** 768px (tablet/mobile boundary) is a common default; adjust to match your design system breakpoints
- **Bottom vs Side Drawer on mobile:** Bottom Drawer is more common for action menus on mobile; Side Drawer suits navigation menus
- **Testing both modes:** Test each rendering mode independently and test the switching behavior across viewport sizes
- **When to use:** Use when the same actions need to work well on both desktop and mobile without duplicating logic in parent components

#49 Date Picker

A calendar-based input for selecting a single date.

Specialization: Date Range Picker (select a start and end date)

Description

Date Picker allows users to select a single calendar date through a visual calendar interface. It combines a text input (showing the selected date in a human-readable format) with a calendar popover for interactive date selection. Users can type a date directly into the input or open the calendar to pick one visually. Date Picker should handle locale-specific date formatting, keyboard navigation through the calendar grid, and validation of entered dates.

Main Features

- **Text input display:** Shows the selected date in a formatted, human-readable string
- **Calendar popover:** Visual month calendar opens on input focus or click
- **Day selection:** Clicking a day in the calendar selects that date and closes the popover
- **Month/year navigation:** Previous and next controls for navigating between months; year can be selected directly
- **Today highlight:** Current calendar date visually distinguished
- **Selected date highlight:** The chosen date is visually marked in the calendar
- **Min/max date constraints:** Dates outside the allowed range are disabled and unselectable

Secondary Features

Accessibility

- **ARIA grid pattern:** Calendar uses role="grid" with role="gridcell" for day cells
- **Selected date:** aria-selected="true" on the selected day cell
- **Disabled dates:** aria-disabled="true" on out-of-range day cells
- **Live region:** Month/year changes announced to screen readers
- **Label association:** Text input linked to a visible label
- **Dialog:** Calendar popover uses role="dialog" with aria-label or aria-labelledby

Keyboard Navigation

- **Tab to input:** Input receives focus via Tab; opening the calendar traps focus within it
- **Arrow keys in calendar:** Left/Right navigate days; Up/Down navigate weeks
- **Page Up/Down:** Move to previous/next month
- **Home/End:** Jump to first/last day of the current week or month (configurable)
- **Enter/Space to select:** Confirms focused day as the selection and closes calendar
- **Escape to close:** Closes calendar without changing selection; returns focus to input
- **Type in input:** User can type a date directly; calendar updates to reflect typed value

Touch-screen

- **Touch-friendly day cells:** Day cells meet minimum tap target size (44×44px preferred)
- **Swipe month navigation:** Swiping left/right in the calendar navigates to the next/previous month
- **Tap to select:** Tapping a day selects it and closes the calendar

- **Input keyboard:** Tapping the text input opens the device keyboard; calendar may also open

Responsive Behavior

- **Popover positioning:** Calendar opens below or above the input depending on available viewport space
- **Full-width on mobile:** On small screens, calendar may expand to full width or full screen
- **Month grid sizing:** Day cells maintain legible size across screen widths
- **Single or double month:** On wide viewports, optionally show two months side by side (useful for Range Picker)

Internationalization

- **Locale-aware formatting:** Date displayed in the text input uses locale-appropriate format (MM/DD/YYYY, DD/MM/YYYY, YYYY-MM-DD, etc.)
- **Locale-aware week start:** First day of week (Sunday vs Monday) follows locale convention
- **Localized month/day names:** Month names and day-of-week abbreviations are localized
- **RTL calendar grid:** Calendar layout mirrors for right-to-left languages

Validation & Feedback

- **Invalid date handling:** If user types an unparseable date, input shows an error state
- **Out-of-range feedback:** Dates outside min/max range are visually disabled; typing an out-of-range date shows an error
- **Error message:** Descriptive error message displayed below input
- **Callbacks:** onChange fires with the new date value on selection or valid typed input; onOpen/onClose fire on calendar toggle

Test Scenarios

- **Input renders:** Text input displays with label and optional placeholder
- **Calendar opens:** Clicking input or calendar icon opens the calendar popover
- **Calendar closes on selection:** Selecting a day closes the calendar and updates the input
- **Today highlighted:** Current calendar date is visually distinguished
- **Selected date highlighted:** The chosen date is visually marked after selection
- **Month navigation:** Clicking previous/next month controls changes the displayed month
- **Year navigation:** Year can be changed via direct selection or navigation controls
- **Min date disabled:** Days before the minimum date are disabled and cannot be selected
- **Max date disabled:** Days after the maximum date are disabled and cannot be selected
- **Arrow key navigation:** Left/Right/Up/Down keys move focus through day cells
- **Page Up/Down:** Page keys navigate months
- **Enter selects:** Enter on a focused day cell selects that date
- **Escape closes:** Escape closes calendar without changing selection
- **Typed input:** Typing a valid date in the input selects that date and updates the calendar view
- **Invalid typed date:** Typing an invalid string shows an error state
- **Locale formatting:** Date displays in locale-appropriate format
- **Locale week start:** Calendar first day of week reflects locale setting
- **Focus trap in calendar:** Tab is contained within calendar while it is open
- **Focus return:** Closing calendar returns focus to the input
- **Callback execution:** onChange fires with correct value; onOpen/onClose fire correctly
- **RTL calendar:** Calendar grid mirrors correctly in RTL

Date Range Picker Specialization

Additional Features:

- **Two-date selection:** user selects a start date and then an end date
- **In-progress range highlighted:** days between the hovered/tentative end date and the start date are visually highlighted during selection
- **Confirmed range highlighted:** all days within the selected range are visually marked
- **Start and end inputs:** two separate text inputs (or a combined “start → end” input) display the selected range
- **Optionally shows two months side by side** for easier range selection
- **Min/max range length constraints** (e.g., minimum 2 nights, maximum 30 days)

Additional Test Scenarios:

- **Two-step selection:** Clicking a start date enters range-selection mode; clicking a second date confirms the range
- **Hover preview:** Hovering over days after selecting start date highlights the tentative range
- **Range highlight:** All days between start and end dates are visually highlighted after selection
- **Start/end inputs update:** Both start and end inputs update correctly when the range is confirmed
- **Reversed range handling:** If user selects end before start, component swaps or re-prompts correctly
- **Min range constraint:** Dates too close to the start to satisfy minimum range length are disabled
- **Max range constraint:** Dates too far from the start to satisfy maximum range length are disabled
- **Clear range:** Selecting a new start date after a range is confirmed resets and starts a new selection

#50 Time Picker

An input for selecting a specific time.

Specialization: Time Range Picker (select a start and end time)

Description

Time Picker allows users to select a specific time of day. It combines a text input (showing the selected time in a formatted string) with an interactive time selection panel—which may be a clock face, scrollable columns of hours and minutes, or simple increment/decrement controls. Time Picker should respect 12-hour vs 24-hour format based on locale or configuration, and validate that entered times are within any configured bounds.

Main Features

- **Text input display:** Shows selected time in a formatted string (e.g., “2:30 PM” or “14:30”)
- **Time selection panel:** Interactive panel for picking hours, minutes (and optionally seconds and AM/PM)
- **12/24-hour format:** Configurable or locale-driven display format
- **Minute step increments:** Configurable minute intervals (e.g., every 15 minutes)
- **Min/max time constraints:** Times outside the allowed range are disabled
- **Direct text entry:** User can type a time directly into the input

Secondary Features

Accessibility

- **Label association:** Input linked to a visible label
- **ARIA for spinners:** Hour/minute spinner controls use role=“spinbutton” with aria-valuenow, aria-valuemin, aria-valuemax

- **Live updates:** Changes to hour/minute values announced to screen readers
- **Dialog:** Time panel uses role=“dialog” with accessible label
- **Error state:** aria-invalid and aria-describedby for validation feedback

Keyboard Navigation

- **Tab to input:** Input receives focus via Tab
- **Arrow keys in spinners:** Up/Down arrows increment/decrement hour or minute value
- **AM/PM toggle:** A or P keys (or arrow keys) switch between AM and PM in 12-hour mode
- **Enter to confirm:** Confirms the selected time and closes the panel
- **Escape to close:** Closes panel without changing the time; returns focus to input
- **Type in input:** User can type a time string directly; panel updates to reflect

Touch-screen

- **Scrollable columns:** Hours and minutes presented as scrollable drum-roll columns on mobile
- **Touch-friendly targets:** Increment/decrement buttons and column items meet 44×44px minimum
- **Tap AM/PM:** AM/PM toggle is a large tap target
- **Swipe columns:** Swiping up/down on columns scrolls through values

Responsive Behavior

- **Popover positioning:** Panel opens below or above input depending on viewport space
- **Column vs clock face:** May use scrollable columns on mobile, clock face or spinners on desktop
- **Full-width on mobile:** Panel may expand to full width on narrow viewports

Internationalization

- **Locale-driven format:** 12 vs 24-hour format defaults based on locale

- **Locale-appropriate separators:** Time separator (colon, period) follows locale
- **RTL spinner direction:** Spinner layout adapts for right-to-left languages
- **Localized AM/PM labels:** AM/PM strings are localizable

Validation & Feedback

- **Invalid time handling:** Unparseable typed input triggers error state with message
- **Out-of-range feedback:** Times outside min/max are disabled in the panel and flagged if typed
- **Callbacks:** onChange fires with new time value; onOpen/onClose fire on panel toggle

Test Scenarios

- **Input renders:** Text input displays with label and optional placeholder
- **Panel opens:** Clicking input or clock icon opens the time selection panel
- **Hour selection:** Changing the hour in the panel updates the displayed time
- **Minute selection:** Changing the minute updates the displayed time
- **AM/PM toggle:** In 12-hour mode, toggling AM/PM updates the time value correctly
- **Minute steps:** Only valid minute increments (per configured step) are selectable
- **Min time disabled:** Times before the minimum are disabled in the panel
- **Max time disabled:** Times after the maximum are disabled in the panel
- **Arrow key increment:** Up/Down arrows increment/decrement the focused hour or minute spinner
- **Enter confirms:** Pressing Enter closes the panel and confirms the selected time
- **Escape closes:** Pressing Escape closes panel without changing value; focus returns to input
- **Typed input:** Typing a valid time string updates the selection
- **Invalid typed input:** Typing an invalid string shows an error state

- **24-hour format:** When configured, time displays and inputs in 24-hour format
- **12-hour format:** When configured, AM/PM selector is present
- **Locale format:** Time format reflects locale configuration
- **Callback execution:** onChange fires with correct value on change

Time Range Picker Specialization

Additional Features:

- Two-time selection: user selects a start time and an end time
- Two separate inputs (or a combined “start → end” display) for start and end times
- End time is automatically constrained to be after the start time
- Optional minimum and maximum duration constraints between start and end
- Linked panels: changing start time may update end time minimum accordingly

Additional Test Scenarios:

- **Two inputs:** Start time and end time inputs both render
- **End after start:** End time panel disables times earlier than or equal to the selected start time
- **Start change updates end constraint:** After changing start time, end time minimum updates accordingly
- **Min duration:** End times that would result in a range shorter than minimum duration are disabled
- **Max duration:** End times that would result in a range longer than maximum duration are disabled
- **Both values in callback:** onChange fires with both start and end time values

#51 DateTime Picker

Combined input for selecting both a date and a time in a single component.
Builds on Date Picker and Time Picker.

Description

DateTime Picker combines date and time selection into a unified component. The user selects a date (via calendar) and a time (via time panel), and the component produces a single combined datetime value. It avoids requiring the consumer to coordinate a separate Date Picker and Time Picker, and ensures the combined value is always internally consistent—for example, constraining valid times based on the selected date when min/max datetime bounds are provided.

Main Features

- **Unified input:** Single trigger and display field shows the combined date and time
- **Sequential selection:** User selects date first (calendar), then time (time panel), in a guided flow
- **Combined value output:** Component emits a single datetime value (not separate date and time values)
- **Min/max datetime constraints:** Constrains both date and time based on combined min/max boundaries
- **Same-day time restriction:** When the selected date is the min or max boundary day, available times are further constrained

Secondary Features

Accessibility

- **Composite labeling:** The unified input has a clear label identifying it as a date and time field

- **Panel dialogs:** Both the calendar and time panels use appropriate dialog ARIA
- **Step announcements:** Screen readers are informed when transitioning from date selection to time selection
- **Error state:** Combined validation error state with descriptive message

Keyboard Navigation

- **Calendar keyboard:** Inherits Date Picker calendar keyboard pattern (arrow keys, Page Up/Down, Enter, Escape)
- **Time keyboard:** Inherits Time Picker panel keyboard pattern (arrow keys for spinners, Enter, Escape)
- **Transition between steps:** After selecting a date, focus moves automatically to the time panel; Tab also moves between panels
- **Escape:** Closes the active panel; pressing Escape again closes the entire picker

Touch-screen

- **Step-by-step panels:** On mobile, date and time selection presented as distinct sequential steps
- **Swipe between steps:** Optionally swipe left/right to move between date and time steps
- **Large tap targets:** All interactive elements in both panels meet 44×44px minimum

Responsive Behavior

- **Combined popover:** Both calendar and time panel appear within a single popover container
- **Sequential or side-by-side:** On wide viewports, date and time panels may appear side by side; on narrow viewports, sequentially
- **Full-screen on mobile:** Picker may expand to a full-screen modal on small devices

Internationalization

- **Locale formatting:** Combined datetime display uses locale-appropriate date and time format
- **Locale week start and 12/24-hour:** Inherits locale rules from both Date Picker and Time Picker

Validation & Feedback

- **Cross-field validation:** Validates the combined datetime value (not date and time independently)
- **Same-day constraints:** When selected date equals the min or max boundary date, available times are restricted accordingly
- **Error message:** Single error message describes the combined validation failure
- **Callbacks:** onChange fires with combined datetime value; step-level callbacks (onDateChange, onTimeChange) available for fine-grained control

Test Scenarios

- **Input renders:** Single input displays with label and combined date-time placeholder
- **Calendar opens first:** Opening the picker shows the date calendar
- **Date selection advances to time:** Selecting a date in the calendar automatically moves to the time selection panel
- **Time selection completes:** Selecting a time closes the picker and emits the combined datetime value
- **Combined value correct:** Emitted value correctly combines the selected date and time
- **Min datetime constraint:** Dates before the minimum boundary are disabled; times on the minimum boundary date are restricted appropriately
- **Max datetime constraint:** Dates after the maximum boundary are disabled; times on the maximum boundary date are restricted appropriately
- **Same-day time restriction:** When selected date is the min boundary date, times before the min time are disabled

- **Manual typed input:** User can type a full datetime string; both date and time components update accordingly
- **Invalid input:** Typing an invalid string shows a combined error state
- **Escape closes step then picker:** First Escape closes the active panel; second Escape closes the entire picker
- **Focus transition:** After date selection, focus moves to the time panel
- **Locale format:** Combined display uses locale-appropriate datetime format
- **Callback fires:** onChange fires with correct combined datetime value

Notes

- **Build on Date Picker and Time Picker:** DateTime Picker should compose the Date Picker and Time Picker components rather than re-implementing their internals
- **Timezone awareness:** If your application handles timezones, DateTime Picker must know the intended timezone for min/max evaluation; document this explicitly
- **ISO 8601 output:** Emit values in a standard format (ISO 8601 string or native Date object) for easy integration with date libraries
- **Same-day edge case:** When min and max are on the same day, the calendar should show only that day as selectable and the time panel should show only the valid time window
- **Step UX:** Making the date-then-time flow clear (progress indicator, panel headings) reduces user confusion in multi-step selection

#52 Calendar

A full calendar view for displaying and navigating dates and events. Distinct from Date Picker in that it is a display/navigation component, not purely an input.

Description

Calendar is a full-page or full-panel view for displaying, navigating, and interacting with dates and events. Unlike Date Picker (which is an input widget for capturing a date value), Calendar is a display component—its primary purpose is to show events, appointments, or highlights across a navigable time span. Users can typically click a date to view or create events, navigate between months/weeks/days, and see event density. Calendar is common in scheduling applications, booking systems, and dashboards.

Main Features

- **Month/week/day views:** Multiple time-granularity views switchable by the user
- **Date cell rendering:** Each date cell is a distinct interactive area displaying events or indicators for that date
- **Navigation controls:** Previous/next period controls and a “today” button for jumping to the current date
- **Event display:** Events or highlights rendered within their corresponding date cells
- **Date selection:** Clicking a date fires a callback (for creating events, viewing details, etc.)
- **Current date indicator:** Today’s date is visually distinguished

Secondary Features

Accessibility

- **ARIA grid:** Calendar grid uses role=“grid” with role=“row” and role=“gridcell” for date cells
- **Selected date:** aria-selected=“true” on the currently selected date cell
- **Today indicator:** aria-current=“date” on today’s cell
- **Event announcements:** Events within a date cell include descriptive text accessible to screen readers

- **Navigation controls:** Previous/next buttons have descriptive aria-labels (e.g., “Previous month”, “Next month”)
- **Live region:** Month/year heading updates announced to screen readers when navigation occurs

Keyboard Navigation

- **Arrow keys:** Left/Right navigate days; Up/Down navigate weeks
- **Page Up/Down:** Navigate to the previous/next month (or week in week view)
- **Home/End:** Jump to the first/last day of the current week or month
- **Enter/Space:** Select the focused date (fires date selection callback)
- **Tab:** Move focus between the navigation controls and the calendar grid

Touch-screen

- **Swipe to navigate:** Swiping left/right navigates to next/previous period
- **Tap to select:** Tapping a date cell fires the date selection callback
- **Touch-friendly cells:** Date cells are large enough for reliable touch (minimum 44×44px)
- **Event tap:** Tapping an event within a cell fires the event selection callback

Responsive Behavior

- **Fluid grid:** Calendar grid fills its container width; cells resize proportionally
- **View switching:** On narrow screens, week or day view may be preferred over month view (fewer columns)
- **Event overflow:** When more events exist than fit in a cell, a “+N more” indicator links to a full list
- **Compact month header:** Day-of-week headers abbreviate (“Mon” → “M”) on very narrow screens

Internationalization

- **Locale week start:** First day of the week follows locale convention (Sunday vs Monday)

- **Localized day/month names:** Day-of-week headers and month names are locale-appropriate
- **RTL grid layout:** Calendar columns reverse direction in right-to-left languages
- **Locale-aware date formats:** Date labels in cells use locale-appropriate short formats

Variants & Features

- **Month view:** Shows a full month grid (the default and most common view)
- **Week view:** Shows a single week with hourly time slots for event scheduling
- **Day view:** Shows a single day with hourly time slots
- **Read-only mode:** Events displayed but no selection or interaction
- **Selectable range:** User can click and drag (or click start then shift-click end) to select a date range
- **Event slots:** Events rendered as blocks within time-slotted week/day views

Test Scenarios

- **Calendar renders:** Calendar displays with correct month/week/day grid
- **Today highlighted:** Current date cell is visually distinguished
- **Today button:** Clicking “Today” navigates to the current period
- **Previous/next navigation:** Navigation controls advance and retreat to adjacent periods
- **Date selection:** Clicking a date cell fires the date selection callback with the correct date
- **Event display:** Events passed in as data appear in their correct date cells
- **Multiple events per cell:** When multiple events exist on a date, they all appear (or an overflow indicator is shown)
- **Event click:** Clicking an event fires the event selection callback with the correct event data
- **View switching:** Switching between month/week/day views re-renders the grid correctly
- **Arrow key navigation:** Arrow keys move focus through date cells
- **Page Up/Down:** Page keys navigate to the previous/next period

- **Enter selects:** Enter on a focused date cell fires the selection callback
- **Locale week start:** Calendar first day of week reflects locale setting
- **Localized labels:** Month and day names display in the configured locale
- **RTL grid:** Column order reverses correctly in RTL layout
- **Event overflow indicator:** “+N more” indicator appears when events exceed cell capacity
- **Screen reader grid:** Calendar grid is announced with correct role attributes by screen readers

Notes

- **Calendar vs Date Picker:** Date Picker is an input widget for capturing a date value. Calendar is a display component for viewing and navigating events over time
- **Event data model:** Define a clear event data model (id, date, title, color, metadata) and document it; Calendar should accept event data as a prop/slot
- **Performance:** Month views with many events can become slow. Consider limiting visible events per cell and lazily loading event details
- **Timezone handling:** Display dates in the correct local or specified timezone; mismatches cause events to appear on wrong days
- **Drag-and-drop events:** Moving events by dragging is a complex extension—define it as a separate optional feature and test thoroughly
- **Integration with Date Picker:** Calendar and Date Picker should share the same underlying date navigation and locale logic to ensure consistency

#53 Tabs

Organizes content into multiple panels, displaying one at a time. User switches panels by clicking tab labels.

Naming note: *Distinct from Segmented Control — Tabs switch content panels, Segmented Control selects an option/value.*

Description

Tabs organize related content into multiple sections accessible via labeled tab triggers. Only one tab's content panel is visible at a time; clicking a tab label shows its panel and hides all others. Tabs are appropriate when users need to switch between views of related but distinct content, and when showing all content simultaneously would create excessive length or cognitive load. They are not appropriate for sequential flows (use Stepper) or option selection (use Segmented Control).

Main Features

- **Tab list:** Horizontal row (or vertical column) of tab trigger buttons
- **Active tab indicator:** Visual indicator showing which tab is currently selected
- **Content panels:** Each tab has a corresponding content panel; only the active panel is shown
- **Panel switching:** Clicking a tab label switches the visible panel
- **Initial tab:** Configurable which tab is active on first render
- **Controlled mode:** Active tab can be driven externally

Secondary Features

Accessibility

- **ARIA tabs pattern:** Tab list uses role="tablist"; each tab uses role="tab"; each panel uses role="tabpanel"
- **aria-selected:** Active tab has aria-selected="true"; inactive tabs have aria-selected="false"
- **aria-controls / aria-labelledby:** Each tab references its panel via aria-controls; each panel references its tab via aria-labelledby
- **Focus management:** When a tab is activated, focus moves to the tab button (not the panel); Tab then moves focus into the panel content
- **Hidden panels:** Inactive panels are hidden from the accessibility tree (aria-hidden or not rendered)

Keyboard Navigation

- **Tab to tab list:** Tab key reaches the tab list and focuses the active tab
- **Arrow keys switch tabs:** Left/Right arrows (horizontal tabs) or Up/Down arrows (vertical tabs) move focus through the tab list
- **Automatic or manual activation:** In automatic mode, arrow key focus also activates the tab; in manual mode, Enter/Space is required to activate the focused tab
- **Tab into panel:** Tab from the active tab button moves focus into the active panel's content
- **Home/End:** Jump to first/last tab in the tab list
- **No tab cycling past panel:** Shift+Tab from the first panel element returns focus to the active tab button

Touch-screen

- **Touch-friendly tabs:** Tab triggers meet minimum tap target height (44px)
- **Swipe between panels:** Optionally, swiping left/right on the content panel switches tabs (mobile pattern)
- **Scrollable tab list:** When there are more tabs than fit the width, the tab list scrolls horizontally

Responsive Behavior

- **Scrollable tab bar:** On narrow screens, the tab bar scrolls horizontally rather than wrapping or collapsing
- **Tab truncation:** Long tab labels truncate with ellipsis; tooltips reveal full label on hover
- **Vertical tabs:** On some layouts, tabs may be displayed vertically (left sidebar) instead of horizontally
- **Dropdown overflow:** When tab count is high, extra tabs can collapse into a “More” dropdown

Internationalization

- **RTL tab order:** In RTL, tab list order and arrow key directions are mirrored
- **Localizable labels:** All tab labels are consumer-provided and fully localizable
- **Text direction:** Content within panels inherits correct text direction

Variants & Features

- **Underline style:** Active tab indicated by an underline (most common web pattern)
- **Pill/button style:** Active tab rendered as a filled pill or button
- **Card style:** Tabs styled as folder tabs attached to a card content area
- **Vertical tabs:** Tab list arranged vertically on the left side
- **Closeable tabs:** Individual tabs have a close (×) button for dismissal (browser-tab pattern)
- **Badge on tab:** Tab label includes a count badge (e.g., number of notifications in that panel)

Test Scenarios

- **Tabs render:** Tab list and initial active panel display correctly
- **Active tab indicated:** Active tab has the correct visual state and aria-selected="true"
- **Tab click switches panel:** Clicking a tab shows its panel and hides all others
- **Only one panel visible:** At any time, exactly one panel is visible
- **Arrow key navigation:** Left/Right (or Up/Down for vertical) arrows move focus through tabs
- **Automatic activation:** In automatic mode, arrow key focus immediately activates and shows the new panel
- **Manual activation:** In manual mode, arrow key focus does not switch panels until Enter/Space is pressed
- **Home/End keys:** Focus jumps to first/last tab
- **Tab into panel:** Pressing Tab from the active tab button moves focus into the panel content

- **Shift+Tab returns to tab:** Shift+Tab from the first element in a panel returns focus to the active tab button
- **ARIA roles:** tablist, tab, and tabpanel roles are present and correctly assigned
- **aria-controls linkage:** Each tab's aria-controls references its panel's id
- **aria-labelledby linkage:** Each panel's aria-labelledby references its tab's id
- **Inactive panels hidden:** Inactive panels are not in the accessibility tree
- **Initial tab:** Configured initial tab is active on first render
- **Controlled mode:** When active tab is controlled externally, component reflects and uses that state
- **Badge display:** When configured, count badge appears on tab label
- **RTL layout:** Tab order and arrow key behavior are mirrored in RTL

Notes

- **Tabs vs Segmented Control:** Tabs switch content panels (significant content areas). Segmented Control selects a value or filter (no panel swap). Use the right pattern for the context
- **Tabs vs Stepper:** Tabs imply non-sequential, peer content areas. Stepper implies a progressive sequence with required ordering
- **Automatic vs manual activation:** Automatic mode (arrow = activate) is simpler but can be problematic if panel content takes time to load. Manual mode (Enter to activate) allows users to browse tab labels without triggering loads
- **Don't use tabs for sequential flows:** If the user must complete tabs in order, use a Stepper instead
- **Lazy loading:** For tabs with expensive content, render panel content only when the tab is first activated, then keep it rendered (avoid destroying and recreating on every switch)
- **URL sync:** For significant content areas, consider syncing the active tab to a URL query parameter so users can share or bookmark a specific tab

#54 Accordion

A vertically stacked list of collapsible sections. Builds on Collapsible. Multiple sections can be open simultaneously (configurable).

Description

Accordion presents a vertical stack of sections, each with a header trigger and a collapsible content area. Users expand sections to reveal content and collapse them to hide it. Unlike Tabs (which show one panel at a time with always-visible triggers), Accordion allows multiple sections to be open simultaneously and the triggers are part of the vertical content flow. Accordion builds directly on the Collapsible primitive, adding coordination behavior across multiple sections.

Main Features

- **Multiple sections:** Vertical list of header + content pairs
- **Expand/collapse:** Clicking a section header expands or collapses its content
- **Single or multiple open:** Configurable whether only one section can be open at a time (exclusive) or multiple can be open simultaneously
- **Default open sections:** Configurable which sections start expanded
- **Smooth animation:** Content areas animate open and closed
- **Visual state indicator:** Icon (chevron, plus/minus) shows current expanded/collapsed state per section

Secondary Features

Accessibility

- **ARIA disclosure pattern:** Each header uses a `<button>` with `aria-expanded` and `aria-controls` referencing its content panel
- **Content panel:** Each panel has `role="region"` (optional for important sections) with `aria-labelledby` referencing its header button

- **State announcement:** Screen readers announce expanded/collapsed state when it changes
- **Focus management:** Focus remains on the header button after toggling; does not jump to content

Keyboard Navigation

- **Tab through headers:** Tab moves focus through accordion section header buttons
- **Enter/Space to toggle:** Enter or Space expands or collapses the focused section
- **Arrow keys (optional):** Up/Down arrows can move focus between header buttons (following the ARIA disclosure pattern's optional keyboard support)
- **Home/End (optional):** Jump to first/last section header
- **Focus persistence:** Focus stays on the header button after toggling open or closed

Touch-screen

- **Touch-friendly headers:** Each header meets minimum tap target height (44px)
- **Full-width tap area:** Entire header row is tappable, not just the label text
- **Touch feedback:** Immediate visual press state feedback on tap

Responsive Behavior

- **Full width:** Accordion fills its container width
- **Content overflow:** Long content within an open section wraps and scrolls if necessary
- **Icon positioning:** Expand/collapse icon positioned consistently (typically trailing/right)

Animation & Transitions

- **Height transition:** Content area transitions smoothly from height 0 to natural height on expand (and reverse on collapse)

- **Icon animation:** Chevron or indicator icon rotates or changes smoothly on state change
- **Stagger (optional):** When multiple sections open simultaneously, they can stagger their animations
- **Reduced motion:** Respects prefers-reduced-motion; transitions are instant or minimal

Internationalization

- **RTL icon placement:** Expand/collapse icon appears on the correct side in right-to-left layouts
- **Icon direction:** Chevron direction may flip for RTL (e.g., pointing left when collapsed in RTL)
- **Localizable headers:** All section header labels are consumer-provided and fully localizable

Variants & Features

- **Exclusive mode:** Only one section open at a time; opening a new section closes the previously open one
- **Multi-open mode:** Multiple sections can be open simultaneously (default behavior)
- **Flush/bordered style:** Sections styled with or without borders and separators
- **Nested accordions:** Sections can contain child accordions (handle with care for usability)
- **Disabled sections:** Individual sections can be marked as non-expandable

Test Scenarios

- **Accordion renders:** All section headers display in correct order; all sections start collapsed (unless configured otherwise)
- **Default open sections:** Sections configured as default-open start in the expanded state
- **Expand on click:** Clicking a header expands its content area

- **Collapse on click:** Clicking an open header collapses its content area
- **Icon indicates state:** Expand/collapse icon updates correctly for each section's state
- **Icon animation:** Icon rotates or changes smoothly during transition
- **Content animation:** Content area expands and collapses with smooth height transition
- **Exclusive mode:** In exclusive mode, opening one section closes the previously open section
- **Multi-open mode:** In multi-open mode, multiple sections can be expanded simultaneously
- **Keyboard toggle:** Enter/Space on a focused header toggles its section
- **Tab through headers:** Tab moves focus through each section header in order
- **aria-expanded:** Each header's aria-expanded reflects its section's current state
- **aria-controls:** Each header's aria-controls references the correct content panel id
- **Screen reader state:** Expanded/collapsed state changes are announced correctly
- **Focus persistence:** Focus remains on the header button after toggling
- **Disabled section:** Disabled sections do not expand on click or keyboard activation
- **Reduced motion:** Content appears/disappears instantly when user prefers reduced motion
- **RTL layout:** Expand/collapse icon appears on the correct side in RTL

Notes

- **Accordion vs Tabs:** Accordion is for long, vertically scanned content where multiple sections may be relevant. Tabs are for switching between peer content areas where only one is relevant at a time
- **Exclusive vs multi-open:** Exclusive mode (only one open) reduces cognitive load but may frustrate users who need to compare sections. Choose based on whether sections are complementary or alternatives

- **Build on Collapsible:** Accordion should compose the Collapsible primitive for each section rather than reimplementing expand/collapse logic
- **Animation height:** CSS transitions from height: 0 to height: auto require workarounds (max-height, JavaScript measurement, or the newer CSS `interpolate-size` property). Plan this carefully
- **Don't nest deeply:** Nested accordions beyond one level become confusing and hard to navigate
- **Performance:** For accordions with many sections or heavy content, consider lazy-rendering content until a section is first opened

#55 Stepper

Guides the user through a multi-step process. Displays current step, completed steps, and remaining steps.

Specializations: Horizontal Stepper, Vertical Stepper

Description

Stepper communicates progress through a defined, ordered sequence of steps—such as a multi-page form, checkout flow, or onboarding wizard. It shows the user where they are, what they have completed, and what remains. Unlike Tabs (peer content areas), steps are sequential and often have dependencies: a later step may not be accessible until earlier ones are completed. Stepper is a navigation and progress indicator; the actual content for each step is rendered by the parent and slotted into the Stepper's content area.

Main Features

- **Step list:** Ordered list of step indicators showing labels and state (upcoming, active, completed, error)
- **Current step indicator:** Clearly marks which step is active
- **Completed step indicator:** Visually distinguishes steps the user has finished

- **Error step indicator:** Marks steps that have validation errors (used in non-linear steppers)
- **Navigation controls:** Next and Back buttons advance or retreat through the sequence
- **Step content area:** A slot or region where the active step's content is rendered
- **Linear vs non-linear:** Configurable whether users can jump to any step or must proceed in order

Secondary Features

Accessibility

- **ARIA progress indicator:** Stepper uses aria-label or visually hidden text to announce overall progress (e.g., “Step 2 of 5”)
- **Active step:** The active step indicator has aria-current=“step”
- **Completed steps:** Completed step indicators communicate their state to screen readers (via visually hidden text or aria-label)
- **Navigation buttons:** Next and Back buttons are standard buttons with descriptive labels
- **Step content region:** The content area has role=“region” with an aria-label matching the active step's label
- **Error state:** Steps with errors have accessible error indication (not color alone)

Keyboard Navigation

- **Tab through controls:** Tab moves through the step indicators (if clickable) and the Next/Back navigation buttons
- **Enter/Space on step:** In non-linear mode, pressing Enter or Space on a step indicator navigates to that step
- **Next/Back buttons:** Standard button keyboard behavior; Enter or Space activates

- **Focus on step change:** When the step changes, focus moves to a logical starting point in the new step's content (typically the step heading or first input)

Touch-screen

- **Touch-friendly step indicators:** Step indicator tap targets meet 44×44px minimum
- **Swipe between steps (optional):** Swiping left/right within the content area can advance or retreat steps
- **Touch-friendly navigation buttons:** Next and Back buttons have adequate height and padding

Responsive Behavior

- **Horizontal to vertical:** On narrow viewports, a horizontal Stepper may collapse to a vertical layout or show only the current step label with a “Step N of M” indicator
- **Label truncation:** Long step labels truncate with ellipsis on narrow screens; full labels available via tooltip
- **Compact mode:** On very narrow screens, step indicators may reduce to dots or numbers without labels

Internationalization

- **RTL step order:** In RTL, step indicators flow right to left; Back and Next button semantics remain consistent but their visual positions swap
- **Localizable labels:** Step labels, Next, Back, and progress text are consumer-provided and fully localizable
- **Locale-aware progress text:** “Step 2 of 5” phrasing is localizable with correct number formatting

Variants & Features

- **Linear mode:** User must complete each step in order; future steps are disabled or non-clickable

- **Non-linear mode:** User can jump to any step; all steps are clickable
- **Optional steps:** Individual steps can be marked as optional and skippable
- **Step validation:** Steps can be marked with error state to indicate incomplete or invalid data
- **Numbered steps:** Step indicators show sequential numbers
- **Icon steps:** Completed steps show a checkmark icon; error steps show a warning icon
- **Step descriptions:** Optional short description text below each step label

Test Scenarios

- **Stepper renders:** All step indicators display in order with correct labels
- **Active step highlighted:** Current step has correct visual state and aria-current="step"
- **Completed steps indicated:** Steps before the current step show completed styling
- **Next advances step:** Clicking Next moves to the next step; content area updates
- **Back retreats step:** Clicking Back moves to the previous step; content area updates
- **First step Back disabled:** Back button is disabled or hidden on the first step
- **Last step Next label:** On the final step, Next button label changes (e.g., "Submit" or "Finish")
- **Step content updates:** The content area renders the correct content for the active step
- **Focus on step change:** When step changes, focus moves to the correct element in the new content
- **Linear mode restriction:** In linear mode, future steps are not clickable or navigable
- **Non-linear step click:** In non-linear mode, clicking a step indicator navigates directly to that step
- **Optional step skip:** Optional steps can be skipped without blocking progression

- **Error state on step:** Step with error shows error styling and is accessible to screen readers
- **Progress announcement:** Screen readers announce current position (e.g., “Step 2 of 5”)
- **aria-current:** Active step indicator has `aria-current=“step”`
- **RTL order:** Step indicators flow in the correct direction in RTL layouts
- **Responsive collapse:** On narrow viewport, stepper adapts layout (vertical or compact indicator)

Horizontal Stepper Specialization

Additional Features:

- Step indicators arranged in a horizontal row with connector lines between them
- Labels appear below each step indicator
- Connector lines between steps change style (active/completed) as the user progresses
- Most appropriate for 3–5 steps; larger step counts become crowded horizontally

Additional Test Scenarios:

- **Horizontal layout:** Step indicators render in a single horizontal row
- **Connector lines:** Lines between step indicators display and update state (pending, completed) as steps are completed
- **Label alignment:** Step labels appear below their corresponding indicators and are centered
- **Overflow handling:** When steps exceed available width, layout adapts (scrolls, collapses, or wraps) without breaking

Vertical Stepper Specialization

Additional Features:

- Step indicators stacked vertically; content for each step can appear inline below its indicator (expanded) rather than in a separate content area
- Connector lines run vertically between step indicators
- Each step's content expands below its header when active; collapses when completed or upcoming
- Well-suited for longer step labels, step descriptions, or when step content is short enough to display inline

Additional Test Scenarios:

- **Vertical layout:** Step indicators render in a vertical stack
- **Vertical connector lines:** Vertical connector lines appear between steps and update with completion state
- **Inline content expansion:** When a step is active, its content expands inline below the step header
- **Completed step collapses:** When moving to the next step, the previous step's inline content collapses (showing a summary if configured)
- **Label wrapping:** Long step labels wrap vertically without truncation

#56 Pagination

Allows navigation through content split across multiple pages. Displays current page, total pages, and prev/next controls.

Description

Pagination breaks large datasets or content collections into discrete pages and provides controls for navigating between them. It communicates the user's current position within the full set and allows direct navigation to specific pages. Pagination is appropriate when the full dataset is too large to display at once and sequential or

random-access page navigation is useful. It differs from infinite scroll (which loads continuously) and “load more” patterns (which append content without replacing it).

Main Features

- **Previous and Next controls:** Buttons or links to move one page backward or forward
- **Page number buttons:** Clickable page numbers for direct navigation to a specific page
- **Current page indicator:** The active page is visually distinguished and communicated to screen readers
- **Ellipsis for large ranges:** When the total page count is high, middle pages are collapsed with an ellipsis (...)
- **First and Last page:** Quick links to the first and last page (configurable)
- **Total pages context:** User can understand total scope (e.g., page 3 of 47)

Secondary Features

Accessibility

- **Semantic navigation:** Pagination wrapped in a `<nav>` element with `aria-label="Pagination"`
- **Current page:** Active page button has `aria-current="page"`
- **Disabled controls:** Previous button has `aria-disabled="true"` on the first page; Next button on the last page
- **Descriptive labels:** Page number buttons have `aria-label` (e.g., “Page 3”, “Go to page 3”) not just the number
- **Ellipsis:** Ellipsis elements are presentational (`aria-hidden`) with no interactive role
- **Screen reader context:** Hidden or visually presented text communicates “Page 3 of 47” to screen readers

Keyboard Navigation

- **Tab through controls:** Tab moves focus through Previous, page numbers, ellipsis (skipped), and Next
- **Enter/Space to activate:** Enter or Space activates the focused page button or Previous/Next
- **Arrow key navigation (optional):** Left/Right arrows can move between page controls when focus is within the pagination component

Touch-screen

- **Adequate tap targets:** All page number buttons and Previous/Next controls meet 44×44px minimum
- **Simplified mobile pagination:** On narrow screens, show only Previous, current page indicator, and Next (hiding individual page numbers)
- **No hover dependency:** All interactions work without hover

Responsive Behavior

- **Compact mode on mobile:** Collapses to “< Prev | Page 3 of 47 | Next >” on narrow viewports
- **Configurable page window:** Number of visible page buttons around the current page is configurable (e.g., show 2 pages on each side)
- **Ellipsis logic:** Always shows first and last page; collapses middle pages with ellipsis when range is large

Internationalization

- **RTL control order:** In RTL, Previous and Next controls swap visual positions; Previous is on the right
- **RTL arrow direction:** Chevron icons on Previous/Next flip direction in RTL
- **Localizable labels:** “Previous”, “Next”, “Page N of M” text is consumer-provided and localizable
- **Locale number formatting:** Page numbers use locale-appropriate numeral systems

Variants & Features

- **Page size selector:** Optional “Rows per page” or “Items per page” dropdown paired with pagination
- **Jump to page input:** Text input allowing direct entry of a page number
- **Simple prev/next only:** Minimal variant showing only Previous and Next (for contexts where total page count is unknown or irrelevant)
- **Load-more variant:** Appends next page to existing content rather than replacing it (different UX pattern but same component can support it)

Test Scenarios

- **Pagination renders:** Page controls display with correct current page highlighted
- **Previous navigates:** Clicking Previous decrements the current page
- **Next navigates:** Clicking Next increments the current page
- **Direct page navigation:** Clicking a page number navigates directly to that page
- **First page Previous disabled:** Previous button is disabled or absent on page 1
- **Last page Next disabled:** Next button is disabled or absent on the last page
- **Current page indicator:** Active page has correct visual state and aria-current=“page”
- **Ellipsis appears:** When total pages exceed the display window, ellipsis appears between page ranges
- **First and last always visible:** Pages 1 and last page always render even when ellipsis is present
- **aria-label on buttons:** Page buttons have descriptive aria-labels beyond just the number
- **nav landmark:** Pagination is wrapped in a <nav> element with aria-label=“Pagination”
- **Keyboard tab order:** Tab moves through controls in logical order, skipping ellipsis elements

- **Page size selector:** When included, changing items-per-page updates the page count and resets to page 1
- **Jump to page:** When included, entering a page number and confirming navigates to that page
- **Callback execution:** `onPageChange` fires with the new page number when navigation occurs
- **RTL layout:** Previous/Next positions and chevron directions are correct in RTL
- **Responsive collapse:** On narrow viewport, pagination collapses to compact mode

Notes

- **Pagination vs infinite scroll:** Pagination is better when users need to find items at specific positions or return to a previous position. Infinite scroll is better for discovery-oriented feeds
- **Unknown total:** When total count is unknown (e.g., streaming API), use simple Prev/Next only—don't show page numbers or a total
- **Page size changes reset page:** When the user changes items-per-page, always reset to page 1 to avoid showing an empty or invalid page
- **URL sync:** Sync the current page to a URL query parameter so users can share, bookmark, or use the browser Back button to return to a page
- **Loading states:** Show a loading indicator when page content is being fetched; disable navigation controls during loading

#57 Table

Displays tabular data in rows and columns with optional sorting, filtering, and row-level actions.

Description

Table displays structured data in a grid of rows and columns, making it easy to compare values across records. At its simplest, it is a semantic HTML table with styled rows and headers. In more advanced configurations, it supports column sorting, row selection, inline editing, row-level actions, column resizing, and pagination. Table is one of the most complex components in a UI library due to the breadth of features it must coordinate.

Main Features

- **Column headers:** Labeled headers that identify each column's data
- **Data rows:** Each row represents one record; cells display the corresponding field value
- **Striped or bordered rows:** Visual row differentiation for readability
- **Empty state:** Displays an empty state component when there is no data to show
- **Loading state:** Skeleton or spinner overlay while data is being fetched
- **Responsive overflow:** Horizontally scrollable when columns exceed the container width

Secondary Features

Accessibility

- **Semantic HTML:** Uses `<table>`, `<thead>`, `<tbody>`, `<tr>`, `<th>`, `<td>` elements
- **Column scope:** `<th>` elements have `scope="col"`; row header cells have `scope="row"`
- **Caption:** Optional `<caption>` element describing the table's content
- **Sort state:** Sortable column headers have `aria-sort="ascending"` / `"descending"` / `"none"`
- **Row selection:** Selected rows have `aria-selected="true"`; the selection checkbox has an accessible label

- **Screen reader row context:** Complex tables use headers/id associations so screen readers announce column context for each cell

Keyboard Navigation

- **Tab to table:** Table or its first interactive element is reachable via Tab
- **Arrow key navigation (grid pattern):** In interactive tables, Up/Down/Left/Right arrows navigate between cells
- **Enter to activate:** Enter activates cell-level actions (sort header, inline edit, action button)
- **Space to select row:** Space toggles row selection when focus is on a row or its checkbox
- **Tab through interactive cells:** In simpler tables, Tab moves between interactive elements (buttons, checkboxes) within the table

Touch-screen

- **Horizontal scroll:** Table scrolls horizontally within a scroll container on touch devices
- **Touch-friendly row actions:** Action buttons within rows meet 44×44px minimum
- **Tap to sort:** Tapping a sortable column header cycles the sort state
- **Row tap action:** Tapping a row can trigger a row-level action (navigate, expand detail, etc.)

Responsive Behavior

- **Horizontal scroll container:** Table is wrapped in a scroll area to prevent layout breakage on narrow screens
- **Column pinning:** Key columns (e.g., first column, action column) can be pinned/sticky during horizontal scroll
- **Column hiding:** Non-essential columns can be hidden at narrow breakpoints
- **Card layout on mobile:** Extreme responsive variant transforms each row into a card (column label + value pairs stacked vertically)

Internationalization

- **RTL column order:** In RTL, column order is mirrored and text alignment follows reading direction
- **Locale-aware data formatting:** Dates, numbers, and currencies in cells use locale-appropriate formatting
- **Localizable headers:** Column labels and action labels are consumer-provided and fully localizable

Variants & Features

- **Sortable columns:** Clicking a column header sorts the data by that column; clicking again reverses order
- **Row selection:** Checkboxes in each row for single or multiple row selection; “select all” checkbox in the header
- **Row expansion:** Rows can expand to reveal additional detail in a sub-row beneath them
- **Inline editing:** Cells can be clicked to enter an edit mode with an input field
- **Row-level actions:** Action buttons or a dropdown menu at the end of each row (edit, delete, view)
- **Column resizing:** Column width adjustable by dragging the column border
- **Column reordering:** Columns can be dragged to change their order
- **Sticky header:** Column header row remains visible during vertical scroll
- **Sticky columns:** One or more columns remain fixed during horizontal scroll
- **Pagination integration:** Pairs with Pagination component for large datasets
- **Density control:** Compact, default, and comfortable row height variants

Test Scenarios

- **Table renders:** Headers and data rows display correctly
- **Column headers labeled:** Each `<th>` has the correct label
- **Scope attributes:** Column headers have `scope="col"`; row headers have `scope="row"`
- **Data rows match data:** Each row's cells reflect the correct record values

- **Empty state:** When data is empty, empty state component displays inside the table
- **Loading state:** While data is loading, skeleton or spinner displays
- **Sortable column click:** Clicking a sortable header sorts data ascending; clicking again reverses to descending; clicking a third time may clear the sort
- **Sort indicator:** aria-sort reflects current sort state on the active column
- **Row selection checkbox:** Checking a row checkbox selects that row; row has aria-selected="true"
- **Select all checkbox:** Header checkbox selects all rows; deselecting it clears all selections
- **Row expansion:** Clicking an expand control reveals the sub-row; clicking again collapses it
- **Row action fires:** Clicking a row-level action button fires the correct callback with the row's data
- **Horizontal scroll:** Table scrolls horizontally when columns exceed container width
- **Sticky header:** Column headers remain visible when scrolling vertically through many rows
- **Sticky column:** Pinned column remains visible when scrolling horizontally
- **Inline edit:** Double-clicking (or clicking) an editable cell enters edit mode; confirming fires the edit callback
- **Keyboard arrow navigation:** In grid-pattern navigation mode, arrow keys move between cells
- **RTL layout:** Column order, text alignment, and sort indicators are correct in RTL
- **Locale data formatting:** Dates and numbers in cells use locale-appropriate formats

Notes

- **Complexity management:** Table is the most feature-rich component in most libraries. Build the base (static, semantic table) first and add features incrementally

- **Server-side vs client-side:** Sorting, filtering, and pagination can be handled client-side (component manages data) or server-side (component fires events; parent fetches new data). Design the API to support both
- **Virtualization:** For datasets with 100+ rows, virtualize row rendering (only render visible rows) for performance
- **Column definition model:** Define a clear column definition object (key, label, sortable, width, renderCell, etc.) that drives all column behavior
- **Accessibility complexity:** Complex interactive tables (grid pattern) require significant ARIA investment. The ARIA grid pattern is distinct from the simpler table pattern—choose the right one for the level of interactivity
- **Mobile tables:** True tabular data is difficult on mobile. Consider whether a card layout, a simplified table with fewer columns, or a detail-on-tap pattern better serves mobile users

#58 Sortable List

A list whose items can be reordered by dragging and dropping.

Description

Sortable List allows users to reorder items within a list by dragging them to a new position. It is used for priority lists, ordered tasks, custom rankings, playlist ordering, and any scenario where the sequence of items carries meaning and should be user-configurable. The component handles drag state, drop target highlighting, and emits the new order when a drag operation is completed.

Main Features

- **Draggable items:** Each list item can be grabbed and dragged to a new position
- **Drop target indicator:** Visual feedback shows where the dragged item will be dropped
- **Reorder on drop:** Releasing the item at a new position updates the list order

- **Drag handle:** Optional dedicated drag handle icon per item (better than making the entire row draggable when items contain interactive elements)
- **Order callback:** Fires with the new item order when a drag is completed
- **Keyboard reordering:** Items can be repositioned via keyboard for accessibility

Secondary Features

Accessibility

- **Keyboard drag-and-drop:** Items can be “picked up” with Space, moved with arrow keys, and “dropped” with Space again—as a keyboard alternative to drag-and-drop
- **ARIA live region:** Announces reorder operations to screen readers (“Item moved from position 2 to position 4”)
- **Drag handle label:** If a drag handle is used, it has an accessible aria-label (e.g., “Drag to reorder [item label]”)
- **Instructions:** A visually hidden or visible hint explains how to reorder via keyboard (shown on focus)
- **Role:** Items may use role=“listitem” within a role=“list” container; during keyboard drag, appropriate aria-grabbed or aria-roledescription is used

Keyboard Navigation

- **Tab to item or handle:** Tab moves focus to each list item or its drag handle
- **Space to pick up:** Space “picks up” the focused item and enters reorder mode
- **Arrow keys to move:** Up/Down (or Left/Right) arrows move the picked-up item toward its new position
- **Space to drop:** Space drops the item at the current position and exits reorder mode
- **Escape to cancel:** Escape cancels the reorder operation and returns the item to its original position
- **Announcement on move:** Each arrow key press announces the new position to screen readers

Touch-screen

- **Touch drag:** Long-press on an item (or drag handle) initiates drag; dragging repositions the item
- **Auto-scroll:** When dragging near the top or bottom of a scrollable list, the list auto-scrolls
- **Touch drop target:** Drop target indicator is visible and adequately sized for touch
- **Haptic feedback (optional):** On supported devices, haptic feedback confirms pick-up and drop

Responsive Behavior

- **Flexible width:** List fills its container
- **Scroll within list:** If the list has a fixed height, it scrolls while drag auto-scroll activates near edges
- **Item size consistency:** Items should be uniform in height for predictable drop targeting (configurable)

Internationalization

- **RTL drag direction:** In RTL, horizontal drag semantics reverse; vertical remains unchanged
- **Localized announcements:** Screen reader live region announcements are consumer-localizable
- **RTL handle position:** Drag handle icon appears on the correct side in RTL layouts

Variants & Features

- **Handle-only drag:** Drag is initiated only from a dedicated handle icon, not the entire item (allows interactive elements within items)
- **Whole-item drag:** Entire item surface initiates drag (simpler, appropriate when items have no interactive sub-elements)

- **Drag overlay:** A ghost/clone of the dragged item follows the cursor during drag
- **Disabled items:** Individual items can be marked as non-draggable (fixed position)
- **Multi-list drag:** Items can be dragged between two or more sibling lists (cross-list transfer)

Test Scenarios

- **List renders:** All items display in initial order
- **Mouse drag reorders:** Dragging an item and dropping at a new position updates the order
- **Drop target indicator:** Visual indicator shows where the dragged item will land during drag
- **Order callback fires:** onReorder fires with the correct new item order after a drop
- **Drag cancel:** Releasing the item outside a valid drop zone returns it to its original position
- **Drag handle restricts drag origin:** When handle-only mode is set, drag only initiates from the handle icon
- **Disabled item fixed:** A disabled item cannot be dragged and other items cannot be dropped onto its position
- **Keyboard pick up:** Pressing Space on a focused item enters keyboard reorder mode
- **Keyboard arrow move:** Arrow keys move the picked-up item; position updates visually with each press
- **Keyboard drop:** Pressing Space drops the item at the current position; list order updates
- **Keyboard escape cancels:** Pressing Escape returns the item to its original position
- **Screen reader announcement:** Each keyboard move announces the new position (e.g., “Item moved to position 3 of 7”)
- **Touch drag:** Long-press initiates drag on touch devices; releasing at a new position reorders

- **Auto-scroll:** Dragging near the top or bottom of a scrollable list causes the list to scroll
- **RTL layout:** Drag handle and any horizontal drag semantics are correct in RTL

Notes

- **Handle vs whole-item drag:** Always use a dedicated drag handle when list items contain buttons, links, or other interactive elements—otherwise dragging conflicts with clicking
- **Keyboard is mandatory:** Mouse-only drag-and-drop is inaccessible. Keyboard reordering is not optional
- **Implementation libraries:** Drag-and-drop logic is complex. Libraries like dnd-kit, react-beautiful-dnd, or SortableJS handle the heavy lifting reliably
- **Drag overlay:** Rendering a clone/ghost of the item that follows the cursor (rather than animating the original) produces smoother visual feedback
- **Order as data:** Emit the new order as an array of item IDs (or the full item objects), not positional indices, to decouple from rendering order
- **Performance:** For lists with many items, ensure drag tracking does not cause excessive re-renders. Use virtualization if needed

#59 Transfer List

Two lists side by side allowing items to be moved between them. Common in permission and assignment interfaces.

Description

Transfer List presents two lists—typically “Available” and “Selected” (or “Source” and “Target”)—and allows users to move items between them. Users select one or more items in one list and use action buttons (or drag-and-drop) to transfer them to the other list. It is commonly used for assigning permissions, selecting team

members, configuring feature sets, or any scenario where users must explicitly choose a subset of items from a larger pool.

Main Features

- **Two lists:** Source list (available items) and target list (selected/assigned items) displayed side by side
- **Item selection:** Users can select one or more items in either list (checkboxes or click-to-select)
- **Transfer buttons:** Action buttons between the lists move selected items from one side to the other (>, <, >>, <<)
- **Move all:** Controls to move all items from one list to the other at once
- **Order preservation:** Items appear in a consistent order within each list; optionally items can be reordered after transfer
- **Filtering:** Optional search/filter inputs above each list to find items in large sets

Secondary Features

Accessibility

- **List roles:** Each list uses role="listbox" with role="option" for items; or role="list" / role="listitem" depending on interaction model
- **aria-selected:** Selected items have aria-selected="true"
- **aria-multiselectable:** Listboxes are multi-selectable (aria-multiselectable="true")
- **Transfer button labels:** Action buttons have descriptive aria-labels ("Move selected to available", "Move all to selected") rather than just symbols
- **Live region:** A live region announces transfer results ("3 items moved to Selected")
- **List headings:** Each list has a visible heading (or aria-label) identifying its purpose

Keyboard Navigation

- **Tab between lists and buttons:** Tab moves focus between the source list, transfer buttons, and target list
- **Arrow keys within list:** Up/Down arrows navigate items within the focused list
- **Space to select item:** Space toggles selection of the focused item
- **Enter on transfer button:** Enter activates the focused transfer button
- **Ctrl+A to select all:** Ctrl+A selects all items in the focused list (optional but useful)

Touch-screen

- **Touch-friendly item rows:** Each item row meets 44×44px minimum tap target
- **Tap to select:** Tapping an item toggles its selection state
- **Transfer button size:** Transfer action buttons are large enough for reliable touch activation
- **Responsive stacking:** On narrow screens, the two lists stack vertically rather than side by side

Responsive Behavior

- **Side-by-side on desktop:** Lists appear horizontally with transfer buttons between them
- **Stacked on mobile:** Lists stack vertically with transfer buttons above/below or between
- **Scrollable lists:** Each list scrolls independently when the item count exceeds its visible height
- **List height:** Lists share equal height or have configurable individual heights

Internationalization

- **RTL list order:** In RTL, source list appears on the right and target list on the left; transfer button directions reverse accordingly

- **RTL button icons:** Transfer button arrow icons flip direction in RTL
- **Localizable labels:** List headings, button labels, and status messages are consumer-provided and fully localizable

Variants & Features

- **Ordered target list:** Items in the target list can be reordered (combining Transfer List with Sortable List)
- **Drag-and-drop transfer:** Items can be dragged directly from one list to the other instead of using buttons
- **Search/filter per list:** Each list has its own search input to filter visible items
- **Item count display:** Each list header shows the current item count (e.g., “Available (12)”)
- **Disabled items:** Individual items can be non-transferable (grayed out, excluded from selection)

Test Scenarios

- **Renders two lists:** Source and target lists display side by side with headings
- **Items in correct lists:** Initial items appear in their configured starting list
- **Item selection:** Clicking or pressing Space on an item toggles its selected state (aria-selected updates)
- **Transfer selected right:** Clicking the “>” button moves all selected source items to the target list
- **Transfer selected left:** Clicking the “<” button moves all selected target items back to the source list
- **Transfer all right:** Clicking the “>>” button moves all source items to the target list
- **Transfer all left:** Clicking the “<<” button moves all target items back to the source list
- **Button disabled when no selection:** Transfer-selected buttons are disabled when no items are selected in the relevant list
- **Move-all button always available:** Move-all buttons are only disabled when the source list is empty

- **Items removed from origin:** After transfer, moved items are no longer present in the source list
- **Selection cleared after transfer:** After transfer, the moved items' selection state is cleared
- **Live region announcement:** Screen reader hears confirmation of how many items were moved
- **Keyboard navigation:** Arrow keys navigate within a list; Space selects; Tab moves between lists and buttons
- **Filter input:** When present, typing in the filter input narrows visible items; hidden items are excluded from transfer
- **Disabled item:** Disabled items cannot be selected or transferred
- **RTL layout:** Lists appear in reversed positions; transfer button icons flip direction
- **Callback execution:** onTransfer fires with updated source and target item arrays after each transfer

Notes

- **Clear button affordance:** Transfer button symbols (>, <, >>, <<) are not universally understood. Always include a tooltip or visible label
- **Ordered target list:** If order matters in the target list (e.g., priority ranking), combine Transfer List with Sortable List in the target panel
- **Large item sets:** For hundreds of items, add a search/filter above each list—scrolling through a long list to find items is painful
- **Drag-and-drop:** Drag-and-drop transfer feels more direct but requires a keyboard fallback. Button-based transfer is simpler and accessible by default
- **Empty state:** Both lists should display an empty state when all items have been transferred out of them

#60 Carousel

Displays a set of items in a horizontally scrollable or sliding view, one or more at a time.

Description

Carousel presents a collection of items (images, cards, or content blocks) in a horizontally sliding or scrolling view. Users can navigate through items using previous/next controls, pagination dots, or direct touch/drag interaction. Carousels can show one item at a time (full-bleed) or multiple partial items (peeking) to hint at additional content. They are commonly used for image galleries, featured content, product showcases, and testimonials.

Main Features

- **Sliding panels:** Items arranged horizontally; one or more are visible at a time
- **Previous/Next controls:** Buttons to advance or retreat through items
- **Pagination indicators:** Dots or thumbnails showing position within the set and allowing direct navigation
- **Loop behavior:** Optional continuous looping (after last item, next returns to first)
- **Peek:** Partially visible adjacent items hint that more content exists
- **Auto-play:** Optional automatic advancement on a timer

Secondary Features

Accessibility

- **ARIA roving tabindex or role="group":** Each slide uses role="group" with aria-label (e.g., "Item 1 of 5")
- **Live region (auto-play):** Auto-playing carousels use aria-live="off" by default; when content changes, the region is announced appropriately
- **Pause auto-play on focus:** Auto-play pauses when any element within the carousel receives focus (WCAG 2.2.2)
- **Pause on hover:** Auto-play pauses when the user hovers over the carousel
- **Previous/Next labels:** Navigation buttons have descriptive aria-labels, not just icon-only

- **Pagination dots:** Dot indicators have aria-labels (“Go to item 3”) and the active dot has aria-current=“true”
- **Reduced motion:** Auto-play is disabled and slide animations are suppressed when prefers-reduced-motion is set

Keyboard Navigation

- **Tab to controls:** Previous, Next, and pagination dots are reachable via Tab
- **Arrow keys (optional):** Left/Right arrow keys can advance the carousel when focus is within it
- **Enter/Space on controls:** Activates the focused Previous, Next, or pagination dot
- **Tab into slide content:** Focusable elements within visible slides are reachable via Tab; items in hidden slides are not focusable

Touch-screen

- **Swipe to navigate:** Swiping left/right navigates to the next/previous item
- **Momentum and snap:** After a swipe, the carousel snaps to the nearest item
- **Touch-friendly controls:** Previous/Next buttons and dots meet 44×44px minimum tap target
- **Drag to scroll:** Dragging the carousel slowly shows a panning motion before snapping

Responsive Behavior

- **Items per view:** Configurable number of items visible at one time; can change at different breakpoints
- **Fluid item width:** Items fill the available width divided by the configured visible count
- **Control visibility:** Previous/Next arrow buttons may hide on touch devices where swiping is the primary interaction
- **Dot count:** When total items are many, dots may be replaced by a progress bar or abbreviated indicator

Animation & Transitions

- **Slide transition:** Items animate horizontally when navigating (slide or fade)
- **Transition duration:** Configurable; typically 300–500ms
- **Reduced motion:** Transitions are instant or minimal when user prefers reduced motion
- **Auto-play timing:** Configurable delay between auto-advances (typically 4–6 seconds)

Internationalization

- **RTL slide direction:** In RTL, the slide direction reverses (Previous/Next semantics remain the same but visually swap)
- **RTL arrow icons:** Previous/Next button icons flip in RTL
- **Localizable controls:** All button labels and aria-labels are consumer-provided and localizable

Variants & Features

- **Image gallery:** Full-bleed images with optional captions; lightbox on click
- **Card carousel:** Shows multiple cards partially; peeking effect hints at more
- **Auto-play with pause:** Timer-driven advancement with visible pause/play control
- **Thumbnail navigation:** Strip of thumbnail images below the main slide for direct navigation
- **Fade transition:** Alternative to slide—items fade in/out rather than sliding

Test Scenarios

- **Carousel renders:** Initial item(s) display correctly
- **Next advances:** Clicking Next shows the next item
- **Previous retreats:** Clicking Previous shows the previous item
- **Previous disabled on first item:** Previous button disabled or hidden on first item (when not looping)

- **Next disabled on last item:** Next button disabled or hidden on last item (when not looping)
- **Loop behavior:** When looping, Next on the last item returns to the first; Previous on the first item goes to the last
- **Pagination dots render:** Dots display matching the item count; active dot indicates current item
- **Dot click navigates:** Clicking a pagination dot navigates to that item
- **Swipe next:** Swiping left navigates to the next item on touch devices
- **Swipe previous:** Swiping right navigates to the previous item on touch devices
- **Snap behavior:** After swipe, carousel snaps to the nearest item
- **Auto-play advances:** When enabled, carousel automatically advances after the configured interval
- **Auto-play pauses on hover:** Hovering over the carousel pauses auto-play
- **Auto-play pauses on focus:** When any element in the carousel receives focus, auto-play pauses
- **Auto-play disabled with reduced-motion:** Auto-play does not run when user prefers reduced motion
- **Hidden slide items not focusable:** Interactive elements within non-visible slides are not reachable via Tab
- **ARIA group labels:** Each slide has role="group" and aria-label indicating its position
- **Slide animation plays:** Items animate during navigation (unless reduced motion)
- **RTL direction:** Slide direction and arrow icons are correct in RTL

Notes

- **Carousels and usability:** Research consistently shows carousels have low interaction rates, especially for later items. Consider whether a static grid or featured item is a better pattern
- **Auto-play controversy:** Auto-play is generally bad for accessibility and usability. If required, always provide a pause control and disable for prefers-reduced-motion

- **Hidden slide focusability:** Items in off-screen slides must be hidden from the tab order (visibility: hidden, display: none, or inert attribute) or keyboard users will Tab into invisible content
- **Infinite loop complexity:** Looping carousels require cloning items at the start and end of the track; handle this carefully to avoid layout and accessibility issues
- **Dots vs thumbnails:** Dots are fine for 3–7 items. For larger sets, thumbnails or a progress bar communicate position more clearly
- **CSS scroll snap:** Modern CSS scroll-snap provides a simpler implementation path than JavaScript-driven carousels for basic use cases

#61 Masonry

A variable-height grid layout where items are packed tightly based on available vertical space, similar to a Pinterest-style layout.

Description

Masonry lays out items in a multi-column grid where each column is filled independently from top to bottom. Unlike a regular grid, items are not constrained to uniform row heights—each item occupies only the vertical space its content requires, and the next item is placed in whichever column has the most remaining space. This eliminates large gaps caused by uneven content and creates a visually dense, compact layout. Masonry is commonly used for image galleries, content feeds, and card collections where items have variable heights.

Main Features

- **Variable-height items:** Items can have any height; the layout adapts to each item's natural size
- **Multi-column layout:** Items are distributed across a configurable number of columns

- **Tight packing:** Items placed in the shortest column at each step, minimizing vertical gaps
- **Responsive column count:** Number of columns changes at configurable breakpoints
- **Re-layout on resize:** Layout recalculates when the viewport or container width changes
- **Dynamic content support:** New items added to the list are placed into the layout without a full re-render

Secondary Features

Accessibility

- **Logical reading order:** Masonry layouts are visually column-ordered but should maintain a logical DOM/reading order (typically the order items were added) for screen readers and keyboard users
- **Focus order:** Tab order follows the DOM order (original item sequence), not the visual column order
- **No reordering for AT:** Screen readers traverse items in the order they appear in the DOM, which may differ from visual order—document this clearly and ensure it is still meaningful
- **Image alt text:** Image-heavy masonries must ensure all images have appropriate alt text

Keyboard Navigation

- **Tab through items:** Tab moves through interactive elements within each item in DOM order
- **Arrow keys (optional grid navigation):** If implementing grid-style keyboard navigation, arrow keys should follow a logical (not visual) grid path
- **Focus visible:** Focus indicators are clearly visible on all interactive items

Touch-screen

- **Touch scrolling:** Masonry container scrolls natively with touch

- **Tap to interact:** Tapping an item triggers its primary action (expand, navigate, select)
- **No horizontal scroll:** Masonry should never require horizontal scrolling

Responsive Behavior

- **Breakpoint-driven columns:** Column count steps down as viewport narrows (e.g., 4 columns → 2 columns → 1 column)
- **Configurable column gap:** Horizontal and vertical gaps between items are configurable
- **Container width aware:** Layout uses the container's width, not the viewport, for column calculations (works within sidebars and constrained panels)
- **Re-layout on item change:** Adding, removing, or resizing items triggers a re-layout calculation

Internationalization

- **RTL column fill:** In RTL, column filling begins from the right rather than the left
- **Content direction:** Item content inherits the correct text direction
- **No text-specific layout logic:** Masonry layout is purely positional and does not depend on text directionality

Variants & Features

- **CSS column-count:** Simplest implementation using CSS multi-column layout; limited control over item placement
- **JavaScript-calculated:** Precise control over column assignments; supports dynamic content and accurate gap management
- **Infinite scroll integration:** New items append to the layout as the user scrolls down
- **Skeleton loading:** Placeholder skeleton items shown while real content loads
- **Item transitions:** New items fade or slide in as they are added to the layout

Test Scenarios

- **Masonry renders:** Items display in a multi-column layout
- **Variable heights accommodated:** Items with different heights are laid out without uniform row alignment
- **Column count correct:** The correct number of columns renders for the given container width
- **Tight packing:** New items are placed in the shortest column, minimizing vertical gaps
- **No large empty gaps:** Layout does not leave large vertical gaps due to mismatched item heights
- **Responsive column change:** Reducing viewport/container width reduces the column count at the correct breakpoints
- **Re-layout on resize:** Layout recalculates correctly when the container is resized
- **Dynamic item addition:** Adding new items places them correctly in the layout without full re-render
- **DOM order preserved:** Tab order follows the DOM order of items, not visual column order
- **Focus visible:** Focus ring is clearly visible on interactive items
- **RTL columns:** In RTL, columns fill from right to left
- **Container-relative width:** Layout uses the container's width, not the viewport width, for column calculations
- **Gap configuration:** Column and row gaps render at the configured values

Notes

- **DOM order vs visual order:** The biggest accessibility concern with Masonry. Ensure the reading/tab order in the DOM is still meaningful even if it differs from left-to-right, top-to-bottom visual order
- **CSS columns vs JavaScript:** CSS `column-count` is simple but places items in column-first order (top of col 1, then top of col 2, etc.), which often conflicts with chronological/DOM order. JavaScript-calculated layouts give more control but are more complex

- **Layout thrashing:** JavaScript Masonry that reads item heights and writes positions repeatedly can cause layout thrashing. Batch reads before writes
- **Images and async content:** Items containing images or async-loaded content may resize after initial placement. Use `ResizeObserver` or a placeholder aspect ratio to handle late resizing
- **When to use:** Masonry is ideal for purely visual browsing (galleries). For structured content where order and comparison matter, a regular grid is better
- **Libraries:** `Masonry.js`, `react-masonry-css`, and CSS Grid with subgrid (where supported) are common implementation approaches

#62 Timeline

A vertical or horizontal sequence of events displayed in chronological order.

Description

Timeline visualizes a sequence of events, milestones, or steps ordered along a time axis. Each event is represented as a point (node) on the timeline with associated metadata—typically a date/time, title, and description. Timelines are used for activity logs, project histories, onboarding progress, order tracking, audit trails, and any narrative that benefits from temporal visualization. The key distinction from `Stepper` is that `Timeline` is a display component for historical or concurrent events, not an interactive guide through a process.

Main Features

- **Event nodes:** Visual markers (dots, icons, or badges) representing individual events on the timeline
- **Connector line:** A continuous line or track connecting all events in chronological sequence
- **Event metadata:** Each event displays at minimum a title; optionally a date/time, description, icon, and status
- **Chronological ordering:** Events ordered by time (ascending or descending)

- **Scrollable:** Long timelines scroll within their container
- **Status differentiation:** Events can have status variants (completed, active, upcoming, error) with corresponding visual treatment

Secondary Features

Accessibility

- **Semantic list structure:** Events rendered as an ordered list (`<ol>`) with each event as a `<li>`, communicating chronological sequence to screen readers
- **Date/time markup:** Date and time values use `<time>` element with machine-readable datetime attribute
- **Status communication:** Event status communicated via text or visually hidden label, not color alone
- **Connector line decorative:** The connector line between events is decorative (aria-hidden) and not part of the content structure
- **Heading structure:** If events have titles, they use appropriate heading levels within the list item context

Keyboard Navigation

- **Tab through events:** If events are interactive (clickable to expand or navigate), Tab moves between event nodes or their expand controls
- **Enter/Space to activate:** Activates the focused event (expands detail, navigates, etc.)
- **Focus visible:** Focus indicator is clearly visible on interactive event nodes

Touch-screen

- **Vertical scroll:** Long vertical timelines scroll natively with touch
- **Tap to expand:** If events have expandable detail, tapping the event node expands/collapses the detail
- **Touch targets:** Interactive event nodes meet 44×44px minimum tap target

Responsive Behavior

- **Vertical default:** Vertical orientation is the most common and works at any width
- **Condensed at narrow widths:** On narrow screens, event metadata may condense (hide descriptions, abbreviate dates)
- **Horizontal timeline on wide screens:** Horizontal orientation can be used on wide, fixed-height panels (e.g., project roadmaps)
- **Alternate sides:** In vertical timelines, events can alternate left/right of the connector line for a compact, symmetric layout

Internationalization

- **RTL connector side:** In RTL, the connector line and event node alignment mirror to the right side
- **Locale date formatting:** Dates/times displayed in locale-appropriate format using the `<time>` element
- **RTL text direction:** Event titles and descriptions use correct text direction in RTL
- **Localizable labels:** Status labels, date labels, and any action text are consumer-provided and fully localizable

Variants & Features

- **Vertical timeline:** Events stacked top-to-bottom with connector on the left (most common)
- **Horizontal timeline:** Events arranged left-to-right with connector across the top or bottom
- **Alternating timeline:** Events alternate left and right of the connector line
- **Compact timeline:** Minimal variant with smaller nodes and reduced metadata (suitable for sidebar activity feeds)
- **Expandable events:** Event detail hidden by default; expanding reveals full description or sub-events
- **Status icons:** Event nodes display contextual icons (checkmark for completed, clock for pending, warning for error)

- **Grouped by date:** Events grouped under date headings (e.g., “Today”, “Yesterday”, “March 2025”)

Test Scenarios

- **Timeline renders:** All events display in chronological order with connector line
- **Event nodes visible:** Each event has a visible node (dot, icon, or badge)
- **Event metadata displays:** Title, date/time, and description render for each event
- **Connector line present:** A visual line connects all event nodes in sequence
- **Status variants:** Events with different statuses render with correct visual treatment
- **Status not color-only:** Status is communicated via text or icon label, not color alone
- **Semantic list:** Events are rendered in an ordered `<ol>` list
- **Time element:** Date/time values use `<time>` with a `datetime` attribute
- **Connector decorative:** Connector line is aria-hidden and not announced by screen readers
- **Ascending order:** Events display earliest-first by default (or as configured)
- **Descending order:** When configured, events display latest-first
- **Expandable events:** If expandable, clicking/tapping an event shows its detail; clicking again collapses it
- **Tab through events:** Interactive event nodes are reachable via Tab in correct order
- **Locale date format:** Dates display in the configured locale format
- **RTL layout:** Connector line and event node positioning mirror correctly in RTL
- **Grouped events:** When grouping is enabled, date group headings appear above each day’s events
- **Scroll:** Long timelines scroll within their container without page overflow

Notes

- **Timeline vs Stepper:** Timeline is a display component for historical or parallel events. Stepper is an interactive guide through a sequential process the user is currently undergoing
- **Timeline vs Activity Feed:** Activity feeds (like social media) are similar but typically append at the top and have infinite scroll. Timeline implies a bounded, chronological view of a specific set of events
- **Accessibility and chronological order:** Because visual order and DOM order align in a timeline (unlike Masonry), accessibility is simpler—just ensure correct `<ol>` structure and `<time>` elements
- **Dense event handling:** When many events occur on the same date or time period, consider grouping them or using a compact node variant to avoid visual clutter
- **Real-time updates:** For live event streams (audit logs, order tracking), new events should append to the timeline without disrupting the user's scroll position

#63 Navigation / Menu Bar

The primary horizontal navigation component, typically at the top of the page. Contains links, dropdowns, and branding.

Description

Navigation Bar (also called Navbar, Menu Bar, or Header) is the primary navigation landmark of an application or website. It sits at the top of every page, provides access to the main sections of the product, and typically contains branding (logo), primary navigation links, optional mega-menu or dropdown navigation, and utility actions (search, user account, notifications). As a persistent, page-level component it carries significant structural importance: it is the first landmark screen reader users encounter and the anchor for all primary wayfinding.

Main Features

- **Branding area:** Logo or product name, typically linking to the home page
- **Primary navigation links:** Top-level links to main sections of the product
- **Active link indicator:** Visual and semantic indicator of the current section
- **Dropdown navigation:** Sub-navigation items accessible via hover or click on top-level items
- **Utility actions:** Secondary actions such as search, user avatar/menu, notifications, or settings
- **Mobile menu trigger:** On narrow viewports, a trigger (Icon Button) opens a mobile navigation panel (Drawer or overlay menu)
- **Sticky/fixed positioning:** Navbar remains visible at the top of the viewport as the user scrolls

Secondary Features

Accessibility

- **Navigation landmark:** Navbar is wrapped in a `<nav>` element with `aria-label="Main navigation"` to distinguish it from other nav landmarks
- **Active link:** Current section link has `aria-current="page"`
- **Dropdown triggers:** Dropdown-triggering links or buttons have `aria-expanded` and `aria-haspopup="true"`
- **Skip link:** A "Skip to main content" link appears before the Navbar, allowing keyboard users to bypass navigation
- **Focus management:** Dropdowns trap focus within the open menu; Escape closes the dropdown and returns focus to the trigger
- **Screen reader structure:** Links and menus are in a logical, sequential order matching their visual presentation

Keyboard Navigation

- **Tab through top-level items:** Tab moves focus through top-level links and buttons in order

- **Enter/Space to open dropdown:** Activates a dropdown trigger to open its sub-menu
- **Arrow keys in dropdown:** Down Arrow opens a dropdown from its trigger; Up/Down navigate within the open dropdown; Left/Right move between top-level items (closing the open dropdown)
- **Escape to close dropdown:** Closes the open dropdown and returns focus to its trigger
- **Home/End within dropdown:** Jump to first/last item in the open dropdown
- **Tab past Navbar:** Tab past the last Navbar item (or activation of the skip link) moves focus into the main content area

Touch-screen

- **Tap navigation links:** All top-level links and utility actions are tappable with adequate target size (44×44px)
- **Tap dropdown trigger:** On touch, dropdown triggers open on tap (not hover)
- **Mobile menu:** On narrow viewports, navigation collapses into a mobile menu triggered by an Icon Button (hamburger/close icon); the menu opens as a Drawer or overlay
- **Close on outside tap:** Open dropdowns or the mobile menu close when the user taps outside them

Responsive Behavior

- **Breakpoint collapse:** At a configured breakpoint, primary navigation links collapse and are replaced by a mobile menu trigger (hamburger button)
- **Mobile navigation:** Links previously in the horizontal Navbar are presented in a Drawer or full-screen overlay on mobile
- **Flexible utility area:** Utility actions may be reduced or reorganized on narrow viewports (e.g., hiding labels, showing only icons)
- **Logo scaling:** Branding area adapts gracefully to available space (logo may shrink or be replaced by a mark-only version)

- **Overflow handling:** If too many links exist for the available width, excess items collapse into a “More” dropdown before the full mobile breakpoint is reached

Internationalization

- **RTL layout:** In RTL, the branding area moves to the right, navigation links flow right to left, and dropdown arrow directions reverse
- **Localizable links:** All navigation labels, aria-labels, and utility action labels are consumer-provided and fully localizable
- **Bidirectional text:** Link labels handle RTL text correctly

Variants & Features

- **Transparent/overlay Navbar:** No background on hero sections; transitions to solid on scroll
- **Mega-menu:** A large dropdown panel containing grouped links, images, or rich content
- **Condensed/slim Navbar:** Reduced height on scroll (shrinks from tall to slim after a scroll threshold)
- **Multi-level navigation:** Nested dropdowns for deep site structures
- **Search integration:** Expandable search bar within the Navbar or as a full-width overlay
- **Notification badge:** Count badge on notification utility icon

Test Scenarios

- **Navbar renders:** Branding, navigation links, and utility actions display correctly
- **Skip link present:** A “Skip to main content” link is the first focusable element on the page
- **Skip link functional:** Activating the skip link moves focus to the main content landmark
- **Active link indicated:** Current section link has aria-current=“page” and correct visual state

- **Navigation landmark:** Navbar is inside a `<nav>` element with `aria-label="Main navigation"`
- **Dropdown opens:** Clicking/hovering a dropdown trigger opens its sub-menu
- **aria-expanded updates:** Dropdown trigger's `aria-expanded` is true when open, false when closed
- **Dropdown closes on outside click:** Clicking outside an open dropdown closes it
- **Escape closes dropdown:** Pressing Escape closes the open dropdown; focus returns to its trigger
- **Arrow key dropdown navigation:** Down Arrow opens dropdown from trigger; Up/Down navigate within it
- **Tab through top-level items:** Tab cycles through all top-level interactive elements
- **Mobile trigger visible:** At the breakpoint, hamburger/menu trigger appears; top-level links are hidden
- **Mobile menu opens:** Clicking the mobile trigger opens the navigation Drawer or overlay
- **Mobile menu closes:** Clicking the close button, backdrop, or pressing Escape closes the mobile menu
- **Logo links home:** Clicking the logo navigates to the home/root page
- **Utility actions functional:** Search, user avatar, notifications, and other utility controls fire correct callbacks
- **Sticky positioning:** Navbar remains visible at the top of the viewport when scrolling
- **RTL layout:** Branding, links, and dropdowns are correctly mirrored in RTL

Notes

- **Skip link is non-negotiable:** Every application with a Navbar must have a skip link. Keyboard users should not have to Tab through all navigation items on every page

- **Hover vs click for dropdowns:** Hover-open dropdowns are inaccessible to keyboard and touch users. Always support click/Enter to open, with hover as an enhancement only
- **Sticky Navbar and focus:** When the Navbar is sticky, ensure it does not obscure focused elements below it (use `scroll-margin-top` on landmark targets)
- **Mobile menu is a Drawer:** The mobile navigation menu is not a unique component—it is the Drawer component (Side or Bottom) configured with the navigation Menu content (see components 39 and 47)
- **Mega-menus:** Large mega-menus require careful focus management and must be closeable by Escape from any point within them; they function as modal-like overlays anchored to the Navbar
- **Logo alt text:** The logo image must have meaningful alt text (the product name), not an empty alt or “logo”
- **Performance:** The Navbar is present on every page. Keep its JavaScript and CSS lean; avoid heavy libraries that add to every page’s load cost

#64 Sidebar

A persistent or collapsible vertical navigation panel, typically on the left side of the interface.

***Naming note:** The Hamburger Menu is simply an Icon Button (1) that triggers the Sidebar open/close — it is not a separate component.*

Description

Sidebar is a vertical navigation panel that runs alongside the main content area, typically on the left side of the interface. It provides access to secondary navigation, section-level links, filters, or contextual tools. Unlike the Navbar (which is page-level and horizontal), Sidebar is typically application-level and vertical, and is common in dashboards, admin panels, documentation sites, and productivity tools. Sidebar may be persistently visible (always shown), collapsible (can be minimized

to an icon rail), or conditionally visible (hidden on mobile and triggered by a hamburger Icon Button).

Main Features

- **Vertical navigation list:** Primary container is a vertical Menu (component 39) of links and grouped sections
- **Active link indicator:** Visual and semantic indicator of the currently active section
- **Persistent or collapsible:** Configurable whether the Sidebar is always visible or can be collapsed/hidden
- **Collapsed icon rail:** When collapsed, Sidebar reduces to a narrow rail showing only icons with tooltips
- **Toggle control:** A trigger (typically an Icon Button) expands or collapses the Sidebar
- **Nested navigation:** Supports expandable sub-sections within the navigation structure

Secondary Features

Accessibility

- **Navigation landmark:** Sidebar is wrapped in a `<nav>` element with aria-label that distinguishes it from other nav landmarks (e.g., aria-label="Application navigation")
- **Active link:** Currently active link has aria-current="page"
- **Collapsed state:** When collapsed to icon rail, each icon has an aria-label providing the link name (since visible label is hidden)
- **Expanded state announcement:** When the Sidebar expands or collapses, the state change is communicated to screen readers (e.g., via aria-expanded on the toggle button)
- **Sub-navigation:** Expandable sections follow the same pattern as Collapsible —aria-expanded on the trigger, aria-controls on the content panel
- **Focus management:** Collapsing the Sidebar while focus is within it moves focus to the toggle button

Keyboard Navigation

- **Tab through links:** Tab moves through all navigation links and expandable section triggers
- **Arrow keys (optional):** Up/Down arrows can navigate between items within the Sidebar (following the Menu keyboard pattern)
- **Enter/Space to expand section:** Activates expandable section triggers to show or hide sub-navigation
- **Toggle keyboard:** The collapse/expand toggle button is reachable and operable via Tab, Enter, and Space
- **Focus retention on collapse:** Collapsing the Sidebar moves focus to the toggle button rather than losing it

Touch-screen

- **Touch navigation links:** All links and section triggers meet 44×44px tap target
- **Swipe to open/close (optional):** On mobile, swiping from the edge of the screen can open the Sidebar; swiping it away closes it
- **Tap toggle:** The hamburger/collapse Icon Button is large enough for reliable touch
- **Mobile Sidebar as Drawer:** On narrow viewports, the Sidebar renders as a Drawer (component 47) overlaying the content rather than pushing it

Responsive Behavior

- **Persistent on desktop:** Sidebar is always visible on wide viewports, occupying a fixed column next to main content
- **Collapsible to icon rail on medium viewports:** At a mid-range breakpoint, Sidebar collapses to a narrow icon-only rail
- **Hidden on mobile:** On narrow viewports, Sidebar is hidden by default and opens as a Drawer when triggered
- **Main content reflow:** When Sidebar expands or collapses in persistent mode, the main content area smoothly adjusts its width (push, not overlay)

- **Sidebar width:** Configurable; typically 200–280px expanded, 56–72px collapsed

Animation & Transitions

- **Expand/collapse animation:** Sidebar slides or transitions smoothly between expanded and collapsed states
- **Icon label fade:** Text labels fade in/out as the Sidebar expands/collapses
- **Reduced motion:** Transitions are instant or minimal when user prefers reduced motion
- **Content reflow:** Main content area width transitions smoothly to match the Sidebar's new width

Internationalization

- **RTL Sidebar position:** In RTL, Sidebar appears on the right side of the interface
- **RTL icon positions:** Icons within links appear on the correct side in RTL (typically closer to the reading start)
- **RTL collapse direction:** Collapse/expand animation direction mirrors in RTL
- **Localizable labels:** All navigation labels and aria-labels are consumer-provided and fully localizable

Variants & Features

- **Persistent:** Always visible; never collapses (common in dense admin panels with sufficient screen real estate)
- **Collapsible:** Can be toggled between full-width and icon rail (the most common variant for responsive dashboards)
- **Overlay:** On all viewport sizes, Sidebar overlays content (does not push it); useful when content area space is precious
- **Multi-level navigation:** Nested expandable sections for deep navigation hierarchies

- **Sidebar with footer:** Fixed footer within the Sidebar for account controls, settings link, or logout
- **Pinned/floating toggle:** Collapse/expand toggle button pinned to the Sidebar edge or inside the Sidebar header

Test Scenarios

- **Sidebar renders:** Navigation links, section headings, and icons display correctly
- **Navigation landmark:** Sidebar is inside a `<nav>` element with a distinct aria-label
- **Active link:** Currently active link has `aria-current="page"` and correct visual state
- **Link navigation:** Clicking a link navigates to the correct destination and updates the active state
- **Expand section:** Clicking an expandable section trigger shows its sub-navigation links
- **Collapse section:** Clicking an open section trigger hides its sub-navigation
- **aria-expanded on sections:** Expandable trigger's `aria-expanded` reflects open/closed state correctly
- **Toggle collapses Sidebar:** Clicking the toggle button collapses the Sidebar to icon rail
- **Toggle expands Sidebar:** Clicking the toggle again expands the Sidebar back to full width
- **Icon rail mode:** When collapsed, only icons display; text labels are hidden
- **Icon tooltips:** In collapsed mode, hovering or focusing an icon reveals a tooltip with the link label
- **aria-label on icons:** In collapsed mode, icon links have aria-labels with the full link name
- **Focus moves on collapse:** Collapsing while focus is inside the Sidebar moves focus to the toggle button
- **Main content refloWS:** In persistent mode, main content width adjusts when Sidebar expands/collapses

- **Mobile Drawer mode:** On narrow viewport, Sidebar opens as a Drawer overlay rather than pushing content
- **Slide animation:** Expand/collapse animates smoothly
- **Reduced motion:** Animation is instant when user prefers reduced motion
- **RTL layout:** Sidebar appears on the right side in RTL; icons and collapse direction mirror correctly
- **Sidebar footer:** When configured, footer content is pinned to the bottom of the Sidebar

Notes

- **Hamburger Menu is not a component:** The trigger that opens/closes the Sidebar is a standard Icon Button (component 16/17). There is no separate “Hamburger Menu” component to build
- **Sidebar vs Navbar:** Navbar is horizontal, page-level, and universally present. Sidebar is vertical, application-level, and often specific to authenticated or dashboard contexts
- **Sidebar vs Drawer:** Sidebar is structural navigation that may push content. Drawer is a temporary panel that overlays content (see component 47). On mobile, Sidebar renders using the Drawer pattern
- **Collapsed icon rail accessibility:** In icon-only mode, every icon must have an accessible label. Visible tooltips on hover are an enhancement—the aria-label on the link is the requirement
- **Sticky within scroll:** The Sidebar’s content list may itself need to scroll if it contains many items. Use a Scroll Area (component 34) within the Sidebar rather than letting the whole page scroll
- **Active state source:** Like Menu, the active link state should be driven by the router or parent (current route), not managed internally by the Sidebar

#65 Dock

Mobile primary navigation bar fixed at the bottom of the screen with 3–5 main destinations. Also called *Bottom Navigation* or *Tab Bar* in mobile design systems.

Description

Dock is the primary navigation component for mobile applications. It is a fixed bar at the bottom of the viewport containing 3–5 icon-and-label navigation items, each representing a top-level destination in the application. Because it is always visible and positioned within thumb reach, it provides fast, one-tap access to the most important sections of the app. Dock is analogous to the Navbar for mobile contexts—it replaces (or supplements) a Navbar on small screens and is always present across all pages of the application.

Main Features

- **Fixed bottom positioning:** Always visible at the bottom of the viewport regardless of scroll position
- **3–5 navigation items:** Each item contains an icon and a short label representing a top-level destination
- **Active item indicator:** Visual highlight and semantic indicator on the currently active destination
- **Navigation callbacks:** Tapping an item fires a navigation callback or follows a link
- **Safe area awareness:** Bottom padding accounts for device safe areas (iOS home indicator, Android gesture bar)
- **Icon badges:** Optional count or dot badges on items to indicate unread counts or alerts

Secondary Features

Accessibility

- **Navigation landmark:** Dock is wrapped in a `<nav>` element with `aria-label="Main navigation"` (or a distinct label if a Navbar is also present on the same page)
- **Active item:** The active destination item has `aria-current="page"`
- **Icon labels:** Every item has a visible text label; icons alone are never the sole identifier (avoids ambiguity and supports accessibility)
- **Descriptive aria-labels:** If icon + label is not sufficiently descriptive, items have `aria-label` providing additional context
- **Badge announcement:** Badge counts are included in the accessible name or described via `aria-label` (e.g., "Messages, 3 unread")

Keyboard Navigation

- **Tab through items:** Tab moves focus through each Dock item in order
- **Arrow keys (optional):** Left/Right arrows can navigate between items when focus is within the Dock
- **Enter/Space to activate:** Activates the focused item (navigates to its destination)
- **Focus visible:** Focus indicator is clearly visible on each item against the Dock background

Touch-screen

- **Large touch targets:** Each item's tap target spans the full height of the Dock and an equal portion of the width (minimum 44×44px, ideally larger)
- **No hover dependency:** All interactions are tap-based; no hover states required
- **Immediate feedback:** Active state updates immediately on tap, before navigation completes
- **No swipe conflicts:** Dock does not intercept vertical swipe gestures used for system navigation (pull-to-refresh, back gesture)

Responsive Behavior

- **Mobile-only by default:** Dock is typically shown only on narrow viewports; on desktop, a Navbar or Sidebar takes the navigational role
- **Full-width:** Dock spans the full viewport width
- **Safe area inset:** Bottom padding uses `env(safe-area-inset-bottom)` (or platform equivalent) to avoid overlap with system UI
- **Breakpoint hiding:** At a configured breakpoint (e.g., above 768px), Dock hides and the desktop navigation pattern takes over
- **Item count constraint:** Limited to 3–5 items; more destinations must be accessed via a “More” item that opens a secondary menu (or via a Drawer)

Internationalization

- **RTL item order:** In RTL, Dock items flow from right to left
- **Localizable labels:** All item labels and aria-labels are consumer-provided and fully localizable
- **Short label constraint:** Labels must be short (1–2 words); translations must respect this constraint to avoid overflow

Variants & Features

- **With badge:** Items display a count badge (number) or dot badge for unread/alert states
- **Floating Dock:** Dock rendered as a floating pill above the system bar rather than edge-to-edge (common in some design systems)
- **With “More” item:** When the application has more than 5 top-level destinations, a “More” item opens a secondary navigation sheet or Drawer listing the remaining destinations
- **Icon-only mode:** On very narrow screens or when labels are too long to display, items show icon only (use sparingly; always include aria-label)
- **Contextual highlighting:** Active item icon changes style (filled vs outlined, different weight) in addition to color change

Test Scenarios

- **Dock renders:** All items display with icons and labels at the bottom of the viewport
- **Fixed positioning:** Dock remains visible at the bottom of the viewport when content scrolls
- **Safe area padding:** Dock does not overlap the system home indicator or gesture bar
- **Active item highlighted:** The current destination item has correct visual state and `aria-current="page"`
- **Tap navigates:** Tapping an item fires the navigation callback or follows the link
- **Active state updates:** Active indicator moves to the newly tapped item immediately on tap
- **Navigation landmark:** Dock is inside a `<nav>` element with an accessible label
- **Icon labels visible:** Text labels are visible beneath each icon
- **Badge renders:** When configured, badge (count or dot) displays on the correct item
- **Badge accessible:** Badge count is included in the item's accessible name (e.g., "Messages, 3 unread")
- **Tab through items:** All items are reachable via Tab in correct order
- **Focus visible:** Focus ring is clearly visible on each item
- **Arrow key navigation:** When configured, Left/Right arrows move focus between items
- **Item count constraint:** Dock does not render fewer than 3 or more than 5 items without a "More" overflow item
- **"More" item:** When present, tapping "More" opens a secondary navigation sheet with additional destinations
- **Breakpoint hiding:** Above the configured breakpoint, Dock is hidden and desktop navigation is shown
- **RTL item order:** Items flow right to left in RTL layouts
- **Icon-only fallback:** In icon-only mode, each item has an `aria-label` with the full destination name

Notes

- **3–5 items is a firm constraint:** Dock items must be immediately recognizable top-level destinations. More than 5 creates crowding and ambiguity. If the application has more destinations, use a “More” item that opens a Drawer or secondary sheet
- **Label text length:** Labels should be 1–2 short words. Long translations will overflow; work with localization teams to keep all labels within a character budget (typically 10–12 characters max)
- **Dock vs Navbar on mobile:** Many applications use both—a Navbar for branding and utility actions (search, profile) at the top, and a Dock for primary destination switching at the bottom. Ensure their nav landmarks have distinct aria-labels to avoid confusion for screen reader users
- **Dock vs Tabs:** Dock is the primary application navigation. Tabs (component 53) switch content panels within a page. Do not use Dock as a tab bar for in-page content switching
- **Safe area is non-negotiable:** Failing to account for `env(safe-area-inset-bottom)` on iOS results in Dock content being hidden behind the home indicator. Test on physical devices
- **Active state on navigation:** The active item should reflect the current page/route, driven externally (by the router), not internal state. When the URL changes without a Dock tap (e.g., deep link, back button), the Dock active state must update accordingly
- **Haptic feedback:** On native mobile platforms, a subtle haptic tap on Dock item activation reinforces the interaction. On web, this is not available by default but can be implemented via the Vibration API where supported

3.5 Component Summary Checklist

The table below brings the full component list together in one place — each entry cross-referenced by functional category and complexity tier. It’s useful in a few different ways: as a planning tool when deciding which components to build first, as a progress tracker during an active build, or as a quick reference when specifying a subset of components to a development team or an AI system. Not every project needs every component — use this list to make deliberate choices about scope rather than defaulting to “build everything”.

#	Component	Category	Tier
1	Icon	Content & Typography	Basic
2	Typography	Content & Typography	Basic
3	KBD (Keyboard Key)	Content & Typography	Basic
4	Code Snippet	Content & Typography	Basic
5	Separator / Divider	Layout & Structure	Basic
6	Spacer	Layout & Structure	Basic
7	Aspect Ratio	Layout & Structure	Basic
8	Image	Content & Typography	Basic
9	Avatar	Identity & Status Indicators	Basic
10	Badge	Identity & Status Indicators	Basic
11	Chip / Tag	Identity & Status Indicators	Basic
12	Skeleton	Identity & Status Indicators	Basic
13	Loader	Identity & Status Indicators	Basic
14	Color Swatch	Content & Typography	Basic
15	Link	Actions & Navigation Primitives	Basic

#	Component	Category	Tier
16	Button	Actions & Navigation Primitives	Basic
17	Label	Form Inputs & Controls	Intermediate
18	Form Field	Form Inputs & Controls	Intermediate
19	Text Input	Form Inputs & Controls	Intermediate
20	Password Input	Form Inputs & Controls	Intermediate
21	PIN / OTP Input	Form Inputs & Controls	Intermediate
22	Textarea	Form Inputs & Controls	Intermediate
23	Checkbox	Form Inputs & Controls	Intermediate
24	Radio Button	Form Inputs & Controls	Intermediate
25	Switch	Form Inputs & Controls	Intermediate
26	Segmented Control	Form Inputs & Controls	Intermediate
27	Range Slider	Form Inputs & Controls	Intermediate
28	Rating	Form Inputs & Controls	Intermediate
29	File Uploader	Form Inputs & Controls	Intermediate
30	Color Picker	Form Inputs & Controls	Intermediate
31	Card	Layout & Structure	Intermediate
32	Alert / Banner	Feedback & Communication	Intermediate
33	Progress	Feedback & Communication	Intermediate
34	Scroll Area	Layout & Structure	Intermediate
35	Collapsible	Layout & Structure	Intermediate
36	Resizable	Layout & Structure	Intermediate
37	Empty State	Feedback & Communication	Intermediate
38	Breadcrumb	Navigation & Wayfinding	Intermediate
39	Menu	Navigation & Wayfinding	Intermediate

#	Component	Category	Tier
40	Select	Form Inputs & Controls	Advanced
41	Combobox / Autocomplete	Form Inputs & Controls	Advanced
42	Dropdown Menu	Overlays & Floating Elements	Advanced
43	Tooltip	Feedback & Communication	Advanced
44	Popover	Overlays & Floating Elements	Advanced
45	Toast / Notification	Feedback & Communication	Advanced
46	Modal / Dialog	Overlays & Floating Elements	Advanced
47	Drawer	Overlays & Floating Elements	Advanced
48	Adaptive Menu	Adaptive Elements	Advanced
49	Date Picker	Form Inputs & Controls	Advanced
50	Time Picker	Form Inputs & Controls	Advanced
51	DateTime Picker	Form Inputs & Controls	Advanced
52	Calendar	Data Display & Visualization	Advanced
53	Tabs	Navigation & Wayfinding	Advanced
54	Accordion	Layout & Structure	Advanced
55	Stepper	Navigation & Wayfinding	Advanced
56	Pagination	Navigation & Wayfinding	Advanced
57	Table	Data Display & Visualization	Advanced
58	Sortable List	Data Display & Visualization	Advanced
59	Transfer List	Data Display & Visualization	Advanced
60	Carousel	Data Display & Visualization	Advanced
61	Masonry	Data Display & Visualization	Advanced
62	Timeline	Data Display & Visualization	Advanced
63	Navigation / Menu Bar	Navigation & Wayfinding	Advanced

#	Component	Category	Tier
64	Sidebar	Navigation & Wayfinding	Advanced
65	Dock	Navigation & Wayfinding	Advanced

4 Building Your Web UI System: Step-by-Step Guide

This guide provides a structured approach to building a complete web UI component library. Each step builds on the previous ones, so follow them in order for best results. All tooling recommendations, code examples, and implementation notes in this guide are web-specific — if you are adapting this guide for a non-web platform, substitute the platform equivalents for each step.

The steps described here reflect a common, proven sequence for building a UI system — but they are not a rigid prescription. Every team, project, and organization brings its own constraints: existing tooling, in-progress migrations, partial design systems, or components already in production. Treat this guide as a reference sequence that you adapt to your reality, not a checklist to follow mechanically. Some teams will start at Step 5 because their tokens already exist. Others will loop back from Step 9 to Step 7 when testing reveals a foundational gap. That's not failure — it's how real systems get built.

Here is an overview of all eleven steps before we go into each one in detail:

1. **Define Your Color Palette** — Establish primary, secondary, semantic, neutral, and state colors with documented accessibility verification
2. **Define Your Typography System** — Set font families, type scale, weights, and line heights as reusable tokens
3. **Define Spacing and Layout System** — Create a base unit, spacing scale, z-index layer system, and responsive breakpoints
4. **Define Look & Feel Decisions** — Decide how states (focus, hover, error, disabled) and feedback patterns will be expressed visually
5. **Set Up Development Environment** — Configure tooling for building, testing, and documenting components in isolation
6. **Implement Design Tokens in Code** — Translate design decisions into CSS Custom Properties, JS/TS constants, or token files

7. **Build Foundational Components** — Implement atomic, dependency-free components that everything else is built on
8. **Build Compound Components** — Assemble more complex components by composing foundational ones
9. **Ensure Test Coverage and Quality** — Write behavioral, accessibility, and visual regression tests across all components
10. **Create Documentation and Usage Guidelines** — Document every component's purpose, API, examples, and accessibility notes
11. **Gather Feedback and Iterate** — Deploy to real projects, measure adoption, and improve continuously

One final recommendation before diving in: share this process with your whole team. UI systems succeed when they are understood and owned collectively — not just by the engineers who build them, but by the designers who specify them, the product managers who prioritize them, and the stakeholders who depend on them. A team that understands why the system is built the way it is will make better decisions when the specification doesn't have a ready answer, and will maintain it with more care than one that treats it as someone else's responsibility.

4.1 Step 1: Define Your Color Palette

Why This Matters

Color is one of the most fundamental design decisions. It establishes your brand identity, creates visual hierarchy, and conveys meaning. A well-designed color palette ensures consistency and accessibility across your entire UI.

What to Define

- **Primary colors** (1-2 colors): Your brand's main colors, used for primary actions, links, and key UI elements
- **Secondary colors** (0-2 colors): Supporting colors for variety and to emphasize less critical elements

- **Semantic colors**
 - Success (green spectrum)
 - Error/Danger (red spectrum)
 - Warning (yellow/orange spectrum)
 - Info (blue spectrum)
- **Neutral colors** (5-10 shades): Grayscale from white to black for text, backgrounds, borders, and dividers
- **State colors**: Specific colors for hover, focus, active, and disabled states

Considerations

Color Theory

- Use color triads, complementary, or analogous schemes for visual harmony
- Ensure brand colors work well together and don't clash
- Test your palette in both light and dark contexts

Accessibility

- **Contrast ratios** must meet WCAG 2.2 standards (Campbell et al., 2023):
 - Text: Minimum 4.5:1 (AA) or 7:1 (AAA) for normal text (SC 1.4.3)
 - Large text: Minimum 3:1 (AA) or 4.5:1 (AAA) (SC 1.4.3)
 - UI components: Minimum 3:1 for interactive elements (SC 1.4.11)
- Never rely on color alone to convey information (use icons, text labels, or patterns too)
- Test with color blindness simulators (protanopia, deuteranopia, tritanopia)

Technical Implementation

- Create color scales (50, 100, 200...900) for each primary/secondary color for flexibility
- Document color usage (when to use primary-500 vs primary-600)
- Consider how colors will look on different backgrounds

Resources

- Color palette generators: Coolers, Adobe Color, Paletton

- Contrast checkers: WebAIM Contrast Checker, Contrast Ratio
- Color blindness simulators: Coblis, Color Oracle

Output: A documented color palette with hex/RGB values, usage guidelines, and accessibility verification

4.2 Step 2: Define Your Typography System

Why This Matters

Typography affects readability, hierarchy, and the overall personality of your UI. A consistent type system makes content scannable and creates visual rhythm.

What to Define

- **Font families:**
 - Primary (body text): Highly readable, works at various sizes
 - Secondary (headings): Can be more distinctive, complements primary
 - Monospace (code): Fixed-width for technical content
- **Type scale:** Defined set of font sizes (e.g., 12, 14, 16, 18, 20, 24, 32, 40, 48, 64px)
- **Font weights:** Available weights (e.g., 300-light, 400-regular, 500-medium, 600-semibold, 700-bold)
- **Line heights:** Corresponding line height for each size (typically 1.2-1.8)
- **Letter spacing:** Adjustments for headings, small text, or specific use cases

Considerations

Readability

- Base body text should be **at least 16px** (never smaller)
- Line height should be 1.5-1.6 for body text (better readability)
- Line length should be 45-75 characters for optimal reading
- Adequate letter spacing improves legibility, especially for uppercase text

Hierarchy

- Clear size distinction between heading levels (H1 should be noticeably larger than H2, etc.)
- Use weight and size together to create emphasis
- Consistent spacing above/below headings

Performance

- Limit font file loading (each weight/style is a separate network request)
- Use system fonts or carefully chosen web fonts
- Consider variable fonts for flexibility with a single file

Accessibility

- Avoid thin weights (under 400) for body text
- Don't use all-caps for long text (harder to read)
- Ensure sufficient contrast with background (see Step 1)

Resources

- Font pairing tools: FontPair, Typewolf, Google Fonts
- Type scale calculators: Type Scale, Modular Scale
- Web font services: Google Fonts, Adobe Fonts, Fonts.com

Output: A documented typography scale with font families, sizes, weights, line heights, and usage examples

4.3 Step 3: Define Spacing and Layout System

Why This Matters

Consistent spacing creates visual rhythm, improves scanability, and makes your UI feel cohesive. A spacing system prevents arbitrary pixel values and ensures predictable layouts.

What to Define

- **Base unit:** Foundation for all spacing (typically 4px or 8px)
- **Spacing scale:** Consistent increments based on base unit
 - Example with 4px base: 4, 8, 12, 16, 20, 24, 32, 40, 48, 64, 80, 96px
 - Example with 8px base: 8, 16, 24, 32, 40, 48, 64, 80, 96px
- **Layout grid:** Optional grid system (12-column is common)
- **Container max-widths:** Maximum widths for different breakpoints
- **Border radius values:** Defined set (e.g., 0, 2, 4, 8, 16, 9999 for pills)
- **Shadow/elevation levels:** 3-5 levels of shadow for depth

Z-Index Organization (Layering System)

Without a systematic approach to z-index, you'll end up with arbitrary values (z-index: 9999!) and components fighting for visual hierarchy. A layering system prevents these issues.

Suggested Z-Index Scale

Define discrete layers for different UI elements.

Name	Z-Index Value	Description
base	0	Default page content
dropdown	1000	Dropdowns, popovers, tooltips
sticky	1020	Sticky headers, fixed navigation
overlay	1030	Modal backdrops
modal	1040	Modal dialogs
popover	1050	Elements that appear over modals, like alerts
toast	1060	Toast notifications, snackbars

Organization Principles

- **Use discrete jumps** (100s or 1000s) between layers to allow sub-layering if needed

- **Group related elements** at similar z-index levels
- **Document the scale** so all developers use the same values
- **Avoid arbitrary values:** Use the defined scale, don't make up numbers
- **Layer stacking within groups:** If multiple dropdowns can be open, ensure proper stacking

Common Scenarios

Dropdown over content:

- Page content: z-index 0
- Dropdown: z-index 1000

Modal over page with dropdown still visible:

- Page content: z-index 0
- Dropdown: z-index 1000 (but hidden/closed when modal opens)
- Modal backdrop: z-index 1030
- Modal: z-index 1040

Toast notification over everything:

- All previous layers remain
- Toast: z-index 1060 (visible over modal if needed)

Implementation*

Define z-index as CSS custom properties or JS/TS constants:

z-index:

```
base: 0
dropdown: 1000
sticky: 1020
overlay: 1030
modal: 1040
popover: 1050
toast: 1060
```

Note: These are suggested values. Adjust based on your needs, but maintain the principle of discrete, documented layers rather than arbitrary values.

Other Considerations

Touch Targets

- Minimum interactive element size: **44×44px** (Apple Inc., n.d.) or **48×48dp** (Google LLC, n.d.)
- This applies to buttons, links, checkboxes, radio buttons, etc.
- Ensure adequate spacing between adjacent touch targets

Visual Rhythm

- Use consistent spacing between related elements (tighter)
- Use larger spacing between unrelated sections (looser)
- Vertical rhythm: maintain consistent spacing down the page

Responsive Spacing

- Consider scaling spacing at different breakpoints
- Smaller spacing on mobile, more generous on desktop
- Maintain proportional relationships

Shadows and Depth

- Use shadows to indicate elevation and interactivity
- Subtle shadows for cards, more prominent for modals/dropdowns
- Consistent shadow direction (typically from above)

Output: Documented spacing scale, shadow system, border radius values, and layout guidelines

4.4 Step 4: Define Look & Feel Decisions

Why This Matters

This step determines how your design tokens will be applied to create specific visual patterns. These decisions affect consistency across all components.

What to Define

Visual State Indicators

- **Focus state:** How should focused elements look?
 - Border change? (color, thickness)
 - Outline/ring? (color, offset, thickness)
 - Background change?
 - Combination of above?
- **Hover state:** What changes on hover?
 - Background color shift?
 - Border color change?
 - Elevation/shadow increase?
 - Opacity change?
- **Active/pressed state:** What indicates an element is being clicked?
 - Darker background?
 - Inset shadow?
 - Scale down slightly?
- **Disabled state:** How to show non-interactive elements?
 - Reduced opacity (typically 0.5-0.6)?
 - Grayscale?
 - Different background color?

Feedback Patterns

- **Error indication:** How are errors shown?
 - Red border around input?
 - Red background with darker text?
 - Red icon with border change?
 - Error message position (below input? tooltip?)
- **Success indication:** How is success shown?
 - Green border?
 - Green checkmark icon?

- Success message display?
- **Loading indication:** How to show loading states?
 - Spinner style and position?
 - Skeleton screens?
 - Progress bars?

Component Styling Decisions

- **Button styles:**
 - Filled vs outlined vs text-only variants?
 - Border radius (rounded, pill-shaped, square)?
 - Padding and height?
- **Input fields:**
 - Border all around or only bottom?
 - Background color (white, gray, transparent)?
 - Label position (floating, above, inline)?
- **Cards/containers:**
 - Border or shadow or both?
 - Background color contrast with page?
 - Padding inside cards?

Animation/Motion

- Transition durations (fast: 100-150ms, normal: 200-300ms, slow: 400-500ms)
- Easing functions (ease-in, ease-out, ease-in-out, custom curves)
- What gets animated? (color, transform, opacity, all?)
- Respect prefers-reduced-motion for accessibility

Considerations

Consistency

- Apply the same visual treatment to the same states across ALL components
- If text-input has a blue focus ring, dropdown should too
- If buttons use a shadow on hover, cards should too (if applicable)

Accessibility

- Focus indicators must be clearly visible (minimum 3:1 contrast ratio; Campbell et al., 2023, SC 1.4.11)
- Don't rely solely on color (use icons, borders, patterns)
- Animations should be optional (respect user preferences)

User Expectations

- Follow platform conventions where they exist
- Users expect blue for links, red for errors, green for success
- Disabled elements should clearly look non-interactive

Output: A style guide documenting visual patterns for states, feedback, and common UI elements with examples

4.5 Step 5: Set Up Development Environment

Why This Matters

Before building components, you need a proper web development environment with tooling that supports testing, documentation, and iteration.

What to Set Up

Core Web Development Tools

- Package manager: npm, yarn, or pnpm
- Build tool: Vite, Webpack, Rollup, or your framework's built-in tooling
- TypeScript (recommended): Type safety prevents bugs and improves developer experience
- Linter: ESLint with an accessibility plugin (e.g., eslint-plugin-jsx-a11y for React, eslint-plugin-vuejs-accessibility for Vue)
- Formatter: Prettier for consistent code style

Testing Infrastructure

- Unit testing: Jest or Vitest (framework-agnostic; works with React, Vue, Svelte, etc.)
- Component testing: Framework-specific utilities (React Testing Library, Vue Test Utils, Svelte Testing Library, etc.)
- Accessibility testing: axe-core, jest-axe, or similar automated a11y tooling
- Visual regression testing (optional): Chromatic, Percy, or Playwright for catching unintended visual changes

Component Development Environment

- Storybook (recommended): Build and document components in isolation, independent of the consuming application
- Hot reloading: Instant browser feedback during development
- Component playground: Test different states and prop combinations interactively

Documentation

- README with setup instructions and contribution guidelines
- Component API documentation (props, events, slots)
- Usage examples and copy-paste code snippets
- Accessibility notes per component
- Changelog with clear descriptions of what's added or changed

Considerations

- Start simple — add tools and complexity only as genuine needs arise
- Ensure all team members can run the project locally without friction
- Automate what you can: linting, formatting, and testing should all run automatically in CI/CD

Output: A working web development environment where components can be built, tested, and documented

4.6 Step 6: Implement Design Tokens in Code

Why This Matters

Design tokens from Steps 1–3 need to be translated into code that web components can consume. This creates the styling foundation that every component will build on.

What to Implement

Token Format

Choose a format appropriate for your web stack:

- CSS Custom Properties (recommended for web): `--color-primary-500`, `--spacing-4` — works in any framework or vanilla JS
- Tailwind CSS config: `colors.primary[500]`, `spacing[4]` defined in `tailwind.config.js` — Tailwind consumes these directly and generates utility classes; pair with CSS Custom Properties for runtime theming
- JavaScript/TypeScript objects: `colors.primary[500]`, `spacing[4]` — useful for CSS-in-JS solutions
- Sass/SCSS variables: `$color-primary-500`, `$spacing-4` — good for Sass-based systems
- Design token files: JSON or YAML source-of-truth files that generate platform outputs via tools like Style Dictionary

Token Categories to Implement

- Color palette (all colors from Step 1)
- Typography (font families, sizes, weights, line heights)
- Spacing scale
- Border radius values
- Shadow definitions
- Breakpoints
- Animation durations and easing

Token Organization

```
tokens/  
├─ colors.ts  
├─ typography.ts  
├─ spacing.ts  
├─ shadows.ts  
├─ borders.ts  
└─ index.ts (exports all)
```

Considerations

Naming Conventions

- Be consistent and descriptive
- Use semantic names where possible: `color-error` instead of `color-red`
- Organize hierarchically: `color.primary.500` or `color-primary-500`

Accessibility

- Include accessible color combinations (pre-validated contrast ratios)
- Document which color combinations are safe to use together

Flexibility

- Allow theming (light/dark mode, brand variations) via CSS Custom Properties or equivalent
- Make tokens easy to override for customization

Documentation

- Export token documentation (auto-generate if possible)
- Show visual examples of each token (color swatches, type specimens)

Output: Code-based design tokens that can be imported and used by components across the web stack

4.7 Step 7: Build Foundational Components

Why This Matters

Start with the simplest, most atomic components that other components will depend on. These are the building blocks of your UI system.

What to Build First

Essential Foundational Components

1. **Button**: Various variants (primary, secondary, text, icon)
2. **Text Input**: As specified in the Component Catalog
3. **Label**: Standalone label component (used by inputs)
4. **Icon**: Icon wrapper/system
5. **Typography components**: Heading, Text, Code
6. **Layout components**: Container, Stack (vertical/horizontal), Grid
7. **Divider**: Horizontal/vertical separators

For Each Component

- Implement all **Main Features** from the Component Catalog spec
- Implement relevant **Secondary Features** based on your requirements
- Write **Test Scenarios** from the Component Catalog
- Document usage with examples
- Ensure accessibility features are included

Development Order

1. Start with **Typography components** (Heading, Text) — used everywhere
2. Build **Layout components** (Container, Stack) — needed for composition
3. Implement **Button** — most common interactive element
4. Build **Text Input** — fundamental form control
5. Add **Label** — supports form inputs
6. Create **Icon system** — visual enhancement

7. Add **Divider** — organizational element

Considerations

Quality Over Quantity

- Build fewer components well rather than many components poorly
- Ensure each component is production-ready before moving on
- Get feedback and iterate on foundational components

Testing

- Write tests as you build (not after)
- Test both visual appearance and behavior
- Include accessibility tests (keyboard navigation, screen reader support)
- Aim for high coverage on foundational components (80%+ recommended)

Documentation

- Document each component immediately after building
- Include code examples for common use cases
- Note any browser-specific behaviors or limitations
- Document accessibility features and keyboard interactions

Output: A set of well-tested, documented foundational components

4.8 Step 8: Build Compound Components

Why This Matters

With foundational components in place, you can now build more complex components that combine or extend the basics.

What to Build

Intermediate Complexity

1. **Dropdown Menu** (as specified in the Component Catalog)
2. **Checkbox** and **Radio Button**
3. **Toggle/Switch**
4. **Textarea** (multi-line text input)
5. **Card**: Container for related content
6. **Badge/Tag**: Small status indicators
7. **Tooltip**: Contextual help on hover/focus

Higher Complexity

1. **Modal** (as specified in the Component Catalog)
2. **Tabs**: Tabbed navigation
3. **Accordion**: Expandable/collapsible sections
4. **Table**: Data table with sorting/filtering
5. **Pagination**: Navigate through lists
6. **Toast/Notification**: Temporary messages
7. **Date Picker**: Calendar-based date selection

Development Approach

Reuse Foundational Components

- Build on what you've already created
- Modal uses Button for actions
- Dropdown uses foundational text and icons
- Card uses layout components internally

Progressive Enhancement

- Start with basic functionality
- Add advanced features incrementally
- Test thoroughly at each stage

Component Composition

- Favor composition over complex monolithic components

- Example: `Modal.Header`, `Modal.Body`, `Modal.Footer` over one giant `Modal`
- Makes components more flexible and easier to test independently

Considerations

Specialized Variants

- Create variants by extending base components
- Example: `email-input` extends `text-input`
- Example: `confirmation-modal` extends `modal`
- Document the relationship and what the variant adds

Cross-Component Consistency

- Ensure similar behaviors across components
- All overlays (dropdown, modal, tooltip) should handle focus similarly
- All form controls should handle error states the same way

Output: A comprehensive component library with both simple and complex components

4.9 Step 9: Ensure Test Coverage and Quality

Why This Matters

Comprehensive testing prevents regressions, ensures accessibility, and gives you confidence to iterate and refactor.

What to Test

Unit Tests (Component Behavior)

- All test scenarios from the Component Catalog
- Props/parameters work correctly
- Events/callbacks fire as expected

- State changes behave correctly
- Edge cases and error conditions

Accessibility Tests

- Keyboard navigation works
- Focus management is correct
- ARIA attributes are present and correct
- Color contrast meets standards
- Screen reader compatibility (manual testing with NVDA, JAWS, or VoiceOver strongly recommended)

Visual Regression Tests (Optional)

- Components render consistently across browsers
- No unintended visual changes from code updates
- Works across supported browsers and devices

Integration Tests

- Components work together correctly
- Forms submit properly
- Multi-step flows work end-to-end

Quality Targets

Code Coverage

- Aim for **80%+ coverage** on foundational components
- Aim for **70%+ coverage** on compound components
- 100% is ideal but can have diminishing returns
- Focus on critical paths and edge cases

Accessibility

- **WCAG AA compliance** as minimum (AAA preferred where feasible)
- Test with actual assistive technologies — automated tools catch roughly 30-40% of issues

- Use automated tools (axe, Lighthouse) as a first pass, not the only pass

Browser/Device Support

- Define supported browsers and versions explicitly before you start
- Test on actual devices, not just emulators
- Consider progressive enhancement for older browser support

Tools

- Testing frameworks: Jest, Vitest, Playwright
- Accessibility: axe-core, WAVE, NVDA/JAWS/VoiceOver screen readers
- Visual testing: Chromatic, Percy, Playwright screenshots
- CI/CD: Run tests automatically on every commit

Considerations

Test Maintenance

- Keep tests simple and focused
- Avoid brittle tests (don't test implementation details, test behavior)
- Update tests when requirements change
- Delete obsolete tests

Documentation of Testing

- Document how to run tests locally
- Explain the testing philosophy and approach
- Note any manual testing requirements (especially accessibility)

Output: Comprehensive test suite with good coverage and automated CI/CD integration

4.10 Step 10: Create Documentation and Usage Guidelines

Why This Matters

Even the best components are useless if developers don't know how to use them correctly. Great documentation accelerates adoption and reduces support burden.

What to Document

For Each Component

- Purpose: What is this component for?
- When to use: Appropriate use cases
- When NOT to use: Common misuses or alternatives
- Props/API: All parameters, types, defaults
- Examples: Common usage patterns with code
- Accessibility: Keyboard shortcuts, ARIA usage, screen reader behavior
- Variants: Different versions and when to use each
- Customization: How to extend or customize styling

General Documentation

- Getting started guide: Installation, setup, first component
- Design token documentation: Colors, typography, spacing with visual examples
- Theming guide: How to customize the library
- Migration guide: If upgrading from another library
- Contribution guide: How to add or modify components
- Accessibility guidelines: General principles and standards followed
- Browser support matrix: What browsers and versions are supported and tested

Documentation Formats

Interactive Documentation

- Storybook: Component playground with live controls
- Live code examples: Editable examples users can experiment with directly
- Visual examples: Screenshots or live renders of each variant

Written Documentation

- Markdown files: Easy to maintain and version-control friendly
- API reference: Auto-generated from code comments (JSDoc, TSDoc)
- README files: Quick reference in each component directory

Design Resources

- Figma/Sketch files: Design tokens and components for designers
- Style guide: Visual examples of patterns and principles
- Component checklist: Reference guide for building new components

Considerations

Keep Documentation Updated

- Update docs when component behavior changes
- Review documentation as part of the code review process
- Auto-generate what you can (API docs from code)

Make It Searchable

- Good navigation and search functionality
- Clear categorization and tags
- Cross-references between related components

Examples Over Explanation

- Show code examples for common use cases
- “Copy-paste ready” examples when possible
- Visual examples alongside code

Accessibility Documentation

- Document keyboard interactions clearly
- Explain ARIA usage and why it's there
- Note any known accessibility limitations

Output: Comprehensive, searchable documentation that helps developers use your component library effectively

4.11 Step 11: Gather Feedback and Iterate

Why This Matters

Real-world usage will reveal issues, missing features, and improvement opportunities you didn't anticipate. Iteration based on feedback is crucial for a successful component library.

How to Gather Feedback

Internal Testing

- Use the library in real projects within your team or organization
- Track issues and feature requests systematically
- Note pain points and confusing areas
- Measure adoption and usage patterns

User Feedback

- Conduct usability testing with developers who will use the library
- Survey users about their experience
- Monitor support channels for common questions
- Track GitHub issues or bug reports

Analytics and Metrics

- Component usage frequency (which components are most used?)

- Bundle size impact
- Performance metrics (render time, load time)
- Accessibility audit results

What to Iterate On

Immediate Fixes

- Critical bugs affecting functionality
- Accessibility violations
- Security issues
- Breaking changes in dependencies

High-Value Improvements

- Frequently requested features
- Components with low adoption (why aren't they being used?)
- Documentation gaps (frequently asked questions)
- Performance optimizations

Long-Term Enhancements

- New components based on common patterns
- Advanced features for power users
- Better theming and customization options
- Broader browser and device support

Iteration Process

1. Collect: Gather feedback from all sources
2. Prioritize: Rank by impact and effort
3. Plan: Create roadmap for improvements
4. Implement: Build, test, document
5. Release: Version appropriately (semantic versioning)
6. Communicate: Changelog, migration guides, announcements
7. Repeat: Continuous improvement cycle

Considerations

Versioning

- Use semantic versioning (major.minor.patch)
- Maintain a changelog with all changes
- Provide migration guides for breaking changes
- Consider long-term support for major versions

Component-Level Versioning

When a single component needs significant changes that might break existing implementations, you have several strategies:

Strategy 1: Major Version Bump (Recommended for library-wide changes)

- Increment the entire library version (v2.0.0)
- Update all components to new patterns
- Provide migration guide
- **When to use:** When the change affects the philosophy or patterns across multiple components

Strategy 2: New Component Variant

- Create a specialized variant: `TextInput` → `EnhancedTextInput`
- Keep original component for backward compatibility
- **When to use:** When adding optional advanced features that not all users need
- **Example:** `TextInput` (basic) + `RichTextInput` (with formatting toolbar)

Strategy 3: Versioned Component Names

- Create explicitly versioned component: `TextInputV2`
- Deprecate old version with clear timeline
- **When to use:** When making breaking changes to a single popular component
- **Pros:** Clear migration path, both versions can coexist
- **Cons:** Naming clutter, requires eventual cleanup

Strategy 4: Feature Flags/Props

- Add new behavior behind a prop: `<TextInput enhanced={true} />`
- Default to old behavior for backward compatibility
- **When to use:** When new behavior can be toggled
- **Pros:** Smooth migration, users control adoption
- **Cons:** Increased component complexity, technical debt

Decision Framework

Breaking change affects many components? → **Major version bump** (v2.0.0)

Adding optional advanced features? → **New variant component**

(EnhancedTextInput)

Breaking change to single component, users need time to migrate? → **Versioned name** (TextInputV2) with deprecation timeline

Change can be controlled with a prop? → **Feature flag prop** with sensible default

Deprecation Best Practices

- Announce deprecation clearly in documentation and release notes
- Show deprecation warnings in development (console.warn, TypeScript deprecation comments)
- Provide migration guide with code examples
- Give users adequate time (at least 2-3 major versions or 6+ months)
- Remove deprecated code only in major versions

Example Deprecation Timeline*

```
v2.0.0: Introduce TextInputV2, mark TextInput as deprecated
v2.1.0: Continue supporting both, remind users to migrate
v2.2.0: Continue supporting both
v3.0.0: Remove TextInput (breaking change), TextInputV2 becomes
TextInput
```

Backward Compatibility

- Avoid breaking changes when possible
- Deprecate features before removing them
- Provide clear upgrade paths

Community Building

- Encourage contributions (if open source)
- Recognize and thank contributors
- Build a community around your library

Output: A living, evolving component library that improves based on real-world usage and feedback

5 Conclusion

Building a UI component library is an iterative journey, not a one-time project. By following this Step-by-Step Guide and using the behavioral specifications in the Component Catalog, you can create a consistent, accessible, and maintainable web UI system tailored to your needs.

The specifications in this document represent patterns that have been refined through years of web platform evolution, accessibility research, and accumulated UX knowledge. They are not arbitrary rules — each behavioral requirement exists because real users, including those who rely on keyboards, screen readers, or other assistive technologies, depend on components working in predictable, well-understood ways. Treating these specifications as a floor rather than a ceiling is the right approach: meet them without compromise, then build on top of them with your own creativity and product vision.

One of the most important things this specification can do is save you from rediscovering known solutions. The keyboard navigation patterns, ARIA roles, focus management strategies, and interaction behaviors documented here are the result of standards work and community experience — not something you need to derive from scratch. Use that head start. Spend your team's energy on the problems unique to your product, not on relitigating what a modal's Escape key behavior should be.

A few suggestions as you move forward:

Start narrow, go deep. It's far more valuable to have ten well-specified, thoroughly tested, fully accessible components than fifty rough ones. Foundational components like Button, Text Input, and Modal are used everywhere — if they have subtle bugs or accessibility gaps, those defects propagate across your entire application. Invest the time to get them right before expanding the catalog.

Treat accessibility as architecture, not polish. Accessibility is significantly harder to retrofit than to build in from the start. The ARIA patterns, keyboard behaviors, and focus management strategies in this specification should be baked

into each component from day one. Automated tools like axe-core will catch some issues, but plan for manual testing with real screen readers (NVDA, JAWS, VoiceOver) before calling any component production-ready.

Document decisions, not just APIs. Future contributors — and your future self — will benefit enormously from knowing *why* a component behaves a certain way, not just *what* it does. The Notes sections in each component specification model this habit: capture the reasoning behind non-obvious choices alongside the implementation details.

Gather feedback early and often. The developers who consume your library will surface problems you never anticipated. A component that feels complete in isolation may reveal missing props, confusing defaults, or composition gaps the moment it meets a real product page. Build internal feedback loops early — a Slack channel, a GitHub issue template, a short periodic survey — so that signal reaches you consistently.

Remember:

- Start with solid foundations (design tokens and core principles before components)
- Build incrementally, testing and documenting as you go
- Prioritize accessibility and user experience as first-class requirements, not afterthoughts
- Gather feedback and iterate continuously — no component library is ever truly “done”
- Don’t rebuild everything from scratch — use this specification to make conscious, informed decisions about what to build and why

Whether you’re building a new web UI library from scratch, migrating an existing system to a new stack, or using this specification to generate components with AI assistance, the goal is always the same: components that are predictable, accessible, and genuinely good to use. This specification gives you the behavioral blueprint. The craftsmanship is yours.

6 References

Standards & Guidelines

Campbell, A., Adams, C., Bradley Montgomery, R., Cooper, M., & Kirkpatrick, A. (Eds.). (2023, October 5). *Web Content Accessibility Guidelines (WCAG) 2.2* (W3C Recommendation). World Wide Web Consortium (W3C). <https://www.w3.org/TR/WCAG22/>

W3C Web Accessibility Initiative (WAI). (n.d.). *ARIA Authoring Practices Guide (APG)*. World Wide Web Consortium (W3C). Retrieved February 2026, from <https://www.w3.org/WAI/ARIA/apg/>

Design Systems & Best Practices

Nielsen, J. (1994, April 24). *10 usability heuristics for user interface design* (Updated January 30, 2024). Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>

Apple Inc. (n.d.). *Human Interface Guidelines*. Apple Developer Documentation. Retrieved February 2026, from <https://developer.apple.com/design/human-interface-guidelines/>

Google LLC. (n.d.). *Material Design 3*. Retrieved February 2026, from <https://m3.material.io/>

